

iViewNG™ SDK

v2.0.1

December 2013

Contents

1 iViewNG™ Software Development Kit **1**

1.1 Introduction 1

2 Installation **4**

2.1 Microsoft Windows 4

3 Concepts **5**

3.1 Synchronous Communication 6

3.2 Asynchronous Communication 7

4 Tutorial **11**

4.1 Using C / C++ 11

4.2 Using C# 38

4.3 Using Python 38

4.4 Using Matlab® 38

5 Frequently Asked Questions **39**

5.1 Frequently Asked Questions 39

6 Licenses **41**

6.1 SMI Licenses 41

6.2 3rd Party Licenses 43

7 Function Documentation **59**

7.1 iViewNG-BasicDatatypes.h File Reference 59

7.2 iViewNG-Calibration.h File Reference 61

7.3 iViewNG-Connection.h File Reference 71

7.4 iViewNG-Core.h File Reference 77

7.5 iViewNG-DataAcquisition.h File Reference 95
7.6 iViewNG-DataStreams.h File Reference 103
7.7 iViewNG-Device-ETG.h File Reference 128
7.8 iViewNG-Device-Validation.h File Reference 136
7.9 iViewNG-Device.h File Reference 140
7.10 iViewNG-Stimulus.h File Reference 147
7.11 iViewNG-Utility.h File Reference 156
7.12 iViewNG.h File Reference 161
7.13 D:/Research/iViewNG/source/iViewNG-SDK/documentation/iViewNG-SDK-Documentation.dox File Reference 164

Index **164**

Chapter 1

iViewNG™ Software Development Kit

1.1 Introduction

This is the documentation of the iViewNG™ Software Development Kit ("SDK"). It was developed to fit the next generation of iView and is used to communicate with devices which are controlled by an iViewNG server.

The iViewNG™ SDK provides an Application Programming Interface ("API") for communication between your C/C++ software application and iViewNG™, allowing you to create eye tracking applications that take advantage of the experience and expertise provided by SensoMotoric Instruments ("SMI") eye tracking devices and the iViewNG™ platform. Specifically, the API was designed for SMI customers who wish to add eye tracking into their own custom applications. Using the functions provided in the API you can control SMI eye tracking devices and retrieve eye tracking data online. The API uses shared memory, TCP and UDP communication to provide maximum speed and a configurable balance between latency and data rate for data transfer.

Bundled with the API are code examples, documentation files and extensions. Together, these modules form the SDK.

The following programming and scripting languages are supported with this release:

Language	Support
C / C++	native
C#	available soon
Python	available soon
Matlab®	available soon

This SDK Guide provides instructions for setting up your SDK environment, a descriptive overview about the used techniques, a tutorial introduction you to developing applications using the SDK and a detailed reference about all SDK functions.

- Installation
- Concepts
- Tutorial

- Frequently Asked Questions

In addition to this document, the SDK includes release notes, which may be found in the `iViewNG-SDK/documentation` directory. The release notes provide a complete list of changes.

Updates are available at the SMI Software Downloads page: http://www.smivision.com/en/gaze-and-eye-tracking-systems/support/software-download.html.

Requirements

Eye Tracker

To use the SDK, you need to have an SMI eye tracking device. The following eye tracking devices are supported with this release:

Eye tracker	Support
ETG	full control

Please note that the availability of devices may depend on the license key you use.

Computer Hardware

The required system performance depends on the use case. In general, use an *Intel Core i7* class processor with approx. 3 GHz and a system with 2 GByte memory.

Operating System

The following operation systems are supported with this release:

System	Support
Windows 7 32 bit	full support
Windows 7 64 bit	supported as 32bit application
Android	available soon

Technical Support

Please request support through the SMI website http://www.smivision.com/en/gaze-and-eye-tracking-sys
html Please note that if you should require technical assistance relating to the SDK and your application, SMI may request or require a copy of your application and elements of your source code. The use of the SDK requires profound programming knowledge. Please note that SMI cannot provide general programming support.

About SMI

SensoMotoric Instruments (SMI) is a world leader in dedicated computer vision applications, developing and marketing eye & gaze tracking systems and OEM solutions for a wide range of applications.

© 2013 SensoMotoric Instruments GmbH

Founded in 1991 as a spin-off from academic research, SMI was the first company to offer a commercial, vision-based 3D eye tracking solution. We now have 20 years of experience in developing application-specific solutions in close collaboration with our clients. We serve our customers around the globe from our offices in Teltow, near Berlin, Germany and Boston, USA, backed by a network of trusted local partners in many countries. Our products combine a maximum of performance and usability with the highest possible quality, resulting in high-value solutions for our customers. Our major fields of expertise are:

- Eye & gaze tracking systems in research and industry
- High speed image processing, and
- Eye tracking and registration solutions in ophthalmology.

More than 5,000 of our systems installed worldwide are testimony to our continuing success in providing innovative products and outstanding services to the market. While SMI has won several awards, the largest reward for us each year is our trusted business relationships with academia and industry.

Please contact us:

Europe, Asia, Africa, South America, Australia
SensoMotoric Instruments GmbH (SMI)
Warthestr. 21
D-14513 Teltow
Germany
Phone: +49 3328 3955 0
Fax: +49 3328 3955 99
Email: info@smi.de

North American Headquarters
SensoMotoric Instruments, Inc.
28 Atlantic Avenue
236 Lewis Wharf
Boston, MA 02110
USA
Phone: +1 - 617 - 557 - 0010
Fax: +1 - 617 - 507 - 83 19
Toll-Free: 888 SMI USA1
Email: info@smivision.com

Please also visit our home page: http://www.smivision.com.

Copyright © 2013 SensoMotoric Instruments GmbH

Chapter 2

Installation

The SDK package consists of a folder containing pre-compiled libraries and examples, a set of header files needed to use the SDK functions and a folder with documentation.

2.1 Microsoft Windows

The SDK comes as installer MSI package. Please execute the installer.

The libraries compiled for Microsoft Windows are built for Window 7 and newer, for 32 bit systems. Although you can use the iViewNG™ SDK binaries on a Windows 64-bit OS, please note that it is not possible to implement the binaries in Windows 64-bit applications.

Chapter 3

Concepts

The iViewNG API enables a client application to communicate with an iViewNG server which controls one or more eye tracking devices. Please note that devices which are operated by an iView X server are not able to provide this communication scheme. They must be controlled using the iView X API 3.x.

The API provides transparent communication to the iViewNG server process running either on the same host as the client program or on a remote host. The communication utilizes the Internet Protocol (IP), more specifically both communication protocols TCP and UDP, as well as shared memory communication. If required, the client application is in full control of the used communication technique. Otherwise, the API will use default settings.

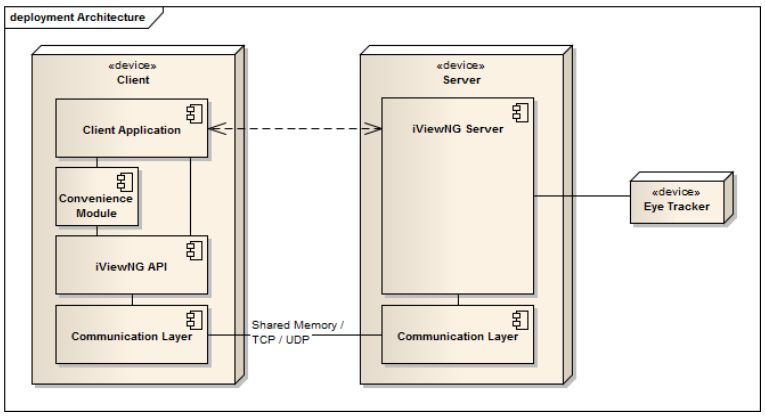


Figure 3.1: The iViewNG SDK architecture.

While the iViewNG API offers the core communication to control an eye tracking device and provides access to the acquired and calculated data, higher level operations are part of *Addon Modules*. The functions in those modules use API like a client application to realize convenient operations. Examples for such operations are:

- synchronous communication,
- automatic calibration,
- validation,

- display of various monitors.

The implementation of those functions may be used as example on how to use the API.

3.1 Synchronous Communication

As part of the *Convenience Modules*, the functions which are most often used have synchronous counterpart. For example, while the asynchronous function to start a recording is iView_StartRecording(iViewTicket *), the synchronous counterpart is iView_Sync_StartRecording(). With the asynchronous function, the control flow will immediately return to the caller without waiting for the operation to complete, the synchronous function will block the caller until either condition is met:

- the command was executed by the server and acknowledged or
- the command was not acknowledged within some time: the API user may control the timeout for the synchronous commands by assigning the number of milliseconds to wait to the variable g_syncFunctionsTimeout.

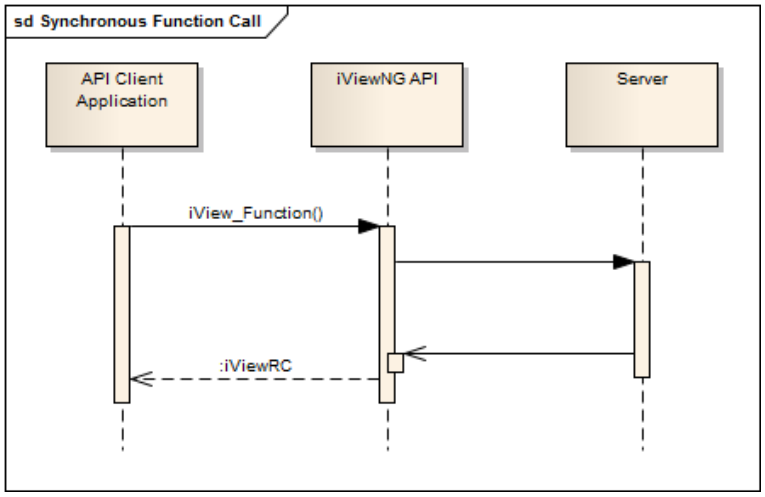


Figure 3.2: Synchronous function calls

The result will be returned to the caller. For synchronous functions, there is no ticketing mechanism.

Please note that the synchronous functions offer limited control as drawback of the reduced complexity. Since the implementation of the synchronous functions is open to the API user, it is easy to modify or extend the behaviour and even to mix synchronous and asynchronous functions.

Please refer to the tutorial for examples on how to use the synchronous functions.

3.2 Asynchronous Communication

All iViewNG API functions which invoke a server command operate *asynchronously*. That means, the API function does not block the client program until the command is processed by the server. Instead, the API backend returns the control flow to the caller immediately after briefly checking the command request and forwarding the request to the iViewNG server. A return code indicates if this brief check was successful: all command functions check if the API was initialized properly, if a connection to the server is available and if the command parameters seem to be plausible.

If you want your commands to behave like *synchronous* calls, please have a look at the *Convenience Modules* which are available as source code. There are functions which provide the functionality to block the control flow until a request is answered.

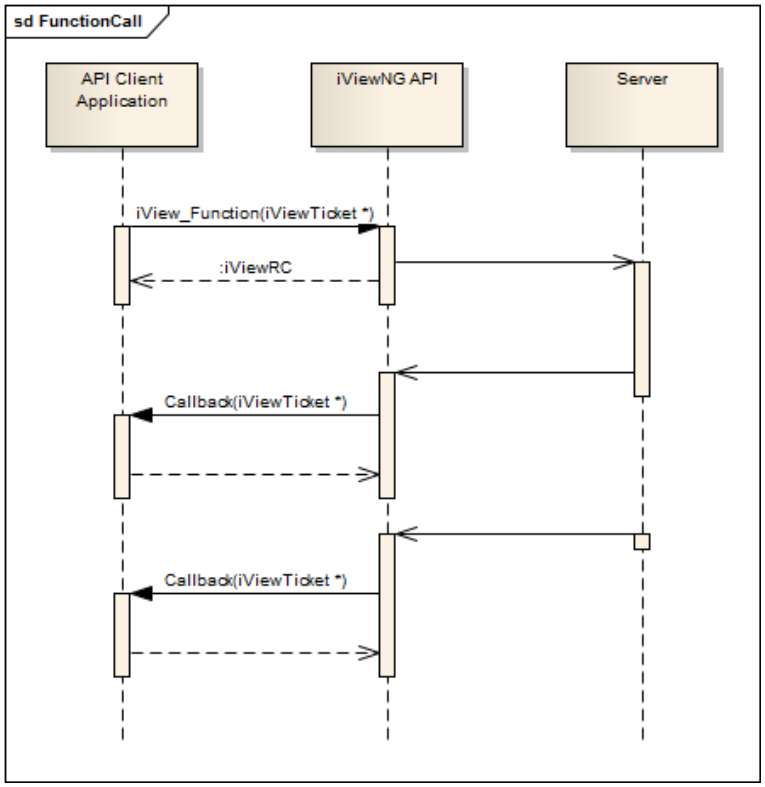


Figure 3.3: Asynchronous function calls

As soon as the asynchronous command is processed by the iViewNG server, a message event is sent to the API client by executing a callback function implemented by the API user. This event includes the result of the command. For some commands, the callback will be called several times with each event providing a new result, e.g. for data streams.

All API functions are **thread-safe**. Command functions use mutual exclusions to prevent other threads to operate until the first called function completed.

Please refer to the `tutorial` for examples on how to use the asynchronous functions.

The iViewTicket data structure

The ticket mechanism offer a way to associate an incoming message event with the command which the event belongs to. If the API user calls an asynchronous command function, a ticket must be provided as well. Whenever the iViewNG server passes a message event to the user callback, the ticket which created the command is provided as parameter of the callback.

When a command is completed, iViewNG updates iViewTicket.state, iViewTicket.progress, iViewTicket.-returnCode and in some cases iViewTicket.result. Thus, it is easily possible to find out which command created the incoming event as well as determining the processing state.

There are multiple ways to associate the incoming message event with the command which is answered:

1. comparing the ticket's address received by the callback with the adresses of all created tickets,
2. comparing the (unique) ticket's number with the numbers of all created tickets,
3. comparing the ticket result type which corresponds with the API command.

Please see the `tutorial` for example implementations.

The ticket data structure contains the following attributes:

- iViewTicket.ticketNumber: a running number which is assigned to the ticket structure by iView_-CreateTicket(). There will never be more than one ticket with this ticket number.
- iViewTicket.functionName: a string describing the function call which this ticket was used for, assigned to the ticket structure by iView_CreateTicket(). There may be more than one ticket with this function name. Use this only for debugging purposes.
- iViewTicket.state: the processing state of the command. The caller may at any time check if the request was already answered by examining the state indicator. If the server updates the state, the callback function will be called to announce the change.
- iViewTicket.progress: a progress indicator to let the user have an idea of how much of the command is already finished. Not all commands provide information about the progress, see the functions documentation. If the server updates the progress, the callback function will be called to announce the change.
- iViewTicket.returnCode: a return code to indicate the (intermediate) result of the command. Defaults to RC_NO_ERROR. A value different to RC_NO_ERROR indicates that iViewTicket.result points to an iViewServerError data structure if not NULL. Otherwise, iViewTicket.result points to a iViewResult data structure if not NULL. See functions documentations for details.
- iViewTicket.result: points to a data structure which includes the result data of the operation. This may be an iViewServerError structure if the command raised an error or an iViewResult structure. Use iViewTicket.returnCode to determine if the result is an error or not and use typecasts.

The caller of the API functions is responsible for explicitly indicating to release all data structures. iViewNG will never release the iViewTicket structure if not ordered explicitly by the API user. The result is released only if:

- the API user orders iViewNG to release a ticket's result and uses the function iView_Release-Result(),
- the API user releases the ticket structure by using the function iView_ReleaseTicket(),
- the API user terminates the API by using iView_Shutdown() or
- a new data entity replaces the old result, which happens only with data streams.

Never try to release a ticket without calling iView_ReleaseTicket()! Never write into the ticket data structure!

The callback function

Whenever some data is submitted asynchronously by the iViewNG server, the callback function is called. The callback function is a function the user must implement and register to the API. There is only one callback function for all incoming types of data.

See this example for an example callback function:

```
void MyCallback (iViewTicket * const ticket) {

    // this should never happen
    if (NULL == ticket)
        return;

    printf("Received ticket #%u with return code %u.\n", ticket->ticketNumber
      , ticket->returnCode);

    return;
}
```

Using the function iView_SetCallback(), this user-defined function is registered to the API. There is only one callback call active at the same time. Should it happen that a new message event arrives while the callback function is still busy with processing the last one, the new message event will be queued and wait until the callback function is left. This is implemented **thread-safe**.

Therefore, it is important to keep the time spent inside the callback function as short as possible. For example, if you expect data to arrive with an average sample rate of 100 Hz, the time spent inside the callback function must not be larger or close to 10 milliseconds. Otherwise, the queue size would increase over time. For a tight timing, it is most essential to let the callback finish as fast as possible.

Please note, while the client application is busy with executing the callback function, no other API calls or anwers will be processed. It is not possible to use any API command function but iView_ReleaseResult() from inside the callback since this would result in a deadlock!

Handling data streams

Data streams are used to submit a large number of data samples to the API client. For example, gaze samples are delivered periodically with approximately the eye camera's sample rate. The gaze samples are a data stream. Once the client application subscribes to this stream, the user-defined callback is called once for each incoming gaze sample.

The API client may customize the behaviour of the data stream using *stream specifiers*:

- Using the stream specifier iViewDataStreamSpecSampleRate the client application may reduce the sample rate. This means that the client application will not receive more than the specified amount of samples per second. This behaviour can be specified for each data stream individually.
- Using the stream specifier iViewDataStreamSpecQualityOfService the client application may request a dedicated communication channel quality. Currently, there are two qualities available:
  - IVIEWQOS_LOSSY: Lossy communication is realized by using UDP communication which is lean and has only little overhead. As result, there is only a minimal lag from network communication, but data may be lost.
  - IVIEWQOS_LOSSLESS: Lossless communication is realized by using TCP communication by adding some protocol overhead. As result, data transmitted over TCP is not lost unless the connection is not lost at all, but there may be a lag as result of possible retransmission of data.

    This behaviour can be specified for each data stream individually.

For every data sample, the same ticket structure is used. With each call, the ticket's result pointer iViewTicket.result will point to a new iViewResult structure which contains the new sample. Subsequent message events overwrite the predecessing result. If a result is not released explicitly with iView_Result(), iViewNG will release the result data before creating a new result and storing the address in the ticket.

In case that server-generated data must exist outside the callback, e.g. image data which is passed to a user interface for display, it is required to copy the data inside the callback function.

Never try to release result data without calling iView_ReleaseTicket() or iView_ReleaseResult()! Never write into the result data structure!

Chapter 4

Tutorial

This section explains the usage of the iViewNG API by example, starting with very easy programs leading to a simple remote viewer application. All examples are available for all supported programming languages.

Please note that the example source code is included as part of the iViewNG SDK package.

4.1 Using C / C++

Step 1: Checking the compiler

Goal of this example is to make sure that your compiler tool chain is set up correctly, that the iViewNG SDK include files are found, that the iViewNG API library is found and that your runtime environment is able to execute your first program which uses the iViewNG SDK.

The source code of the example discussed in this section can be found in the SDK inside the folder *examples/C/Tutorial-GetLibraryVersion*.

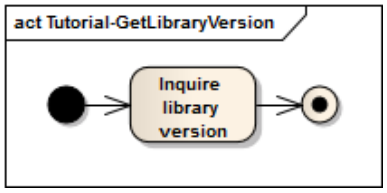


Figure 4.1: Control flow

To check if the compiler and linker are up and running, it is recommended to compile the example program *main.c* which calls a simple API function.

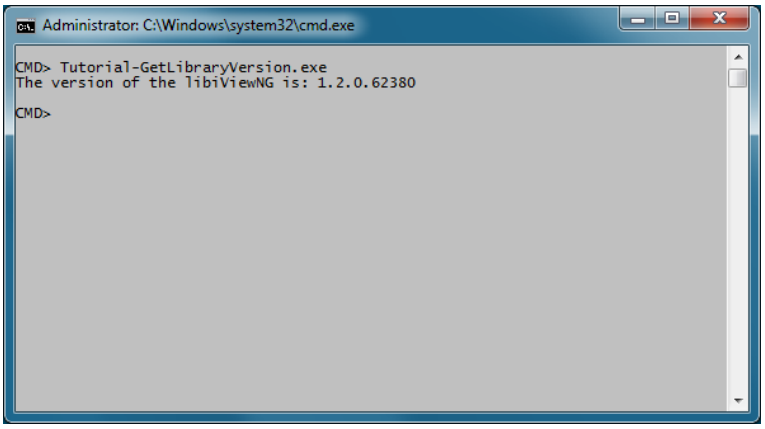


Figure 4.2: Executing the example.

Microsoft Visual Studio

Users of Microsoft Visual Studio find solution files for all available examples in a dedicated folder, e.g. `examples/C/Tutorial-GetLibraryVersion/Tutorial-GetLibraryVersion-Visual-Studio`.

The project files were created for Visual Studio 2008 on Windows 7 / 32bit but will work with Visual Studio 2010 as well.

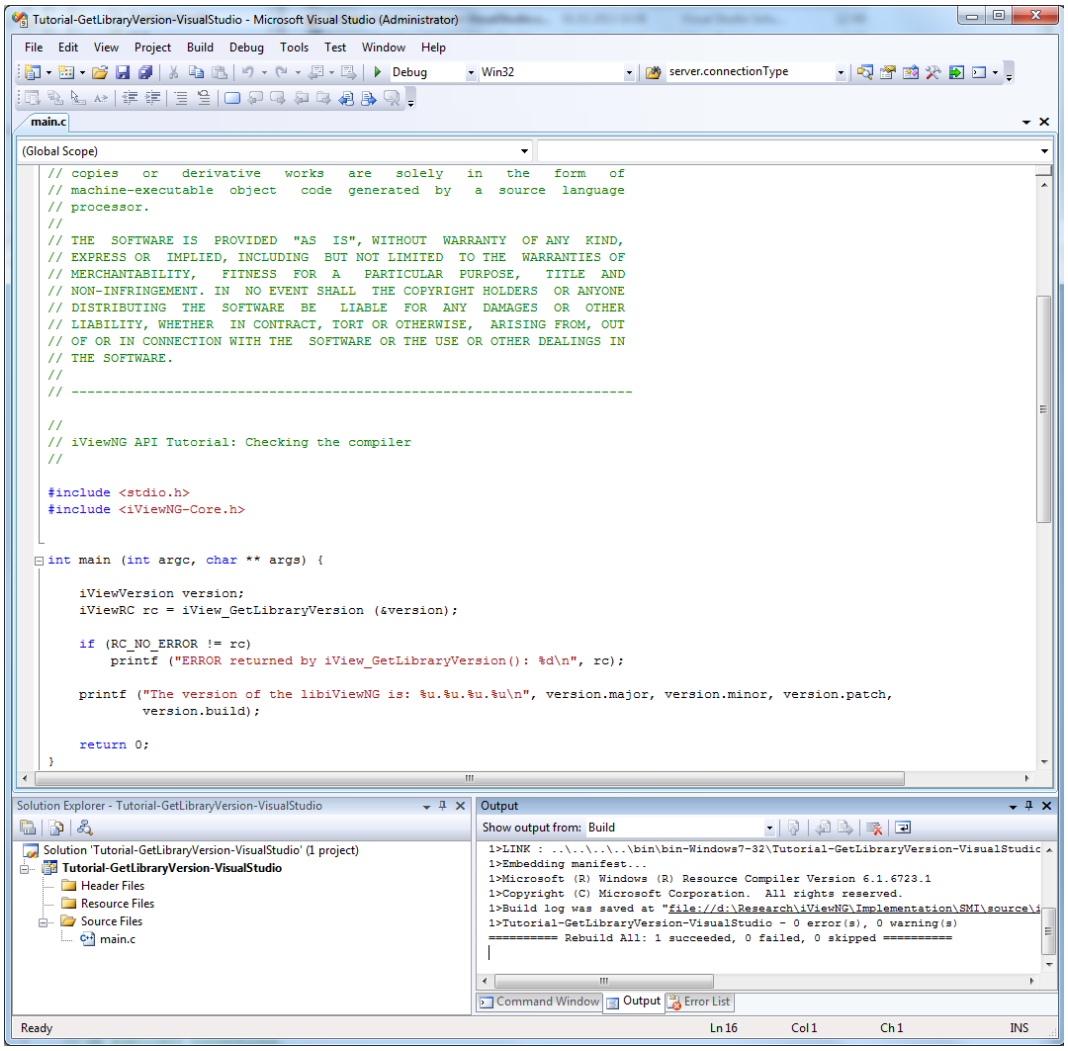


Figure 4.3: Compiling an example with Visual Studio 2008

Please note, when using Visual Studio 2010 or creating new projects, assert that the compiler mode is set to **Compile as C++ Code (/TP)** as shown below. Please note that there is a difference between the selection **Compile as C++ Code (/TP)** being displayed in **boldface** (good):

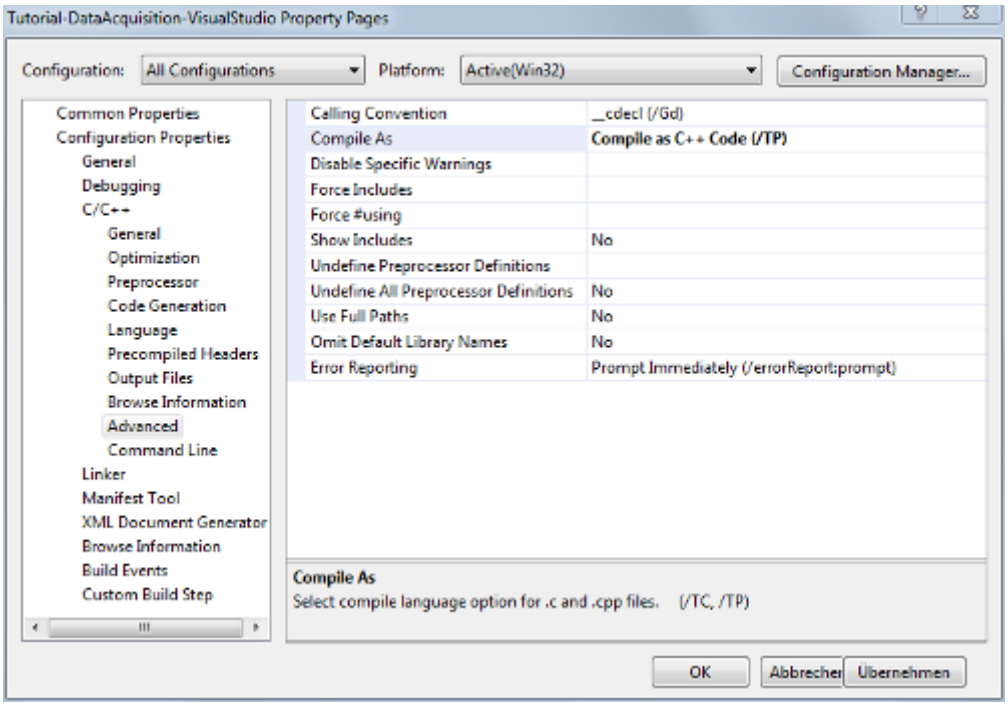


Figure 4.4: width=8cm

and being displayed in **not boldface** (not good):

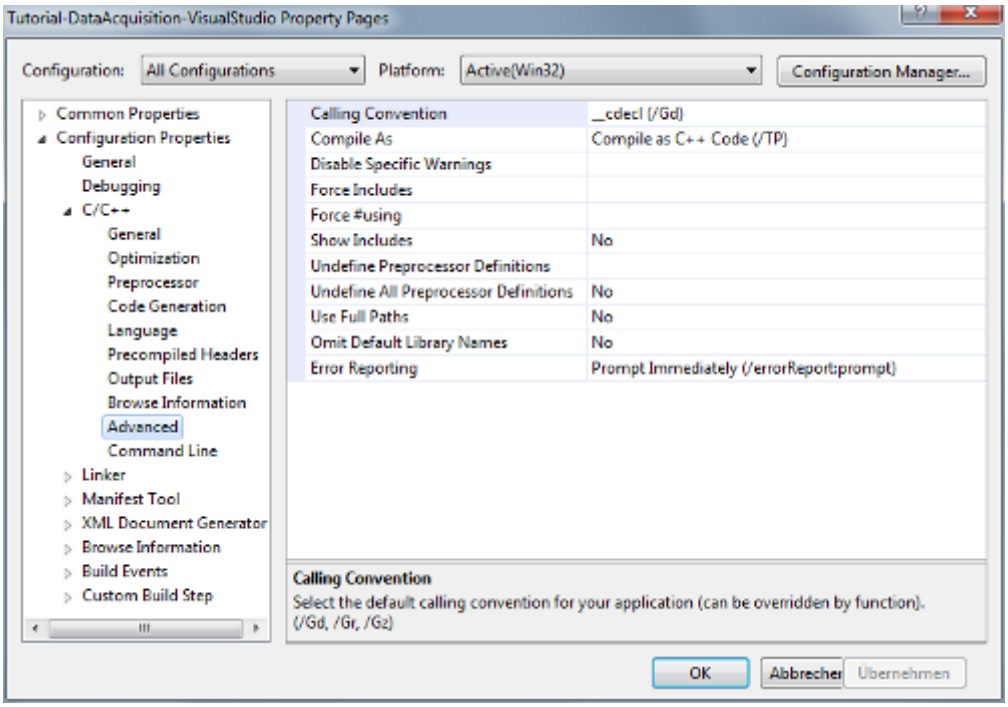


Figure 4.5: width=8cm

GNU C/C++

Users of the GNU C/C++ compiler find a *Makefile* in the example folders, e.g. `examples/C/-Tutorial-GetLibraryVersion`.

The Makefiles were created for MinGW on Windows 7 / 32bit.

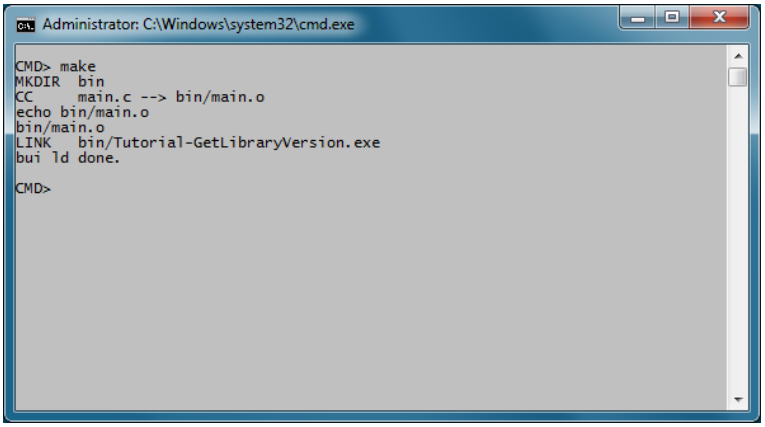


Figure 4.6: Compiling an example from the command prompt

Eclipse

Users of Eclipse find a project file in the example folders, e.g. `examples/C/Tutorial-GetLibrary-Version`.

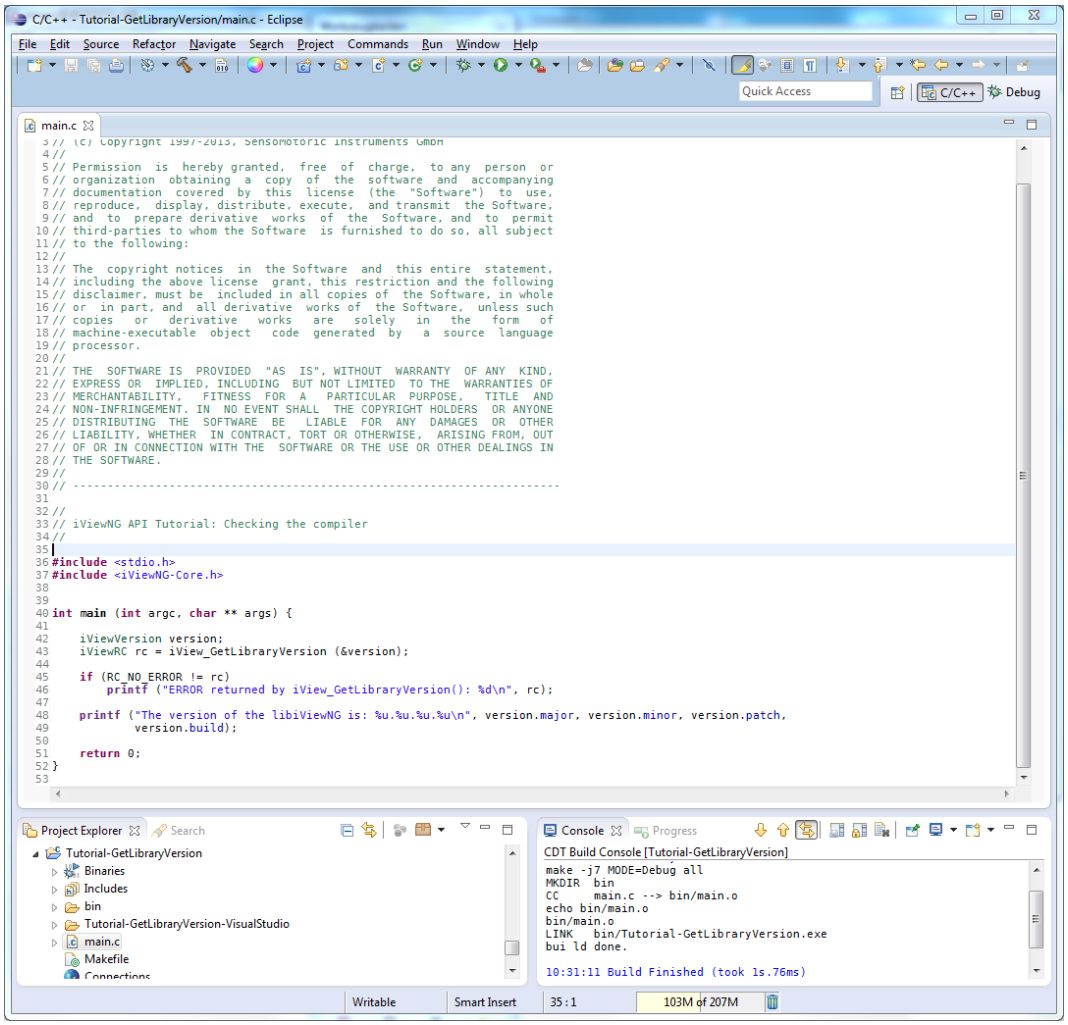


Figure 4.7: Compiling an example with Eclipse Juno

Let's have a look at the source code:

```
#include <stdio.h>
#include <iViewNG-Core.h>

int main(int argc, char ** args) {

  iViewVersion version;

  iViewRC rc = iView_GetLibraryVersion(&version);

  if (RC_NO_ERROR != rc)
    printf("ERROR returned by iView_GetLibraryVersion(): %d\n", rc);

  printf( "The version of the libiViewNG is: %u.%u.%u.%u\n",
          version.major, version.minor, version.patch, version.
      build );

  return 0;
}
```

Please make sure that this program compiles without errors or warnings and that the created program displays the version of the iViewNG API library, for example:

```
The version of the libiViewNG is: 1.1.1.55640
```

Note

For subsequent usage of the API functions, it is most convenient to encapsulate the function call as well as the check if the function call returned RC_NO_ERROR into a preprocessor macro:

```
extern iViewRC gRc;
extern wchar_t gErrorString[128];

#define CALL_API(_fun) { \
        if ( RC_NO_ERROR != (gRc = _fun) ) { \
            size_t gErrorStringSize = sizeof(gErrorString); \
            iView_iViewRcToString (gErrorString, &gErrorStringSize, gRc); \
            fprintf (stderr, "ERROR in '" #_fun "': %ls\r\n", gErrorString); \
            return gRc; \
        }\
}
```

Note

This enhances the readability of the source code and asserts that errors are detected immediately. All subsequent examples will use this preprocessor macro.

Step 2: Starting a Server and Connecting

Goal of this example is that you understand the required steps to connect to a server:

- starting an iViewNG server process,
- connecting to it,
- submitting your license key. For these steps, you will learn how to use the ticket mechanism.

The source code of the example discussed in this section can be found in the SDK inside the folder

examples/C/Tutorial-Connect.

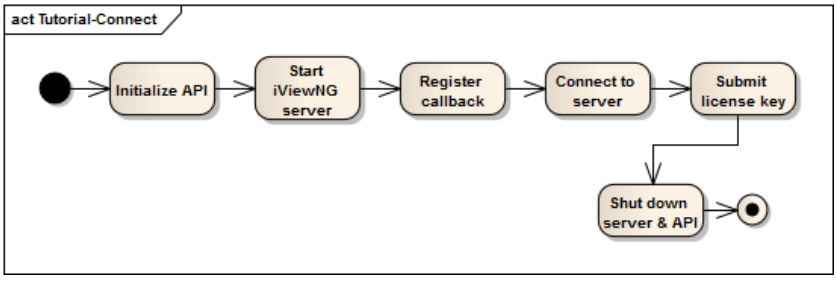


Figure 4.8: Control flow

The client application may use the function iView_StartServer() to spawn an iViewNG server process. The process will be executed on the computer which runs the client application:

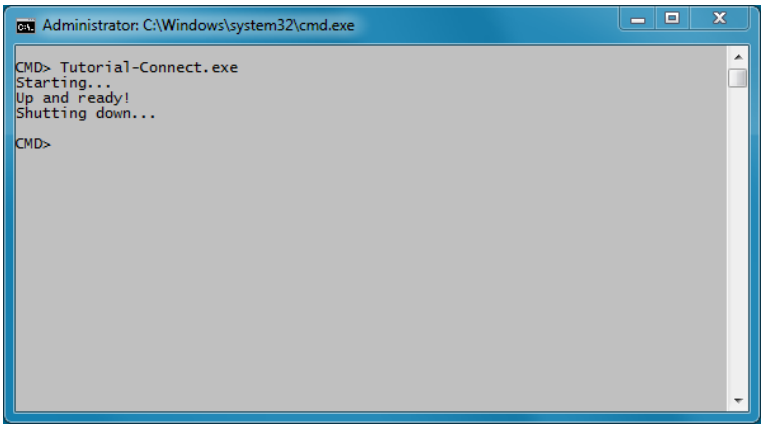


Figure 4.9: Executing the example.

Let's have a look at the source code:

```
#include <stdio.h>
#include <iViewNG-Core.h>
#include <iViewNG-Connection.h>
#include <iViewNG-Utility.h>

void iViewCallback (iViewTicket * const ticket) {}

int main (int argc, char ** args) {

    printf ("Starting...\n");

    CALL_API(iView_Init (IVIEWSDK_IVNG));
    CALL_API(iView_StartServer (NULL, L"--mode etg"));
```

The function iView_Connect() is the first function which is able to return results asynchronously, e.g. indicating that a connection is lost. Therefore, you should create a user callback function and register it to the iViewNG API:

```
CALL_API(iView_SetCallback (iViewCallback));
```

The callback function is discussed `later` in this tutorial. Now you may create a ticket for the iView_-Connect() function and connect to the server using your favourite communication scheme:

```
iViewHost server;
memset (&server, 0, sizeof(iViewHost));
server.hostAddress.type = IVIEW_SERVERADRRESS_SHAREDMEMORY
      ;

const unsigned int TIMEOUT_CONNECT = 1000; // milliseconds
const unsigned int RETRIES = 2; // number of attempts to retry to connect

iViewTicket * ticketConnect = NULL;

CALL_API(iView_CreateTicket (&ticketConnect));
CALL_API(iView_Connect (ticketConnect, &server, TIMEOUT_CONNECT,
      RETRIES, NULL ));
```

The connection is now ready to use.

Before using the API to control the eye tracker, please provide your license key to the iViewNG server, in this example, we use an empty string as license key:

```
iViewTicket * ticketAddLicense = NULL;

CALL_API(iView_CreateTicket (&ticketAddLicense));
CALL_API(iView_AddLicenseKey (ticketAddLicense, ""));
```

When done with your application, either just disconnect (and let the iViewNG server continue to operate) or terminate the server:

```
    printf ("Shutting down...\n");

    CALL_API(iView_ReleaseTicket (&ticketAddLicense));
    CALL_API(iView_ReleaseTicket (&ticketConnect));

    // terminate the iViewNG server and give it some time to shut down
    CALL_API(iView_ShutdownServer (15000));

    // shut down the API
    CALL_API(iView_Shutdown ());

    return 0;
}
```

Step 3: Receiving Data Through the Callback

Goal of this example is that you understand how to handle asynchronously delivered data. You will use a function which returns asynchronously, implement a simple callback and understand how to extract the received data from the iViewTicket structure.

The source code of the example discussed in this section can be found in the SDK inside the folder *examples/C/Tutorial-GetServerTime*.

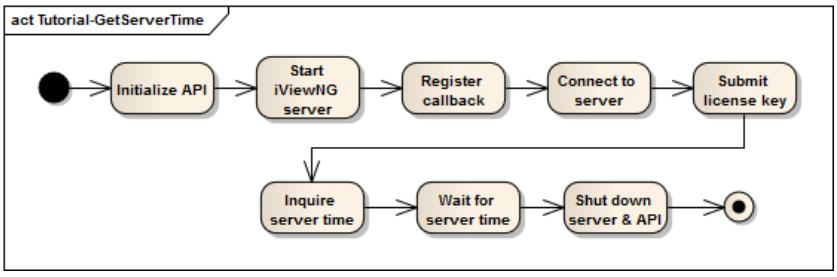


Figure 4.10: Control flow

All asynchronously provided data are submitted using the user-defined callback function. The only parameter for this function is a pointer to an iViewTicket structure which points to the ticket which was used for the API command which caused the asynchronous output. For example, the API function iView_GetServerTime() creates asynchronous output indicating the server time.

Set up the server and connect to it as seen above, but do not forget to create a flag indicating if iView_-GetServerTime() already returned the result.

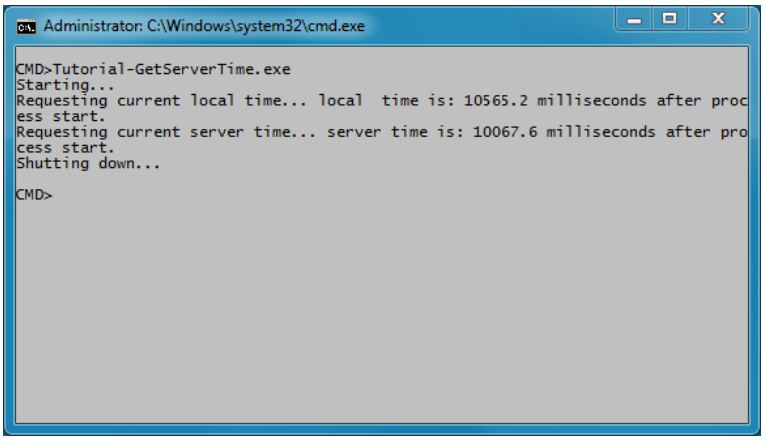


Figure 4.11: Executing the example.

Let's have a look at the source code:

```
int gReceivedTime = 0;
iViewTicket * gTicketConnect = NULL;
iViewTicket * gTicketAddLicense = NULL;
iViewTicket * gTicketGetServerTime = NULL;
iViewTicket * gTicketGetLocalTime = NULL;

int main (int argc, char ** args) {

    printf ("Starting...\n");

    CALL_API(iView_Init (IVIEWSDK_IVNG));
    CALL_API(iView_StartServer (NULL, L"--mode etg"));
    CALL_API(iView_SetCallback (iViewCallback));

    iViewHost server;
    memset (&server, 0, sizeof(iViewHost));
    server.hostAddress.type = IVIEW_SERVERADRRESS_SHAREDMEMORY
      ;

    CALL_API(iView_CreateTicket (&gTicketConnect));
    CALL_API(iView_Connect (gTicketConnect, &server, 1000, 2, NULL
      ));

    CALL_API(iView_CreateTicket (&gTicketAddLicense));
    CALL_API(iView_AddLicenseKey (gTicketAddLicense, ""));
```

Retrieve the local time and for the sake of laziness, let the callback handle the analysis:

```
printf ("Requesting current local time... ");

CALL_API(iView_CreateTicket (&gTicketGetLocalTime));
CALL_API(iView_GetLocalTime (gTicketGetLocalTime));

// let the callback handle the analysis of the result
iViewCallback (gTicketGetLocalTime);
```

Call out for the server time:

```
printf ("Requesting current server time...\n");

CALL_API(iView_CreateTicket (&gTicketGetServerTime));
CALL_API(iView_GetServerTime (gTicketGetServerTime));
```

and wait until the answer arrived or until the timeout limit is exceeded.

```
const unsigned int TIMEOUT_TICKET = 2000; // milliseconds
const unsigned int WAIT_PER_ITERATION = 50; // milliseconds
unsigned int waitingTime = 0; // the time we already waited for the result

do {
    iView_Sleep (WAIT_PER_ITERATION); // let the process rest for
       some time
    waitingTime += WAIT_PER_ITERATION;
}
while (!receivedTime && waitingTime <= TIMEOUT_TICKET);
```

Clean up: release all tickets before they notify that the connection is lost and shut down the server.

```
    printf ("Shutting down...\n");

    CALL_API(iView_ReleaseTicket (&gTicketGetLocalTime));
    CALL_API(iView_ReleaseTicket (&gTicketGetServerTime));
    CALL_API(iView_ReleaseTicket (&gTicketAddLicense));
    CALL_API(iView_ReleaseTicket (&gTicketConnect));

    // terminate the iViewNG server and give it some time to shut down
    CALL_API(iView_ShutdownServer (15000));

    // shut down the API
    CALL_API(iView_Shutdown ());

    return 0;
}
```

The last required function is the callback function which compares the incoming ticket to all open tickets. This is possible since the tickets are globally accessible. In case that the ticket associated with the function iView_GetServerTime() is returned, we double-check for invalid pointers and retrieve the time the server is up and running:

```
void iViewCallback (iViewTicket * const ticket) {

    if (NULL == ticket) {
        printf ("ERROR: NULL ticket");
        return;
    }

    // handle errors
    if (RC_NO_ERROR != ticket->returnCode) {

        wchar_t buffer [128];
        size_t buffersize = sizeof (buffer);

        if (iView_iViewRcToString (buffer, &buffersize,
      ticket->returnCode))
            wcscpy (buffer, L"(invalid)");

        wprintf (L"ERROR: function %s returned returncode %u (%s)\n", ticket->
      functionName, ticket->returnCode, buffer);
        return;
    }

    // handle iView_Connect() answers
    if (ticket == gTicketConnect) {
        // nothing to do.
        return;
    }
```

```
    // handle iView_AddLicenseKey() answers
    if (ticket == gTicketAddLicense) {
        // nothing to do.
        return;
    }

    // handle iView_GetServerTime() answers
    if (ticket == gTicketGetServerTime) {

        // Avoid accessing NULL pointers and check if the result type matches
       the expected type!
        if (NULL == ticket->result || NULL == ticket->result->data)
       {
            wprintf (L"ERROR: iView_GetServerTime() returned invalid result.\n"
      );
        }
        else {
            // If everything is allright, access the time.
            uint64_t time = ((iViewTime *) ticket->result
      ->data)->time;
            printf ("server time is: %4.1f milliseconds after process start.\n"
      , (time / 1000000.f));
        }
    }

    // handle iView_GetLocalTime() answers
    if (ticket == gTicketGetLocalTime) {
        // Avoid accessing NULL pointers and check if the result type matches
       the expected type!
        if (NULL == ticket->result || NULL == ticket->result->data)
       {
            wprintf (L"ERROR: iView_GetLocalTime() returned invalid result.\n")
      ;
        }
        else {
            // If everything is allright, access the time.
            uint64_t time = ((iViewTime *) ticket->result
      ->data)->time;
            printf ("local  time is: %4.1f milliseconds after process start.\n"
      , (time / 1000000.f));
        }
    }

    return;
}
```

Step 4: Receiving Gaze Data

Goal of this example is to demonstrate how to receive data which is provided by the iViewNG as data stream. To keep the source code simple, we use the iViewNG Convenience Library.

The source code of the example discussed in this section can be found in the SDK inside the folder *examples/C/Tutorial-DataAcquisition*.

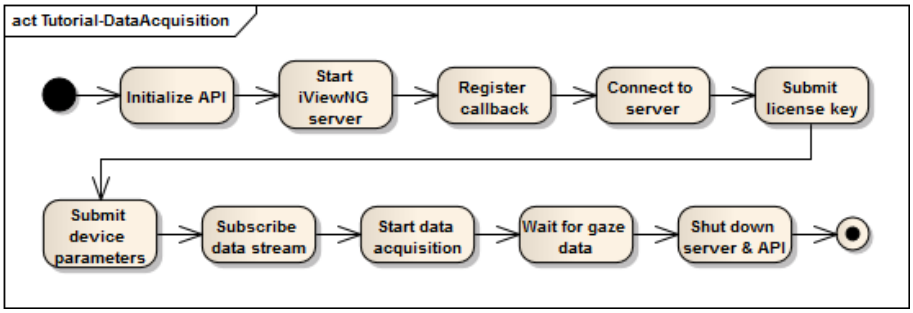


Figure 4.12: Control flow

In many use cases, the client application needs to receive the gaze direction of the person using the eye tracking device. The gaze data is a result of the eye and gaze tracking algorithm which is part of the iViewNG server. The data structure which represents the gaze direction depends on the eye tracking device (e.g. mobile eye tracking device vs. fixed remote eye tracking device), but the workflow is shared. The user has to

1. initialize the iViewNG server,
2. provide the required device parameters,
3. subscribe to the desired data stream, and
4. start data acquisition.

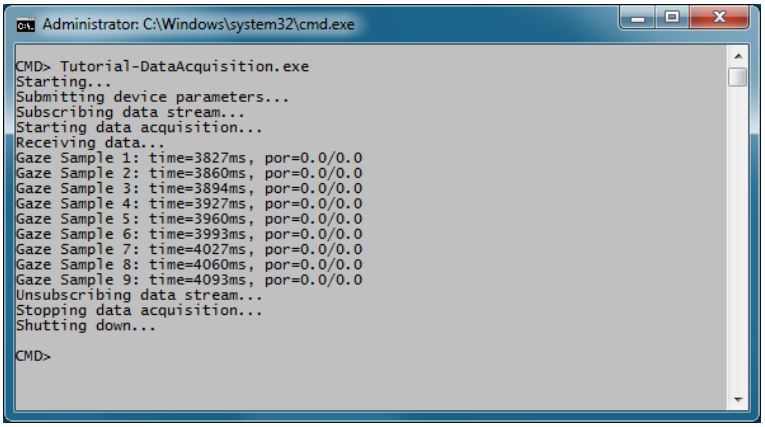


Figure 4.13: Executing the example.

This tutorial demonstrates a brief application which implements this workflow, utilizing the iViewNG Convenience Library.

First, we initialize the iViewNG server as we usually do:

```
#include <stdio.h>
#include <string.h> // for memset()

#include <iViewNG-Core.h>
```

```
#include <iViewNG-Connection.h>
#include <iViewNG-DataAcquisition.h>
#include <iViewNG-Device-ETG.h>
#include <iViewNG-Utility.h>
#include "iViewNG-Convenience.h"

unsigned int receivedGazeSamples = 0;
void MyCallback (iViewTicket * const ticket);

int main (int argc, char ** args) {

    CALL_API(iView_Init (IVIEWSDK_IVNG));
    CALL_API(iView_StartServer (NULL, L"--mode etg"));
    CALL_API(iView_SetCallback (MyCallback));

    iViewHost server;
    memset (&server, 0, sizeof(iViewHost));
    server.hostAddress.type = IVIEW_SERVERADRRESS_SHAREDMEMORY
      ;

    iViewTicket * ticketConnect = NULL;

    CALL_API(iView_CreateTicket (&ticketConnect));
    CALL_API(iView_Connect (ticketConnect, &server, 1000, 2, NULL)
      );

    iViewTicket * ticketAddLicense = NULL;

    CALL_API(iView_CreateTicket (&ticketAddLicense));
    CALL_API(iView_AddLicenseKey (ticketAddLicense, ""));
```

Second, we send the required parameters for an *ETG* device and block until the parameters are acknowledged. All parameters are neutral and use the ETG device just as it is initialized by default.

Please note the use of `iView_StartCatchingTickets()` and `iView_WaitForTicket()` which are no API functions, but open source convenience functions:

```
iViewDeviceParametersEtgCameraPlayback
      specificParameters;

specificParameters.outputDirectory = L".";
specificParameters.baseFilename = L"Tutorial-DataAcquisition";
specificParameters.operationMode = IVIEWTRACKINGMODE_CURRENT
      ;
specificParameters.cameraResolutionScene =
      IVIEWRESOLUTION_CURRENT;
specificParameters.cameraResolutionEye =
      IVIEWRESOLUTION_CURRENT;
specificParameters.samplingRateSceneCam =
      IVIEWSAMPLINGRATE_CURRENT;
specificParameters.samplingRateEyeCam =
      IVIEWSAMPLINGRATE_CURRENT;
specificParameters.whiteBalanceProgram =
      IVIEWWHITEBALANCE_CURRENT;
specificParameters.audioState = IVIEWAUDIOSTATE_CURRENT
      ;
specificParameters.overlaySpecList = NULL;

iViewDeviceParameters deviceParameters;

deviceParameters.deviceType = IVIEWDEVICE_ETG_CAMERAPLAYBACK
      ;
deviceParameters.parameters = &specificParameters;

iViewTicket * ticketDeviceParameters = NULL;
```

```
CALL_API(iView_CreateTicket (&ticketDeviceParameters));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_SetDeviceParameters (
      ticketDeviceParameters, &deviceParameters));

if (false == iView_WaitForTicket (ticketDeviceParameters, 5000)) {
    printf ("Failed to set device parameters.\n");
    return -1;
}
```

Third, we subscribe to the gaze information data stream so that we receive data immediately after starting the data acquisition:

```
iViewStreamSubscription subscription;
memset (&subscription, 0, sizeof(iViewStreamSubscription
      ));
subscription.streamType = IVIEWDATASTREAM_GAZE_INFORMATION
      ;

iViewTicket * ticketSubscription = NULL;

CALL_API(iView_CreateTicket(&ticketSubscription));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_SubscribeDataStream(ticketSubscription,
       &subscription));

if (false == iView_WaitForTicket (ticketSubscription, 2000)) {
    printf ("Failed to set subscribe data stream.\n");
    return -1;
}
```

Finally, we start data acquisition and, again, block until this is acknowledged:

```
iViewTicket * ticketStartAcquisition = NULL;

CALL_API(iView_CreateTicket(&ticketStartAcquisition));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_StartDataAcquisition(
      ticketStartAcquisition));

if (false == iView_WaitForTicket (ticketStartAcquisition, 2000)) {
    printf ("Failed to start data acquisition.\n");
    return -1;
}
```

At this point, our callback will receive gaze data. We explicitly wait until we receive 10 gaze samples or until the timeout strikes:

```
const unsigned int TIMEOUT_TICKET = 2000; // milliseconds
const unsigned int WAIT_PER_ITERATION = 50; // milliseconds
unsigned int waitingTime = 0; // the time we already waited for the result

do {
    iView_Sleep (WAIT_PER_ITERATION); // let the process rest for
       some time
    waitingTime += WAIT_PER_ITERATION;
}
while (9 > receivedGazeSamples && waitingTime <= TIMEOUT_TICKET);
```

Before terminating everything, we explicitly release all memory structures. This is not required since the application terminates anyway, but it is considered good style to release what you no longer need:

```
    iViewTicket * ticketUnsubscription = NULL;

    CALL_API(iView_CreateTicket(&ticketUnsubscription));
    CALL_API(iView_UnsubscribeDataStream(
      ticketUnsubscription, IVIEWDATASTREAM_GAZE_INFORMATION));

    iViewTicket * ticketStopAcquisition = NULL;

    CALL_API(iView_CreateTicket(&ticketStopAcquisition));
    CALL_API(iView_StopDataAcquisition(
      ticketStopAcquisition));

    CALL_API(iView_ReleaseTicket (&ticketConnect));
    CALL_API(iView_ReleaseTicket (&ticketAddLicense));
    CALL_API(iView_ReleaseTicket (&ticketDeviceParameters));
    CALL_API(iView_ReleaseTicket (&ticketStartAcquisition));
    CALL_API(iView_ReleaseTicket (&ticketSubscription));
    CALL_API(iView_ReleaseTicket (&ticketUnsubscription));
    CALL_API(iView_ReleaseTicket (&ticketStopAcquisition));

    // terminate the iViewNG server and give it some time to shut down
    CALL_API(iView_ShutdownServer (15000));

    // shut down the API
    CALL_API(iView_Shutdown ());

    return 0;
}
```

When you wear the ETG device while executing the program shown above, you should see the gaze data printed to the console, for example:

```
Gaze Sample 1: time=3156ms, por=137.0/833.9
Gaze Sample 2: time=3189ms, por=567.3/792.1
Gaze Sample 3: time=3222ms, por=511.5/696.1
Gaze Sample 4: time=3255ms, por=517.7/677.6
Gaze Sample 5: time=3289ms, por=521.3/661.8
Gaze Sample 6: time=3322ms, por=525.5/652.7
Gaze Sample 7: time=3355ms, por=522.2/643.8
Gaze Sample 8: time=3389ms, por=531.2/641.4
Gaze Sample 9: time=3422ms, por=527.3/638.7
Gaze Sample 10: time=3455ms, por=527.9/634.7
```

Step 5: ETG Remote Viewer Application

Tutoral-RemoteViewer is a versatile command-line application which can:

- start a server
- connect to a server
- parametrize the server to produce 30Hz or 60Hz gaze
  - please note that 60Hz gaze is possible with ETG 1.9 or newer devices
- subscribe to various data streams
  - gaze
  - eye images

  - scene images decoded by server
  - scene images decoded by client
  - scene images with gaze drawn onto them
- start / stop data acquisition
- perform 1pt and 3pt calibration

Tutorial-RemoteViewer can assert different amount of control over an iViewNG-Server. It will either have full control (sets device parameters, start/stops acquisition) or be a passive listener (will only subscribe to data streams). Being a passive listener is useful when connecting to a server which is being controlled by an iViewETG-Client. In such a use-case we regard iViewETG-Client as the main client, we do not want Tutorial-RemoteViewer to change the device parameters or start/stop acquisition.

To be a passive listener, you can use the parameter:

- `--iviewetg <server_ip>`

To have full control, you can use the parameter:

- `--server <server_ip>`

  Also any server started by the Tutorial-RemoteViewer itself will be in full control of the tutorial. A server will automatically be started if you specify neither `---iviewetg` nor `---server` .

Step 5.1: Command-Line Parameters

If you start Tutorial-RemoteViewer without any parameters, it will automatically display a list of all command line parameters with a brief description.

```
C:\Program Files (x86)\SMI\iView NG SDK\examples\C\Tutorial-RemoteViewer\bin>
      Tutorial-RemoteViewer.exe
ERROR: Refusing to run without showing any data stream!

USAGE
        Tutorial-RemoteViewer.exe [OPTION]...

OPTIONS

        --help               to print this information
        --show-gaze          to print the gaze coordinates to the console
        --show-eyes          to open two windows displaying the eye images
        --scale-eyes f       rescale eye images by multiplying image size by f
        --show-scene         to open a window displaying the scene images (the
                             scene video is decoded by the server and full
      image
                             will be transferred to the remote viewer)
        --show-scene-h264    to open a window displaying the scene images (the
                             scene video is transferred as H.264 stream,
                             decoded by the remote viewer)
        --show-scene-with-gaze  to open a window displaying the scene images (
      the
                             scene video is decoded by the server and already
                             contains the gaze cursor, full image is
      transferred to
                             the remote viewer)
        --show-scene-h264-with-gaze    to open a window displaying the scene
```

```
images
                        (the scene video is transferred as H.264 stream,
                        decoded by the client and contains a gaze cursor
                        overlay created by the client)
  --scale-scene f       rescale scene images by multiplying image size by
f
  --server ip_address   connect to the specified remote iViewNG server,
                        the server can be setup (device parameters) by
this
                        Tutorial-RemoteViewer
  --iviewetg ip_address  connect to the specified remote iViewNG server
                        which had already been setup by an iViewETG-Client
.
                        This Tutorial-RemoteViewer will not submit any new
 device
                        parameters, it will only try to subscribe and
listen to
                        the data it wants. Whether the data can be
received
                        depends on the parameters that iViewETG-Client has
 set
  --calibrate1pt        to perform a 1-point calibration
  --calibrate3pt        to perform a 3-point calibration
  --samplingrate n      set eye tracking sampling rate to either 30 or 60
Hz
```

Step 5.2: Standalone-Usage Examples

One Computer Setup

Let's assume you have an ETG device attached to your SDK Laptop and want to get:

- scene with gaze
- eye images
- gaze printed to console

Let's assume you have installed your SDK to

```
C:\Program Files (x86)\SMI\iView NG SDK>
```

and that your Tutorial-RemoteViewer is located at

```
C:\Program Files (x86)\SMI\iView NG SDK\examples\C\Tutorial-RemoteViewer\bin>
```

(let's call the RemoteViewer's location `C:\RV` for simplicity).

You would simply enter:

```
C:\RV\Tutorial-RemoteViewer.exe --show-scene-with-gaze --show-eyes --show-gaze
```

This command line would automatically start an iViewNG-Server and set it up to provide you with a scene with gaze, eyes and gaze. Tutorial-RemoteViewer would then visualize the received data. When you end the tutorial, the server will also end.

Two Computers Setup

Let's assume you have an ETG device attached on a Remote Laptop with Network access and IP Address 123.123.123.123. Let's assume you have an iViewETG or iViewNG-SDK installed on this Remote Laptop. You would manually start a Server on the Remote Laptop by entering in command prompt:

```
C:\iViewETG\iViewNG-Server --mode etg
```

The server will start and minimize itself to the TaskBar / Notification Area (usually as an Icon in the lower right screen). It will automatically start listening to any incoming connection requests, either local (Shared Memory) or remote (via Network).

Now you want to get some data from the Remote Laptop. On your SDK Laptop you would enter in command prompt:

```
C:\RV\Tutorial-RemoteViewer.exe --show-scene-h264-with-gaze --show-eyes --show-
      gaze --server 123.123.123.123
```

By explicitly specifying `---server` , RemoteViewer will not create a new local server but rather connect to the one at specified IP. Please note the change from `-show-scene-with-gaze` to `-show-scene-h264-with-gaze` . Since we are communicating over network, we want the server to send us an h264 stream rather than full decoded scene images (to keep the network traffic low). The API on RemoteViewer side will then decode the h264 stream itself and draw the gaze onto it. When you end the tutorial, the Remote Server will also end. To circumvent this, please see Usage in conjunction with iViewETG.

Calibration

Let's assume you have an ETG device attached to your SDK Laptop and want to do a 3-point calibration. You will need to subscribe at least scene with gaze:

```
C:\RV\Tutorial-RemoteViewer.exe --show-scene-with-gaze --calibrate3pt
```

This will start the server and set it up so that RemoteViewer gets scene with gaze. Now you can direct your Participant to fixate a point and then you click on the scene-with-gaze window to calibrate. For a 3-point calibration you have to do this 3 times.

Calibration can also work remotely. Let's assume you have an ETG device attached on a Remote Laptop with Network access and IP Address 123.123.123.123. Let's assume you have an iViewETG or iViewNG-SDK installed on this Remote Laptop. You would manually start a Server on the Remote Laptop by entering in command prompt:

```
C:\iViewETG\iViewNG-Server --mode etg
```

Let's assume your SDK is installed on a Tablet with low screen resolution. Again you will need to subscribe at least scene with gaze, but keep the network traffic low using the h264 variant. And you want your 1280x960 scene image to fit onto the screen, so you will want to resize it.

```
C:\RV\Tutorial-RemoteViewer.exe --show-scene-h264-with-gaze --scale-scene 0.5 -
      -calibrate1pt --server 123.123.123.123
```

Step 5.3: Usage in conjunction with iViewETG

Let's assume you have an ETG device attached on a Remote Laptop with Network access and IP Address 123.123.123.123. Let's assume you have an iViewETG on this Remote Laptop want to do a recording with it. You would then double click on the iView ETG Shortcut (e.g. on the Desktop) which will start iViewETG. Internally, this would start an iViewNG-Server and iViewETG-Client. The client has a GUI with which you can setup your experiment and participant, perform calibration and perform recordings. The Server will automatically be listening to incoming network connections.

Now you want to oversee the Remote Laptop Recording. On your SDK Laptop you would enter in command prompt:

```
C:\RV\Tutorial-RemoteViewer.exe --show-scene-h264-with-gaze --show-eyes --show-
      gaze --iviewetg 123.123.123.123
```

By explicitly specifying `---iviewetg` , RemoteViewer will not create a new local server but rather connect to the one at specified IP. It will also not try to change the device parameters (eye/scene resolution, sampling rate) nor will it start/stop data acquisition, as the remote iViewETG-Client is regarded as the main client. Please also note the usage of `-show-scene-h264-with-gaze` . Since we are communicating over network, we want the server to send us an h264 stream rather than full decoded scene images (to keep the network traffic low). The API on RemoteViewer side will then decode the h264 stream itself and draw the gaze onto it. When you end the RemoteViewer, the iViewETG will not be affected, thanks to the `---iviewetg` .

Please note that technically, an API user (e.g. Tutorial-RemoteViewer) can always try to start/stop data acquisition, subscribe new data streams or end the server, set device parameters etc. We have introduced the `---iviewetg` parameter as a convenience functionality, where iViewETG-Client is the main controller and Tutorial-RemoteViewer a passive listener of subscribed data streams.

Keep in mind that iViewETG-Client will usually setup the server so that the client decodes the h264 scene stream. This means that Tutorial-RemoteViewer will need to subscribe to `-show-scene-h264` or `-show-scene-h264-with-gaze` , the non-h264 variants will not work. Also a change in eye or scene resolution/framerate by iViewETG-Client will also affect the RemoteViewer.

Step 5.4: In-depth look at ETG Remote Viewer Application

The goal of this final example is to demonstrate how

- to connect over network to a remote iViewNG server,
- multiple data streams are used,
- interaction can be realized, and
- a 1-point calibration is performed.

The source code of the example discussed in this section can be found in the SDK inside the folder *examples/C/Tutorial-RemoteViewer*.

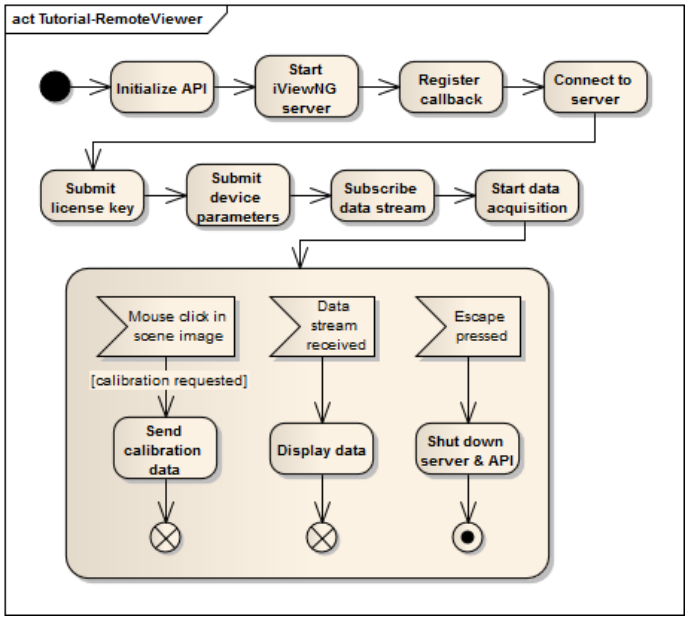


Figure 4.14: Control flow

In this example, we create a simple remote viewer application for the iViewETG device. The application will be able to connect to a local iViewNG server using shared memory communication and to remote servers using network communication. Eye and scene images will be displayed using simple undecorated windows. User interaction will be bound to the console and the scene image window.

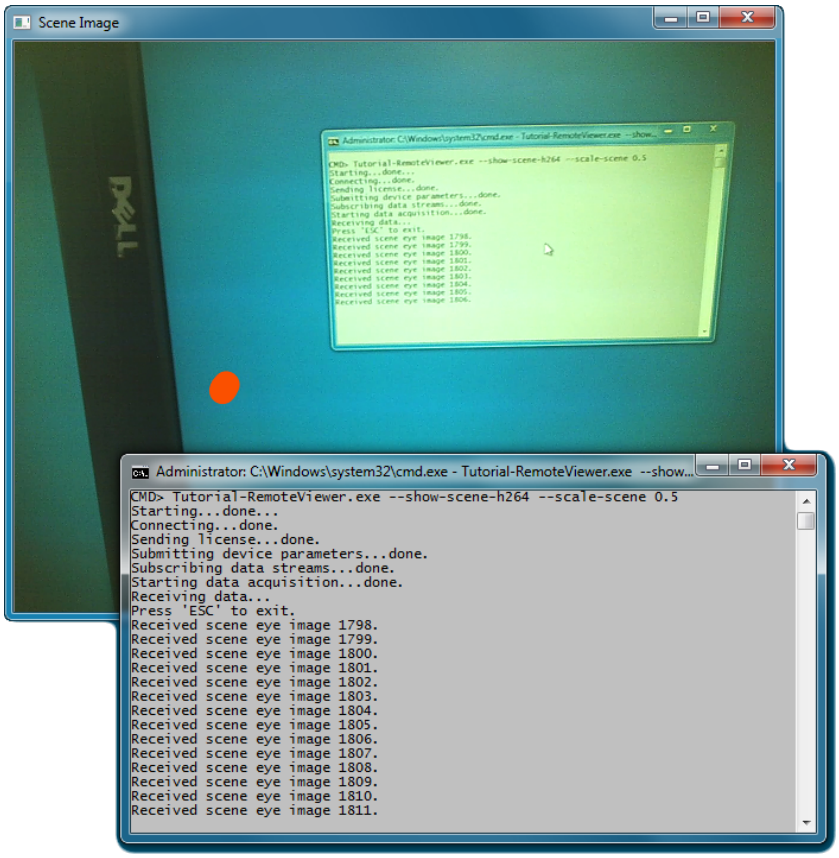


Figure 4.15: Executing the example.

To enhance the readability of the source code, we extract several steps into dedicated functions.

- `ParseCommandLine()` analyses the command line parameters and stores the resulting parameters in globally acessible variables. Please note that variables which have the character `g` as prefix are globally acessible variables. The user may select to subscribe and show received gaze data, eye images and scene images.
- `Setup()` initializes the API, starts a server if the user specified to connect to a local server, connects to the local or remote server, sends the license and ETG device parameters.
- `Subscribe()` subscribes to all data streams the user wishes to receive.
- `Start()` starts the data acquisition.
- `Cleanup()` performs the inverse commands: stops data acquisition, unsubscribes from all data streams, releases all data and shuts down the local or remote server as well as the API.

Since all the required techniques were demonstrated by earlier tutorial steps, we do not discuss them in detail. Please refer to the source code for details.

The main program

The main program invokes the server, connects, subscribes data streams and waits until the user terminates the program:

```
...

#include <iViewNG-Core.h>
#include <iViewNG-Calibration.h>
#include <iViewNG-Connection.h>
#include <iViewNG-DataAcquisition.h>
#include <iViewNG-Device-ETG.h>
#include <iViewNG-Utility.h>

...

/* ***************   DATA     *************** */

iViewTicket * gTicketStartAcquisition = NULL;
iViewTicket * gTicketConnect = NULL;
..
iViewTicket * gTicketCalibration1Pt = NULL;
iViewTicket * gTicketCalibration3Pt[3];

iViewHost gServer;

..

// Local data for calibration
char gCalibrationPointsToDo = 0;
unsigned int gCurrentFrameNumber = 0;

// Local data for gaze overlay
int gGazeX = 0;
int gGazeY = 0;

/* ***************  FUNCTIONS *************** */

int main (int argc, char ** argv) {

    iViewRC rc;
    memset (&gServer, 0, sizeof(iViewHost));

    if ( (rc = ParseCommandLine (argc, argv)) || (rc = Setup ()) || (rc =
      Subscribe ()) || (rc = Start ()) || (rc = SetupCalib ()))
        return -1;

    printf ("Receiving data...\n");
    WaitForUserInteraction ();

    printf ("Cleaning up...\n");
    Cleanup ();

    return 0;
}
```

The callback functions

The used callback analyses the result type and delegates operation to dedicated event handler:

```
void MyCallback (iViewTicket * const ticket) {

    // this should never happen
```

```
if (NULL == ticket)
    return;

// extract the result
iViewResult const * const result = ticket->result;

// Check if we got a 1pt calibration response
if (gTicketCalibration1Pt == ticket) {
    printf ("Sending 1pt calibration data %s successful.\n", RC_NO_ERROR
   == ticket->returnCode ? "was" : "was not");
    iView_ReleaseTicket( &gTicketCalibration1Pt );
    return;
}

// Check if we got a 3pt calibration response
for (int i=0; i<3; i++) {
    if (gTicketCalibration3Pt[i] == ticket) {
        printf ("Sending 3pt calibration data %s successful.\n",
  RC_NO_ERROR == ticket->returnCode ? "was" : "was not");
        iView_ReleaseTicket( &gTicketCalibration3Pt[i] )
  ;
        return;
    }
}

// if the ticket carries no result or if it's not a data stream ticket,
   we're not interested
if (NULL == result || IVIEWRESULT_SUBSCRIBE_DATASTREAM
   != result->type)
    return;

// cast to the proper result type
iViewDataStream const * const stream = (iViewDataStream
   const *) result->data;

if(stream->lastStreamEntity == IVIEWDATASTREAM_END
  ){
    fprintf(stderr,"Got last stream Entity\n");
    return;
}

switch (stream->type) {

    case IVIEWDATASTREAM_GAZE_INFORMATION:

        handleGazeSample ((iViewDataStreamGazeSample
   const *) stream->data);
        break;

    case IVIEWDATASTREAM_EYEIMAGES_LEFT:

        handleEyeImage ((iViewDataStreamEyeImage*)
  stream->data);
        break;

    case IVIEWDATASTREAM_EYEIMAGES_RIGHT:

        handleEyeImage ((iViewDataStreamEyeImage*)
  stream->data);
        break;

    case IVIEWDATASTREAM_SCENEIMAGES:
    case IVIEWDATASTREAM_SCENEIMAGES_WITH_GAZE
  :
    case IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED_WITH_GAZE
  :
```

```
            handleSceneImageWithGaze ((iViewDataStreamSceneImage
      *) stream->data);
            break;

        case IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED
      :

            handleH264DecodedSceneImage ((iViewDataStreamSceneImage
      *) stream->data);
            break;

    }

    iView_ReleaseResult (ticket);
    return;

}
```

The event handler functions extract the data from the data stream structure. In the case of images, the display of the images is delegated to the visualization functions.

```
void handleGazeSample (iViewDataStreamGazeSample const
       * const gazeSample) {

    gGazeX = (int) (gazeSample->pointOfRegard.x + 0.5);
    gGazeY = (int) (gazeSample->pointOfRegard.y + 0.5);

    // Print to console only if requested.
    if (gShowGaze)
        printf ("Gaze Sample %u: time=%ums, por=%d/%d\n", gazeSample->
      eyeFrameNumber,
                (uint32_t) (gazeSample->timestamp / 1000000),
      gGazeX, gGazeY);

    return;

}

void handleEyeImage (iViewDataStreamEyeImage * image) {

    switch (image->eye) {

        case EYE_LEFT:

            displayLeftEyeImage (image->imageData);
            break;

        case EYE_RIGHT:

            displayRightEyeImage (image->imageData);
            break;

        case EYE_UNKNOWN:
            fprintf (stderr, "ERROR in callback: unknown eye type.\n");
            break;
        }

    return;
}
```

For the scene images, the situation is a bit different since there are multiple ways to receive scene videos: full uncompressed images (data streams IVIEWDATASTREAM_SCENEIMAGES and IVIEWDATASTREAM_SCENEIMAGES_WITH_GAZE), images which were uncompressed by the API (data

streams IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED and IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED_WITH_GAZE) and raw H.264 packets (data stream IVIEWDATASTREAM_SCENEIMAGES_H264).

Please mind that the transfer of uncompressed 960p images at 24 Hz creates approximately 675 Mbit/s which is feasible with shared memory communication on a reasonable computer but can be rather difficult to transfer over Gigabit Ethernet or even 10GigE if there is no direct connection (without switches or hubs) between sender and receiver. Either the resolution or frame size must be reduced or images are to be received as H.264 compressed packages, decoded by the API. The tutorial implements both ways and leaves the decision up to the user.

```
void handleSceneImageWithGaze (iViewDataStreamSceneImage
       * image) {

    gCurrentFrameNumber = image->sceneFrameNumber;
    displaySceneImage (image->imageData);

    return;
}

void handleH264DecodedSceneImage (iViewDataStreamSceneImage
       * image) {

    gCurrentFrameNumber = image->sceneFrameNumber;

    // Draw gaze onto scene only if requested.
    if (gShowSceneImagesWithGaze || gShowSceneH264ImagesWithGaze) {
        drawGazeOverlay (image->imageData, gGazeX, gGazeY);
    }

    displaySceneImage (image->imageData);

    return;
}
```

The source code includes a visualization module which implements the data display by using the *OpenCV* library.

Calibration

The iViewNG API provides a function set to perform an explicit calibration. That means, the API client is supposed to provide the coordinates of one or more points within the field of view the participant gazed at and the moment in time when this particular point was the point of regard. This requires a priori knowledge provided by a cooperative subject and an operator.

The fist step is the initialization which specifies the calibration algorithm. This implies the number of calibration points to be submitted:

```
iViewCalibrationParametersEtg
      specificParametersSetup;
specificParametersSetup.calibrationDistance = 0;

iViewCalibrationParameters parametersSetup;
parametersSetup.type = IVIEWCALIBRATIONTYPE_ETG_1POINTSPHERICAL
      ;
if (gCalibrate3Pt) {
```

```
    parametersSetup.type = IVIEWCALIBRATIONTYPE_ETG_3POINT
      ;
}

parametersSetup.applyAtFrame = -1;
parametersSetup.parameters = &specificParametersSetup;

iViewTicket * ticket = NULL;
CALL_API(iView_CreateTicket(&ticket));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_SetupCalibration(ticket, &parametersSetup)
      );

if (0 == iView_WaitForTicket (ticket, 2000)) {
    printf ("Failed to set calibration parameters.\n");
    return -1;
}

// and start calibration
CALL_API(iView_ReleaseTicket(&ticket));
CALL_API(iView_CreateTicket(&ticket));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_StartCalibration(ticket));

if (0 == iView_WaitForTicket (ticket, 2000)) {
    printf ("Failed to start calibration.\n");
    return -1;
}
```

The calibration may be started any time before transmitting the calibration points:

```
CALL_API(iView_ReleaseTicket(&ticket));
CALL_API(iView_CreateTicket(&ticket));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_StartCalibration(ticket));

if (0 == iView_WaitForTicket (ticket, 2000)) {
    printf ("Failed to start calibration.\n");
    return -1;
}
```

The last step of the calibration is the transfer of *n* calibration points where *n* depends on the selected calibration algorithm. If the calibration needs more than 1 point, the server may use other algorithms which use fewer points as intermediate solution which are supposed to improve the gaze calculation during the calibration. This depends on the used device.

If the user specified to perform a calibration and *OpenCV* detects a mouse click inside the scene image window, the calibration function is called which submits the screen coordinates to the iViewNG server so the gaze calculation can be corrected:

```
iViewRC Calibrate (const unsigned int coordinateX, const unsigned int
      coordinateY) {

    if (gCalibrationPointsToDo == 0)
        return RC_INVALID_STATE;

    // Setup, sending calibration type
    if (gCalibrate1Pt) {
        printf ("Sending data for 1-point-calibration (%u/%u) at frame %u.\n",
      coordinateX, coordinateY,
                gCurrentFrameNumber);
    } else if (gCalibrate3Pt) {
```

```
        printf ("Sending data point %u/3 for 3-point-calibration (%u/%u) at
       frame %u.\n",
                3-gCalibrationPointsToDo+1, coordinateX, coordinateY,
      gCurrentFrameNumber);
    } else {
        return RC_INVALID_STATE;
    }

    iViewCalibrationData1Point calibrationPoint;
    calibrationPoint.coordinateX = coordinateX;
    calibrationPoint.coordinateY = coordinateY;

    iViewCalibrationData data;
    data.type = IVIEWCALIBRATIONDATATYPE_1POINT
      ;
    data.sceneFrameNumber = gCurrentFrameNumber;
    data.unit = IVIEWCALIBRATIONUNIT_PIXEL;
    data.usage = 0;
    data.id = 0;
    data.parameters = &calibrationPoint;

    if (gCalibrate1Pt) {
        CALL_API(iView_CreateTicket (&gTicketCalibration1Pt))
      ;
        CALL_API(iView_SetCalibrationData (
      gTicketCalibration1Pt, &data));
        // We don't want to block here, thus we receive the reply ticket via
       MyCallback().
    }

    if (gCalibrate3Pt) {
        CALL_API(iView_CreateTicket (&gTicketCalibration3Pt[3
      -gCalibrationPointsToDo]));
        CALL_API(iView_SetCalibrationData (
      gTicketCalibration3Pt[3-gCalibrationPointsToDo], &data));
        // We don't want to block here, thus we receive the reply ticket via
       MyCallback().
    }


    gCalibrationPointsToDo--;

    return RC_NO_ERROR;
}
```

4.2 Using C#

Available soon.

4.3 Using Python

Available soon.

4.4 Using Matlab®

Available soon.

Chapter 5

Frequently Asked Questions

5.1 Frequently Asked Questions

How many data streams can be subscribed?

The number of data streams which can be subscribed is technically unlimited. However, with some devices, data streams might interfere and are mutually exclusive. Please refer to the description of iViewDataStreamType for further details.

Please note that each data stream consumes communication bandwidth! If server and client processes run on the *same* computer (single PC setup using SMI hardware), bandwidth will not be an issue. If server and client processes run on *different* computers (dual PC setup) and must communicate over network, bandwidth *is* an issue. Refer to the description of iViewDataStreamType to calculate your bandwidth requirement and triple-check if your network connection can provide this bandwidth! Please also note that the theoretical bandwidth mentioned in network infrastructure hardware technical specifications may differ from practically usable bandwidth and in case of wireless LAN, might also depend on the environment.

The bandwidth consumed depends on

- data channel,
- image resolution (in the case of image data),
- sample rate.

If the network bandwidth is insufficient, you may experience an increasing delay and eventually the applications will terminate because the memory is exceeded. Please check the memory footprint of your application!

What computer performance is required for remote applications?

The ETG system uses H.264 data compression for submitting the scene video and audio signals via USB to the server computer. The scene images and audio signal is encoded in the ETG device, thus

the data *must* be decoded at some time if it shall be displayed or played during data acquisition. For technical reasons, the encoding/decoding creates a delay between acquired and displayed scene image. There is *no way* to eliminate this lag.

If the scene video shall be displayed on the screen, this compressed data stream must be decoded. This consumes much CPU time: with an *Intel Core i7-2620M* CPU at 2.8 GHz, the decoding process alone may consume 10-20% CPU time. In a two PC setup, you will usually transfer compressed data and let the API decode it on the client PC. Please assert that the client PC has enough power to decode the video! If the system performance is insufficient, you will experience an increasing delay and eventually the applications will terminate because the memory is exceeded. Please check the memory and CPU footprint of your application!

Displaying a high resolution scene image may put additional load to the system. Please make sure that your computer is equipped with the most recent graphics chipset driver! Use a computer with dedicated graphics hardware.

Chapter 6

Licenses

6.1 SMI Licenses

License Agreement and Warranty for SDK Provided Free of Charge

The SDK is published under the following license:

IMPORTANT – PLEASE READ CAREFULLY: This license agreement ("Agreement") is an agreement between you (either an individual or a company, "Licensee") and SensoMotoric Instruments GmbH ("SMI"). The "Licensed Materials" provided to Licensee free of charge subject to this Agreement include the Software Development Kit (the "SDK") as well as any "on-line" or electronic documentation associated with the SDK, or any portion thereof (the "Documentation"), as well as any updates or upgrades to the SDK and Documentation, if any, or any portion thereof, provided to Licensee at SMI's sole discretion.

By installing, downloading, copying or otherwise using the Licensed Materials, you agree to abide by the following provisions. This Agreement is displayed for you to read prior to using the Licensed Materials.

If you do not agree with these provisions, do not download, install or use the Licensed Materials.

1 License

Subject to the terms of this Agreement, SMI hereby grants and Licensee accepts a non-transferable, non-exclusive, non-assignable license without the right to sublicense to use the Licensed Materials only (i) for Licensee's operations, (ii) with regards to the SMI Eye Tracking application iViewNG™ and (iii) in accordance with the Documentation. Installation of the SDK is Licensee's sole responsibility.

2 Rights in Licensed Materials

Title to and ownership in the Licensed Materials and all proprietary rights with respect to the Licensed Materials and all copies and portions thereof, remain exclusively with SMI. The Agreement does not constitute a sale of the Licensed Materials or any portion or copy of it. Title to and ownership in Licensee's application software that makes calls to but does not contain all or any portion of the SDK remains with Licensee, but such application software may not be licensed or otherwise transferred to third parties without SMI's prior written consent.

3 Confidentiality

Licensed Materials are proprietary to SMI and constitute SMI trade secrets. Licensee shall maintain Licensed Materials in confidence and prevent their disclosure using at least the same degree of care it uses for its own trade secrets, but in no event less than a reasonable degree of care. Licensee shall not disclose Licensed Materials or any part thereof to anyone for any purpose, other than to its employees and sub-contractors for the purpose of exercising the rights expressly granted under this Agreement, provided they have in writing agreed to confidentiality obligations at least equivalent to the obligations stated herein.

4 Limited Warranty and Liability

a) The SDK is provided “as is”. b) SMI’s warranty obligations are limited to fraudulently concealed defects of the Licensed Material. c) SMI is only liable for damages caused by gross negligence or intent. d) With the exception of liability under the Product Liability Law, for defects after having given a guarantee, for fraudulently concealed defects and for personal injury, the above limitations of liability shall apply to all claims, irrespective of their legal basis, in particular to all claims based on breach of contract or tort. e) The above limitations of liability also apply in case of Licensee’s claims for damages against SMI’s employees or agents.

5 Licensee Indemnity

Licensee will defend and indemnify SMI, and hold it harmless from all costs, including attorney’s fees, arising from any claim that may be made against SMI by any third party as a result of Licensee’s use of Licensed Materials.

6 Export Restriction

Licensee will not remove or export from Germany or from the country Licensed Materials were originally shipped to by SMI or re-export from anywhere any part of the Licensed Materials or any direct product of the SDK except in compliance with all applicable export laws and regulations, including without limitation, those of the U.S. Department of Commerce.

7 Non-Waiver; Severability; Non-Assignment.

The delay or failure of either party to exercise any right provided in this Agreement shall not be deemed a waiver. If any provision of this Agreement is held invalid, all others shall remain in force. Licensee may not, in whole or in part, assign or otherwise transfer this Agreement or any of its rights or obligations hereunder.

8 Termination

This Agreement may be terminated (i) by Licensee without cause on 30 days notice; (ii) by SMI, in addition to other remedies, if Licensee fails to cure any breach of its obligations hereunder within 30 days of notice thereof; (iii) on notice by SMI if there is a transfer of twenty-five percent (25%) or more of the ownership interest in Licensee, which in good faith is not acceptable to SMI, and on notice by either party if the other party ceases to do business in the normal course, becomes insolvent, or becomes subject to any bankruptcy, insolvency, or equivalent proceedings. Upon termination by either party for any reason, Licensee shall at SMI’s instructions immediately destroy or return the Licensed Materials and all copies thereof to SMI and delete the SDK and all copies thereof from any computer on which the SDK had been installed.

9 Entire Agreement; Written Form Requirement.

There are no separate oral agreements; any supplementary agreements or modifications hereto must

be made in writing. This also applies to any waiver of this requirement of written form.

10 Notices

All notices under the Agreement must be in writing and shall be delivered by hand or by overnight courier to the addresses of the parties set forth above.

11 Applicable Law and Jurisdiction

German law applies with the exception of its conflict of laws rules. The application of the United Nations Convention on Contracts for the International Sale of Goods (CISG) is expressly excluded. The courts of Berlin, Germany, shall have exclusive jurisdiction for any action brought under or in connection with this Agreement.

© SensoMotoric Instruments GmbH, 2004-2013, Teltow, Germany

iViewNG SDK Examples License

Source code examples which are included in the iViewNG SDK are published under this license:

© SensoMotoric Instruments GmbH, 2004-2013, Teltow, Germany

Permission is hereby granted, free of charge, to any person or organization obtaining a copy of the software and accompanying documentation covered by this license (the "Software") to use, reproduce, display, distribute, execute, and transmit the Software, and to prepare derivative works of the Software, and to permit third-parties to whom the Software is furnished to do so, all subject to the following:

The copyright notices in the Software and this entire statement, including the above license grant, this restriction and the following disclaimer, must be included in all copies of the Software, in whole or in part, and all derivative works of the Software, unless such copies or derivative works are solely in the form of machine-executable object code generated by a source language processor.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT. IN NO EVENT SHALL THE COPYRIGHT HOLDERS OR ANYONE DISTRIBUTING THE SOFTWARE BE LIABLE FOR ANY DAMAGES OR OTHER LIABILITY, WHETHER IN CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

6.2 3rd Party Licenses

OpenCV License

The *OpenCV* binaries and the data structure *IplImage*, used in the iViewNG API, is licensed under the OpenCV license:

IMPORTANT: READ BEFORE DOWNLOADING, COPYING, INSTALLING OR USING.

By downloading, copying, installing or using the software you agree to this license. If you do not agree to this license, do not download, install, copy or use the software.

```
                Intel License Agreement
        For Open Source Computer Vision Library
```

Copyright (C) 2000-2006, Intel Corporation, all rights reserved. Third party copyrights are property of their respective owners.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistribution's of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistribution's in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- The name of Intel Corporation may not be used to endorse or promote products derived from this software without specific prior written permission.

This software is provided by the copyright holders and contributors "as is" and any express or implied warranties, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose are disclaimed. In no event shall the Intel Corporation or contributors be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) however caused and on any theory of liability, whether in contract, strict liability, or tort (including negligence or otherwise) arising in any way out of the use of this software, even if advised of the possibility of such damage.

Lesser General Public License

iViewETG software uses software licensed under the GNU Lesser General Public License (LGPL), section 7.2.1 contains the license text:.

- Eigen, see http://eigen.tuxfamily.org
- FFmpeg, see http://ffmpeg.org/

www.gnu.org/licenses/lgpl-3.0.en.html

License Text

Version 3, 29 June 2007

Copyright © 2007 Free Software Foundation, Inc. http://fsf.org/

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

This version of the GNU Lesser General Public License incorporates the terms and conditions of version 3 of the GNU General Public License, supplemented by the additional permissions listed below.

1. Additional Definitions. As used herein, “this License” refers to version 3 of the GNU Lesser General Public License, and the “GNU GPL” refers to version 3 of the GNU General Public License.“-The Library” refers to a covered work governed by this License, other than an Application or a Combined Work as defined below. An “Application” is any work that makes use of an interface provided by the Library, but which is not otherwise based on the Library. Defining a subclass of a class defined by the Library is deemed a mode of using an interface provided by the Library. A “Combined Work” is a work produced by combining or linking an Application with the Library. The particular version of the Library with which the Combined Work was made is also called the “Linked Version”.The “Minimal Corresponding Source” for a Combined Work means the Corresponding Source for the Combined Work, excluding any source code for portions of the Combined Work that, considered in isolation, are based on the Application, and not on the Linked Version. The “-Corresponding Application Code” for a Combined Work means the object code and/or source code for the Application, including any data and utility programs needed for reproducing the Combined Work from the Application, but excluding the System Libraries of the Combined Work.

1. Exception to Section 3 of the GNU GPL. You may convey a covered work under sections 3 and 4 of this License without being bound by section 3 of the GNU GPL.

1. Conveying Modified Versions. If you modify a copy of the Library, and, in your modifications, a facility refers to a function or data to be supplied by an Application that uses the facility (other than as an argument passed when the facility is invoked), then you may convey a copy of the modified version:

a) under this License, provided that you make a good faith effort to ensure that, in the event an Application does not supply the function or data, the facility still operates, and performs whatever part of its purpose remains meaningful, or

b) under the GNU GPL, with none of the additional permissions of this License applicable to that copy.

1. Object Code Incorporating Material from Library Header Files. The object code form of an Application may incorporate material from a header file that is part of the Library. You may convey such object code under terms of your choice, provided that, if the incorporated material is not limited to numerical parameters, data structure layouts and accessors, or small macros, inline functions and templates (ten or fewer lines in length), you do both of the following:

a) Give prominent notice with each copy of the object code that the Library is used in it and that the Library and its use are covered by this License.

b) Accompany the object code with a copy of the GNU GPL and this license document.

1. Combined Works. You may convey a Combined Work under terms of your choice that, taken together, effectively do not restrict modification of the portions of the Library contained in the Combined Work and reverse engineering for debugging such modifications, if you also do each of the following:

a) Give prominent notice with each copy of the Combined Work that the Library is used in it and that the Library and its use are covered by this License.

b) Accompany the Combined Work with a copy of the GNU GPL and this license document.

c) For a Combined Work that displays copyright notices during execution, include the copyright notice for the Library among these notices, as well as a reference directing the user to the copies of the GNU GPL and thislicense document.

d) Do one of the following

- Convey the Minimal Corresponding Source under the terms of this License, and the Corresponding Application Code in a form suitable for, and under terms that permit, the user to recombine or relink the Application with a modified version of the Linked Version to produce a modified Combined Work, in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.

- Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (a) uses at run time a copy of the Library already present on the user's computer system, and (b) will operate properly with a modified version of the Library that is interface-compatible with the Linked Version.

e) Provide Installation Information, but only if you would otherwise be required to provide such information under section 6 of the GNU GPL, and only to the extent that such information is necessary to install and execute a modified version of the Combined Work produced by recombining or relinking the Application with a modified version of the Linked Version. (If you use option 4d0, the Installation Information must accompany the Minimal Corresponding Source and Corresponding Application Code. If you use option 4d1, you must provide the Installation Information in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.)

1. Combined Libraries. You may place library facilities that are a work based on the Library side by side in a single library together with other library facilities that are not Applications and are not covered by this License, and convey such a combined library under terms of your choice, if you do both of the following:

a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities, conveyed under the terms of this License.

b) Give prominent notice with the combined library that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

1. Revised Versions of the GNU Lesser General Public License. The Free Software Foundation may publish revised and/or new versions of the GNU Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library as you received it specifies that a certain numbered version of the GNU Lesser General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that published version or of any later version published by the Free Software Foundation. If the Library as you received it does not specify a version number of the GNU Lesser General Public License, you may choose any version of the GNU Lesser General Public License ever published by the Free Software Foundation.

If the Library as you received it specifies that a proxy can decide whether future versions of the GNU Lesser General Public License shall apply, that proxy's public statement of acceptance of any version is permanent authorization for you to choose that version for the Library.

Boost Software License

The iViewNG software uses software licensed under the Boost Software License, section 7.2.2 contains the license text:

- Boost, see http://www.boost.org

http://www.boost.org/LICENSE_1_0.txt

License Text

Boost Software License - Version 1.0 - August 17th, 2003

Permission is hereby granted, free of charge, to any person or organization obtaining a copy of the software and accompanying documentation covered by this license (the "Software") to use, reproduce, display, distribute, execute, and transmit the Software, and to prepare derivative works of the Software, and to permit third-parties to whom the Software is furnished to do so, all subject to the following:

The copyright notices in the Software and this entire statement, including the above license grant, this restriction and the following disclaimer, must be included in all copies of the Software, in whole or in part, and all derivative works of the Software, unless such copies or derivative works are solely in the form of machine-executable object code generated by a source language processor.

zlib License

The iViewETG software uses software licensed under the zlib License:

- TinyXml, see http://www.grinninglizard.com/tinyxmldocs/index.html

License Text

This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.

Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgment in the product documentation would be appreciated but is not required.

1. Altered source versions must be plainly marked as such, and must not be misrepresented as being the original software.

1. This notice may not be removed or altered from any source distribution.

BSD License

The iViewETG software uses software licensed under the BSD License, section 7.2.3 contains the license text:

- OpenCV, see http://opencv.willowgarage.com

http://opensource.org/licenses/bsd-license.php

License Text

License Agreement

For Open Source Computer Vision Library Copyright (C) 2000-2008, Intel Corporation, all rights reserved. Copyright (C) 2008-2010, Willow Garage Inc., all rights reserved. Third party copyrights are property of their respective owners.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistribution's of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

- Redistribution's in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

- The name of the copyright holders may not be used to endorse or promote products derived from this software without specific prior written permission.

This software is provided by the copyright holders and contributors "as is" and any express or implied warranties, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose are disclaimed. In no event shall the Intel Corporation or contributors be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) however caused and on any theory of liability, whether in contract, strict liability, or tort (including negligence or otherwise) arising in any way out of the use of this software, even if advised of the possibility of such damage.

GNU General Public License (GPL) with exception described in the GCC Runtime Library Exception

The iViewETG software uses the software licensed under the GNU General Public License (GPL) version 3, with the addition under section 7 of an exception described in the GCC Runtime Library Exception, version 3.1. Sections 7.2.4 and 7.2.5 contain the license text:

- libgcc and libstdc++, see http://gcc.gnu.org/onlinedocs/libstdc++

GNU General Public License

GNU General Public License version 3 Top Previous Next Version 3, 29 June 2007

Copyright © 2007 Free Software Foundation, Inc. http://www.fsf.org/

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

PREAMBLE

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a programm to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the

area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

1. Definitions. "This License" refers to version 3 of the GNU General Public License. “Copyright” also means copyright-like laws that apply to other kinds of works, such as semiconductor masks. “The Program” refers to any copyrightable work licensed under this License. Each licensee is addressed as “you”. “Licensees” and “recipients” may be individuals or organizations. To “modify” a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a “modified version” of the earlier work or a work “based on” the earlier work. A “covered work” means either the unmodified Program or a work based on the Program. To “propagate” a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well. To “convey” a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying. An interactive user interface displays “Appropriate Legal Notices” to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code. The “source code” for a work means the preferred form of the work for making modifications to it. “Object code” means any non-source form of a work. A “Standard Interface” means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language. The “System Libraries” of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A “Major Component”, in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it. The “Corresponding Source” for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object

code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work. The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source. The Corresponding Source for a work in source code form is that same work.

1. Basic Permissions. All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law. You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you. Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

1. Protecting Users' Legal Rights From Anti-Circumvention Law. No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures. When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

1. Conveying Verbatim Copies. You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program. You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

1. Conveying Modified Source Versions. You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

1. The work must carry prominent notices stating that you modified it, and giving a relevant date.

1. The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to “keep intact all notices”.

1. You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

1. If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an “aggregate” if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation’s users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

1. Conveying Non-Source Forms. You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

1. Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.

1. Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.

1. Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.

1. onvey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the

object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.

1. Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A “User Product” is either (1) a “consumer product”, which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, “normally used” refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product. “-Installation Information” for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

1. Additional Terms. "Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that

they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions. When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission. Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

1. Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or

1. Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or

1. Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

1. Limiting the use for publicity purposes of names of licensors or authors of the material; or

1. Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

1. Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered “further restrictions” within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying. If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms. Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

1. Termination. You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11). However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice. Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section.

1. Acceptance Not Required for Having Copies. You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

1. Automatic Licensing of Downstream Recipients. Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License. An “entity transaction” is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party’s predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts. You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

1. Patents. A “contributor” is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor’s “contributor version”.

A contributor’s “essential patent claims” are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, “control” includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor’s essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version. In the following three paragraphs, a “patent license” is any express agreement or commitment, however denominated, not to enforce a patent (such as an

express permission to practice a patent or covenant not to sue for patent infringement). To “grant” such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. “Knowingly relying” means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient’s use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid. If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is “discriminatory” if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

1. No Surrender of Others’ Freedom. If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

1. Use with the GNU Affero Gnereral Public License. Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

1. Revised Versions of this License. The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar

in spirit to the present version, but may differ in detail to address new problems or concerns. Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation. If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program. Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

1. Disclaimer of Warranty. THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

1. Limitation of Liability. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

1. Interpretation of Sections 15 and 16. If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

GCC Runtime Library Exception

http://gcc.gnu.org/onlinedocs/libstdc++/manual/license.html

GCC RUNTIME LIBRARY EXCEPTION

Version 3.1, 31 March 2009

Copyright (C) 2009 Free Software Foundation, Inc.

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

This GCC Runtime Library Exception ("Exception") is an additional permission under section 7 of the GNU General Public License, version 3 ("GPLv3"). It applies to a given file (the "Runtime Library") that bears a notice placed by the copyright holder of the file stating that the file is governed by GPLv3 along with this Exception.

When you use GCC to compile a program, GCC may combine portions of certain GCC header files and runtime libraries with the compiled program. The purpose of this Exception is to allow compilation of non-GPL (including proprietary) programs to use, in this way, the header files and runtime libraries covered by this Exception.

1. Definitions. A file is an "Independent Module" if it either requires the Runtime Library for execution after a Compilation Process, or makes use of an interface provided by the Runtime Library, but is not otherwise based on the Runtime Library. "GCC" means a version of the GNU Compiler Collection, with or without modifications, governed by version 3 (or a specified later version) of the GNU General Public License (GPL) with the option of using any subsequent versions published by the FSF. "GPL-compatible Software" is software whose conditions of propagation, modification and use would permit combination with GCC in accord with the license of GCC. "Target Code" refers to output from any compiler for a real or virtual target processor architecture, in executable form or suitable for input to an assembler, loader, linker and/or execution phase. Notwithstanding that, Target Code does not include data in any format that is used as a compiler intermediate representation, or used for producing a compiler intermediate representation. The "Compilation Process" transforms code entirely represented in non-intermediate languages designed for human-written code, and/or in Java Virtual Machine byte code, into Target Code. Thus, for example, use of source code generators and preprocessors need not be considered part of the Compilation Process, since the Compilation Process can be understood as starting with the output of the generators or preprocessors. A Compilation Process is "Eligible" if it is done using GCC, alone or with other GPL-compatible software, or if it is done without using any work based on GCC. For example, using non-GPL-compatible Software to optimize any GCC intermediate representations would not qualify as an Eligible Compilation Process.

1. Grant of Additional Permission. You have permission to propagate a work of Target Code formed by combining the Runtime Library with Independent Modules, even if such propagation would otherwise violate the terms of GPLv3, provided that all Target Code was generated by Eligible Compilation Processes. You may then convey such a combination under terms of your choice, consistent with the licensing of the Independent Modules.

1. No Weakening of GCC Copyleft. No Weakening of GCC Copyleft. The availability of this Exception does not imply any general presumption that third-party software is unaffected by the copyleft requirements of the license of GCC.

Hopefully that text is self-explanatory. If it isn't, you need to speak to your lawyer, or the Free Software Foundation.

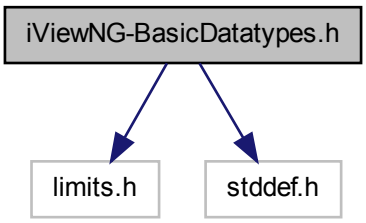
Chapter 7

Function Documentation

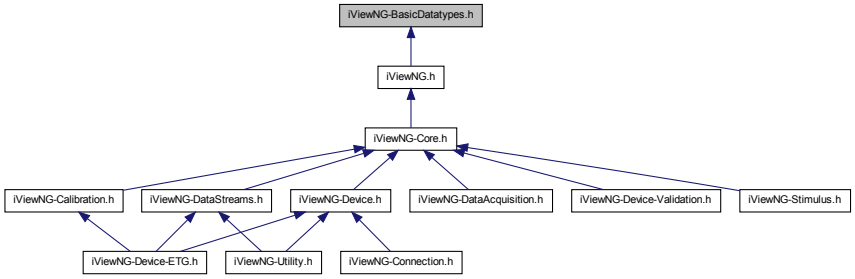
7.1 iViewNG-BasicDatatypes.h File Reference

```
#include <limits.h>
#include <stddef.h>
```

Include dependency graph for iViewNG-BasicDatatypes.h:



This graph shows which files directly or indirectly include this file:



Detailed Description

These are the basic datatypes and limits the iView SDK uses. Most compilers should have these datatypes defined properly. If required, please modify this file locally to assert that your compiler uses the types as intended.

For MinGW (gcc on Windows), the datatypes are defined in stdint.h, limits in limits.h.

Macro Definition Documentation

#define UINT_MAX 0xffffffff

Typedef Documentation

typedef short int16_t

16 bit datatypes

typedef int int32_t

32 bit datatypes

typedef long long int64_t

64 bit datatypes

typedef signed char int8_t

8 bit datatypes

typedef unsigned short uint16_t

typedef unsigned uint32_t

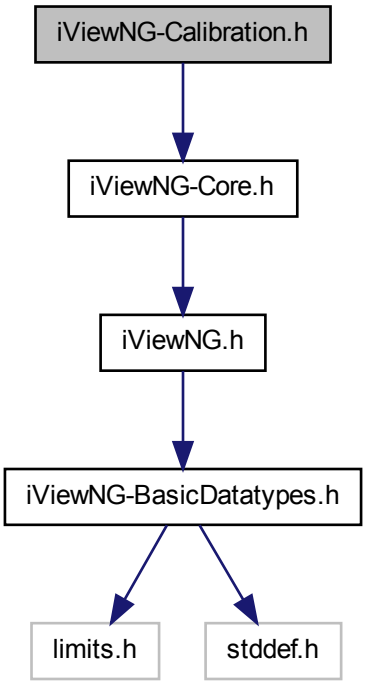
typedef unsigned long long uint64_t

typedef unsigned char uint8_t

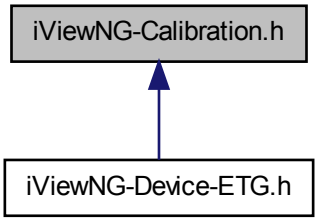
7.2 iViewNG-Calibration.h File Reference

```
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-Calibration.h:



This graph shows which files directly or indirectly include this file:



Detailed Description

This file declares the iViewNG API functions and data types which are used to realize the calibration of an eye tracking device. This includes only the device-independent part. Device-specific definitions are located in the according device header file.

Data Structure Documentation

struct iViewCalibrationParameters

When calling iView_SetupCalibration(), the user must provide information on the requested calibration type and parameters.

Data Fields

int32_t	applyAtFrame	Only meaningful for offline calibration: The scene frame index at which the calibration shall be applied. Use a value of -1 if the frame index is not known / not required and the server shall guess the proper value.
void *	parameters	Calibration scheme dependent data structure. Cast this according to the type, for details see the file iViewNG-Device-*.h.
iView-Calibration-Type	type	The calibration method to use.

struct iViewCalibrationData

Data Fields

iView-Calibration-PointId	id	**Note** irrelevant for ETG
void *	parameters	
int32_t	sceneFrame-Number	The number of the scene frame (taken from SmiImage structure) at which the calibration was performed. If set to -1, the frame number is unknown and the server may make an educated guess on its own, e.g. use the frame number of the last acquired image. **See Also** iView_SubscribeDataStream Subscribing to IVIEWDATA-STREAM_SCENEIMAGES_WITH_GAZE or IVIEWDATAS-TREAM_SCENEIMAGES delivers iViewDataStreamScene-Image structures which contain the frame number to specify here. If type is IVIEWCALIBRATIONDATATYPE_2POINT this value is ignored when passed to the Server. When returned, this value is the frame at which the calibration happened.
iView-Calibration-DataType	type	The type of the calibration data.
iView-CalibrationUnit	unit	The unit in which the coordinates are given. **Note** irrelevant for ETG
iView-Calibration-PointUsage	usage	If the usage is _INVALID and the id is valid this point should be removed from the calibration. **Note** irrelevant for ETG

struct iViewCalibrationData1Point

Data Fields

int32_t	coordinateX	The horizontal position of the mouse click, in image pixel size, zero is leftmost column.
int32_t	coordinateY	The vertical position of the mouse click, in image pixel size, zero is topmost row.

Typedef Documentation

typedef uint32_t iViewCalibrationDataType

The type of the calibration data; not all types are supported by all devices.

typedef uint32_t iViewCalibrationPointId

Each calibration point has a unique id of this type.

typedef uint32_t iViewCalibrationPointUsage

Identifier for different usages of a calibration point (bit mask, may be combined to enumerate types).

typedef uint32_t iViewCalibrationType

Identifier for different calibration method (bit mask, may be combined to enumerate types).

typedef uint32_t iViewCalibrationUnit

The unit of the coordinates in a calibration data.

Function Documentation

iViewRC iView_CancelCalibration (iViewTicket * *ticket*)

Cancel an active calibration procedure.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The calibration parameters were supplied by iView_SetupCalibration().
The calibration procedure was started by calling iView_StartCalibration().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

The successful delivery of a cancel request will set the ticket associated with iView_StartCalibration() into the state IVIEWTICKETSTATUS_CLOSED and its return code to RC_OPERATION_FAILED.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_STATE	if iView_SetupCalibration() was not called in advance
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_SetCalibrationData (iViewTicket * *ticket*, iViewCalibrationData const *const *data*)

Send calibration data for a single calibration point to the server.

The proper number of calibration data sets depends on the calibration type requested by iView_SetupCalibration(). A 1-point-calibration requires one data set, a 3-point-calibration means the calibration is finished after calling iView_SetCalibrationData() three times etc. The client application must assert that either the proper number of calibration data structures is delivered by iView_SetCalibrationData() or the calibration procedure is canceled by iView_CancelCalibration().

Providing calibration data for a multi-point-calibration may have an immediate impact on the calculated

gaze vector, even if the proper number of calibration data sets was not yet provided. The iView server may apply a calibration scheme which optimizes the gaze vector with the amount of calibration data structures given. Subsequent calls of iView_SetCalibrationData() will then further improve the calibration impact.

Sending invalid data may cancel a calibration procedure and acknowledge the failure to the API user.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The calibration parameters were supplied by iView_SetupCalibration().
The calibration procedure was started by calling iView_StartCalibration().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

The successful delivery of calibration data will set the ticket associated with iView_StartCalibration() into the state IVIEWTICKETSTATUS_CLOSED and its return code to RC_NO_ERROR.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_-ReleaseTicket() instead!
`in`	*data*	Pointer to the parameter datastructure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INVALID_STATE	if iView_SetupCalibration() was not called in advance
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_SetupCalibration (iViewTicket * *ticket*, iViewCalibrationParameters const *const *setup*)

Initialize calibration procedure but do not start it.

Use this function to submit the calibration parameters to the iViewNG server. This function call specifies

- which calibration type to use (required)
- and when to apply the calibration (optional, only meaningful for offline calibration).

This function may be called at any time when a calibration procedure was not already started with iView_StartCalibration() and not yet terminated by calling iView_SetCalibrationData() or iView_CancelCalibration().

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,

  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
in	*setup*	Pointer to parameter data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_StartCalibration (iViewTicket * *ticket*)

Start the calibration procedure.

Call this function to indicate that a calibration procedure is in progress. This means, the function iView_SetCalibrationData() will be called soon after.

When iView_StartCalibration() is called, the server might start to buffer calculated gaze samples. To save memory and CPU time, please finish a started calibration procedure by providing the proper amount of calibration data points (the number of required points depends on the calibration method submitted with iView_SetupCalibration()) via iView_SetCalibrationData() or by canceling the procedure via iView_CancelCalibration().

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The calibration parameters were supplied by iView_SetupCalibration().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_INVALID_STATE	if iView_SetupCalibration() was not called in advance
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

Variable Documentation

const iViewCalibrationDataType IVIEWCALIBRATIONDATATYPE_1POINT = 1 [static]

a fixed point

const iViewCalibrationDataType IVIEWCALIBRATIONDATATYPE_INVALID = 0 [static]

const iViewCalibrationType IVIEWCALIBRATIONTYPE_INVALID = 0 [static]

Invalid calibration scheme to detect unset data structure elements.

const iViewCalibrationType IVIEWCALIBRATIONTYPE_NONE = 1 [static]

Default calibration scheme: no calibration. On some systems, this might mean "0-point-calibration".

const iViewCalibrationUnit IVIEWCALIBRATIONUNIT_INVALID = 0 [static]

do not use

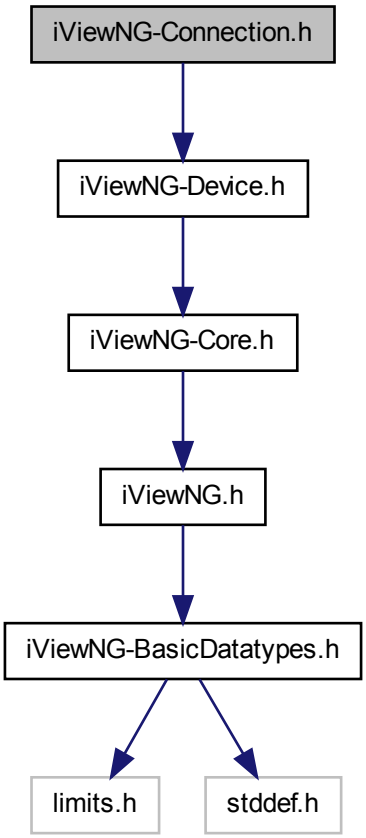
const iViewCalibrationUnit IVIEWCALIBRATIONUNIT_PIXEL = 1 [static]

Calibration coordinates are given in pixels.

7.3 iViewNG-Connection.h File Reference

`#include <iViewNG-Device.h>`

Include dependency graph for iViewNG-Connection.h:



Detailed Description

This file declares the iViewNG API functions and data types which are required for connecting to an iViewNG server.

This file declares the iViewNG API functions and data types which are required for handling data acquisition as well as storing acquired data by the server.

Data Structure Documentation

struct iViewHostAddress

Datastructure which represents the location of a remote iViewNG server instance.

Data Fields

union iView-HostAddress	ipAddress	
unsigned short	port	Port for establishing the connection. If set to zero, the default port (1100) is used. Do not use anything but zero unless you explicitly start the iViewNG server with a different port number!

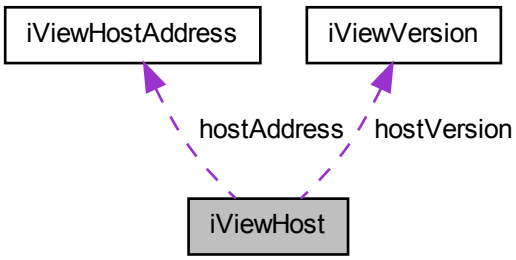
struct iViewHost

Data structure which represents one iViewNG server instance which is willing to provide access to one SMI device. If one iViewNG server is able to provide access to more than one SMI device, each device will be listed as one iViewHost.

Note

Generate via iView_LocateServers() to get all elements filled with proper values.
If not allocated manyally, do not release the iViewHost structure manually, use iView_ReleaseiView-Host() instead.

Collaboration diagram for iViewHost:



Data Fields

void *	capability	A set of provided/requested capabilities for this device. Device-dependent. Not yet implemented.
iView-Connection-Type	connection-Type	State the type of the connection.
iViewDevice-Type	device	The SMI device which is provided/requested. Not yet implemented.

iViewHost-Address	hostAddress	The IP address & UDP/TCP port of a iViewNG server instance, if iViewHost.type implies network communication. Irrelevant for IVIEW_SERVERADRRESS_SHAREDMEMORY communication type.
wchar_t	hostId	An optional string which the server may use to identify itself.
iViewVersion	hostVersion	Provided/requested software version on host. Not yet implemented.

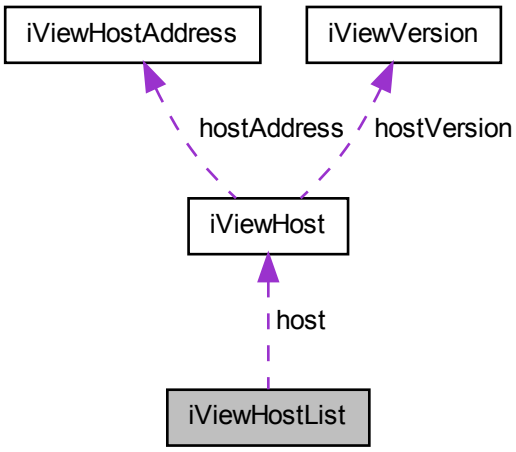
struct _iViewHostListStruct

Data structure which contains a list of iViewNG server address structures.

Note

Do not create the iViewHostList structure manually. Generate via iView_LocateServers().
Do not release the iViewHostList structure manually, use iView_ReleaseiViewHostList() instead.

Collaboration diagram for iViewHostList:



Data Fields

iViewHost	host	Server host structure.
iViewHostList *	next	If more than one server is returned, this member points to the next structure. Defaults to NULL.

union iViewHostAddress.ipAddress

The server host address, either IPv4 or IPv6.

Data Fields

char	ipv4	
char	ipv6	The server host address, IPv4 version.

Typedef Documentation

typedef unsigned int iViewConnectionType

Type to identify IP address types.

typedef uint32_t iViewLocationScope

Bit mask for defining restrictions for iView_LocateServers().

Function Documentation

iViewRC iView_Connect (iViewTicket *const *ticket*, iViewHost const *const *server*, const uint32_t *timeout*, const uint32_t *attempts*, iViewHost const *const *client*)

Immediately return when one server matching the requirements is found. First come, first served. Do not continue to search. Connect to the specified iViewNG server.

This function blocks the caller until a connection was established or until the operation timed out. The ticket state remains IVIEWTICKETSTATUS_IN_PROGRESS until the connection was lost. When this happens, the ticket state will be set to IVIEWTICKETSTATUS_CLOSED, the progress is set to 100 and the return code is set to RC_CONNECTION_FAILED.

Before terminating, an established connection shall be closed using iView_Disconnect().

Precondition

The iView C API was initialized with iView_Init().

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS
- iViewTicket.progress is 100.
  - On success:

iViewTicket.returnCode is RC_NO_ERROR,

- iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS as long as the connection is alive. When the connection was lost, this is delivered as error.
  - On failure:
    - iViewTicket.returnCode indicates the error code,
- iViewTicket.result is NULL or a pointer to iViewServerError.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
`in`	*server*	Specification of the server. Mandatory.
`in`	*timeout*	Do not try to connect to server for more than this time span [milliseconds]. It is recommended to wait not less than 1 seconds and retry 2 times.
`in`	*attempts*	Number of attempts to retry initiating a connection when the first attempt failed. Each attempt will approximately consume the time span defined by 'timeout'. When all attempts fail, the function will return indicating the failure. It is recommended to wait not less than 1 seconds and retry 2 times.
`in`	*client*	Specification of the client. Optional for legacy communication, not used for iVNG communication.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if the ticket data structure is invalid
RC_INVALID_PARAMETER	if any of the parameters are invalid
RC_INVALID_STATE	if the server was already connected
RC_OPERATION_FAILED	if the connection attempt failed (e.g. because of timeout)
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_UNSPECIFIED	on any other error

iViewRC iView_Disconnect ( )

Cut the connection to the iViewNG server. Does not block pending events which may still arrive when this function completed.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

All pending tickets will be set to state IVIEWTICKETSTATUS_CLOSED, the return code is set to RC_NO_ERROR.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_UNSPECIFIED	on any other error

Variable Documentation

const uint8_t HOSTADDRESSLENGTH_IPV4 = 16 [static]

const uint8_t HOSTADDRESSLENGTH_IPV6 = 24 [static]

const iViewLocationScope IVIEW_LOCATE_FIRST_SERVER_ONLY = 0x100 [static]

Search the network for servers.

const iViewLocationScope IVIEW_LOCATE_LOCALHOST_ONLY = 0x1 [static]

const iViewLocationScope IVIEW_LOCATE_NETWORK = 0x2 [static]

Restrict location to localhost only. Do not search the network.

const iViewConnectionType IVIEW_SERVERADRRESS_INVALID = 0 [static]

IP address types: invalid address.

const iViewConnectionType IVIEW_SERVERADRRESS_IPV4 = 1 `[static]`

IP address types: IPv4 address.

const iViewConnectionType IVIEW_SERVERADRRESS_IPV6 = 2 `[static]`

IP address types: IPv6 address.

const iViewConnectionType IVIEW_SERVERADRRESS_SHAREDMEMORY = 3 `[static]`

Shared memory communication to localhost.

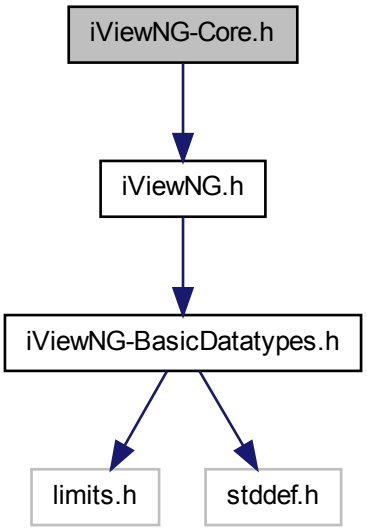
const iViewConnectionType IVIEW_SERVERADRRESS_UNIX_DOMAIN_SOCKET = 4 `[static]`

Unix domain socket communication for inter-process communication. Not usable under MS Windows.

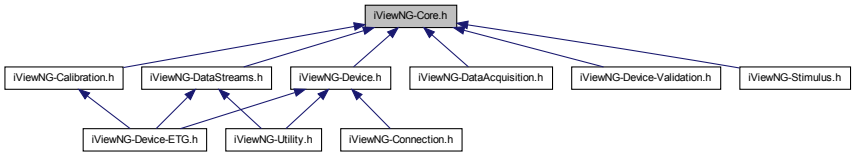
7.4 iViewNG-Core.h File Reference

```
#include <iViewNG.h>
```

Include dependency graph for iViewNG-Core.h:



This graph shows which files directly or indirectly include this file:



Detailed Description

This file declares the iViewNG API functions and data types which are considered the 'core' of the API: initializing and deinitializing the client and server, fundamental interaction with the iViewNG API.

Data Structure Documentation

struct iViewVersion

Data structure to represent a software version.

Data Fields

uint32_t	build	The build number.
uint32_t	major	The major version number.
uint32_t	minor	The minor version number.
uint32_t	patch	The patch version number.

struct iViewTime

Data structure to represent a time scale.

Data Fields

uint64_t	time	Absolute time in nanoseconds.

struct iViewDate

Data structure to represent a date.

Data Fields

uint32_t	day	Number of the day of the month, element of the interval [1..31].
uint32_t	month	Number of the month, element of the interval [1..12].
uint32_t	year	Year A.D. with 4 digits, e.g. 2010.

struct iViewServerError

Data structure for error messages which also include a description.

Data Fields

iViewRC	errorCode	The server may generate a sophisticated error code to further declare what happened. Defaults to RC_UNSPECIFIED.
wchar_t *	message	A null-terminated human-readable message.

struct _iViewResultStruct

Every iViewTicket structure may contain a iViewResult data structure. This result structure is used to submit data back to the user application. To be able to determine the result data type, each iViewResult contains the data type of the included result. Use iViewResult.type to determine how to cast iViewResult.data.

Data Fields

void *	data	The result data. Use the iViewResult.type to determine how to cast.
iViewResult *	next	If more than one result is submitted at once, this is a pointer to the next result structure. Defaults to NULL (no more data).
iViewResultType	type	The type of the data structure data points to.

struct iViewTicket

Datastructure for managing responses to API requests.

Never allocate an iViewTicket structure on your own, use iView_CreateTicket() instead.

Whenever an API function is called, the caller provides a pointer to a (not necessarily initialized) iViewTicket structure. The iViewNG API writes a unique ticket number into the data structure and tags the request according to the processing state. The caller may at any time check if the request was already answered by examining the state indicator. While this flag is not set to IVIEWTICKETSTATUS_IN_PROGRESS, the caller must not release the memory or modify its contents.

Upon completion, iViewNG may call the callback function with the iViewTicket as parameter. At this time, the requestPending flag is set to IVIEWTICKETSTATUS_IN_PROGRESS. If the called API function was supposed to return data, the member 'result' is set to the result data structure.

The caller of the API functions is then responsible for explicitly indicating to release all data structures! iViewNG will not release neither iViewTicket structures nor the result data structure without the caller explicitly requesting to release the data! The user must call iView_ReleaseTicket() to let iViewNG release the iViewTicket structure and all contained structures.

Attention

Do not create the iViewTicket on your stack! Assert that the ticket memory is not explicitly or implicitly released or manipulated. This would corrupt your memory! If no longer required, call iView_release() to explicitly release the ticket data structure!

Data Fields

wchar_t	functionName	The name of the function call which was used for this ticket. There may be more than one ticket with this function name. Use this only for debugging purposes.
iViewTicket-Progress	progress	The progress of data acquisition. Not all command functions provide information about the progress, see the function documentation.
iViewResult *	result	Pointer to the data returned by iView. Cast to iViewResult* if the return code indicates that the operation succeeded. Otherwise, cast to iViewServerError* to find an explanation.
iViewRC	returnCode	A return code to indicate the result of the associated operation. Defaults to RC_NO_ERROR. A value different to RC_NO_ERROR indicates that 'result' points to an iViewServerError data structure if result != NULL. Otherwise, 'result' points to a iViewResult data structure.
iViewTicket-State	state	The processing state of the ticket. Enumeration.
iViewTicket-Number	ticketNumber	The unique number of the request. Use this number to identify tickets. There will never be more than one ticket with this ticket number.

Typedef Documentation

typedef void(* IVIEW_CALLBACK)(iViewTicket *const)

Register the user callback function. The user callback function will be called asynchronously to provide any data to the user. A proper callback function is required for receiving data asynchronously.

It is possible to set the callback to NULL to indicate that no asynchronous results shall be delivered anymore and that all results shall be dismissed immediately.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*callback*	Pointer to the callback function. Use NULL to indicate that no results shall be delivered anymore and that all results shall be dismissed immediately.

Return values

RC_NO_ERROR	on success
RC_UNSPECIFIED	on any other error

typedef uint32_t iViewResultType

The type of the data structure which is addressed by iViewResult.data. To be able to determine the result data type, each iViewResult contains the data type of the included result. Use iViewResult.type to determine how to cast iViewResult.data.

typedef uint32_t iViewSdkType

Declaration of the SDK communication scheme.

typedef uint32_t iViewTicketNumber

Every iViewTicket contains a unique ticket number. Ticket numbers are managed by the iView API and must not be modified by the client.

typedef uint8_t iViewTicketProgress

Every iViewTicket contains a progress indicator to provide the state of request progress with a percentage scale from 0 to 100 (request finished). Not all data requests provide information about the progress, see the function documentation.

typedef uint8_t iViewTicketState

Every iViewTicket contains a ticket state to indicate the current state of processing the request. The state describes the state of the API request itself, not the state of possibly collecting data from the iView server.

Note

Most API functions adjust the ticket state only after performing some initial checks, e.g. the system initialization. The purpose of the state is to indicate the request state based on asynchronous feedback from the server.

Function Documentation

iViewRC iView_AddLicenseKey (iViewTicket *const *ticket*, char const *const *license*)

Provide user license key to iView NG. The license key is used to check the availability of functionality. It is possible to provide more than one license key, all license keys will be stored. If there is no connection to the server, the license(s) will be transferred implicitly when connecting to the server.

Precondition

The iView C API was initialized with iView_Init().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:

iViewTicket.returnCode is RC_NO_ERROR,

- iViewTicket.result is NULL.

On failure:

- iViewTicket.returnCode indicates the error code,
- iViewTicket.result is NULL or a pointer to iViewServerError.

iViewTicket.progress is 100.

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_-ReleaseTicket() instead!
`in`	*license*	The license string as provided to the user.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_PARAME-TER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred

RC_UNSPECIFIED	on any other error

iViewRC iView_CreateTicket (iViewTicket **const *ticket*)

Dynamically generate a ticket data structure.

iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. Use iView_ReleaseTicket() to release the data structure and its contents. Do not release any contained data before calling iView_ReleaseTicket()!

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Precondition

The iView C API was initialized with iView_Init().

Parameters

`in,out`	*ticket*	Pointer to the iViewTicket data structure to initialize.

Return values

RC_NO_ERROR	on success
RC_INVALID_PARAMETER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	if the data structure could not be allocated
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

IVIEW_CALLBACK iView_GetCallback ( )

Retrieve the address of the registered user callback function.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Returns

Pointer to the callback function registered with iView_SetCallback(), or NULL if no callback function was registered.

iViewRC iView_GetLibraryVersion (iViewVersion * *version*)

Retrieve the iViewNG library version numbers and write to provided structure.

This function may be called any time, even before calling iView_Init().

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

`in,out`	*version*	User-provided data structure to be filled.

Return values

RC_NO_ERROR	on success
RC_INVALID_PARAMETER	if the parameter is invalid (e.g. does not point to an initialized datastructure)

iViewRC iView_GetLocalTime (iViewTicket *const *ticket*)

Retrieve the current time from the local host. Use this function for fetching the local time scale and for user-level synchronization of server and client application time scale.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Precondition

The iView C API was initialized with iView_Init().

This function returns synchronously and provides the result of the operation. Use the ticket to retrieve the result. On return, the ticket's elements are set:

- On success:

iViewTicket.returnCode is RC_NO_ERROR,

- iViewTicket.result is a pointer to iViewResult,

iViewResult.type is IVIEWRESULT_GET_LOCAL_TIME,

- iViewResult.data is a pointer to iViewTime,
- iViewResult.next is NULL since there will be no subsequent results.

On failure:

- iViewTicket.returnCode indicates the error code,
- iViewTicket.result is NULL or a pointer to iViewServerError.

iViewTicket.progress is 100.

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_GetServerTime (iViewTicket *const *ticket*)

Retrieve the current time from the iViewNG server. Use this function for user-level synchronization of server and client application time scale.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:

iViewTicket.returnCode is RC_NO_ERROR,

- iViewTicket.result is a pointer to iViewResult,

iViewResult.type is IVIEWRESULT_GET_SERVER_TIME,

- iViewResult.data is a pointer to iViewTime,
- iViewResult.next is NULL since there will be no subsequent results.

On failure:

- iViewTicket.returnCode indicates the error code,
- iViewTicket.result is NULL or a pointer to iViewServerError.

iViewTicket.progress is 100.

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_Init (iViewSdkType *sdkType*)

Initialize the iViewNG C API.

This function sets up the iView C API and puts it into the state where it may try to connect to the iViewNG server. It is safe but useless to try to initialize the API more than one time. If the API is no longer needed, call iView_Shutdown().

This function does not interact with the iViewNG server (if not connected to server). This function does not produce asynchronous return values.

Parameters

in	*sdkType*	defines the communication scheme (iViewNG vs. iViewX)

Return values

RC_NO_ERROR	on success
RC_INVALID_STATE	if the API was already initialized
RC_NO_INIT	if there is an initialization error
RC_INVALID_PARAMETER	if sdkType does not address a proper SDK type
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_ReleaseResult (iViewTicket *const *ticket*)

Recursively free the ticket result. All referred data allocated by iViewNG API is released.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if ticket data structure is invalid or contains no result
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_ReleaseTicket (iViewTicket **const *ticket*)

Recursively free the ticket data structure, including the result. All referred data allocated by iViewNG API is released. The ticket pointer will be set to NULL. Before releasing memory, the structure contents will be set to zero.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if ticket data structure is invalid
RC_OPERATION_FAILED	if ticket could not be released
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_SetCallback (DLLCALL *IVIEW_CALLBACK*)

iViewRC iView_Shutdown ( )

Deinitialize the API client and release the acquired memory. If still connected, the connection will be closed in advance.

This function does not interact with the iViewNG server (if not connected to server). This function does not produce asynchronous return values.

Precondition

The iView C API was initialized with iView_Init().

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_ShutdownServer (const uint32_t *timeout*)

Terminate the iView server and release the acquired memory. This does not deinitialize the C API. Call iView_Shutdown() explicitly afterwards.

This function does not produce asynchronous return values.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Parameters

in	*timeout*	Do not wait for shutdown for more than this timeout [milliseconds]. It is recommended to wait for not less than 15 seconds.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_OPERATION_FAILED	if the server process did not terminate in time
RC_UNSPECIFIED	on any other error

iViewRC iView_StartServer (wchar_t const *const *path*, wchar_t const *const *parameters*)

Start new iViewNG server with the provided command parameter set. The function blocks the caller until it is safe to assume that the server is up and ready.

This function does not produce asynchronous return values.

Parameters

in	*path*	reserved for future use (use NULL pointer)
in	*parameters*	reserved for future use (use NULL pointer)

Return values

RC_NO_ERROR	on success
RC_INVALID_STATE	if an iViewNG server instance is already running (deprecated)
RC_OPERATION_FAILED	if the server could not be started
RC_UNSPECIFIED	on any other error

Variable Documentation

const unsigned int IVIEW_SIZE_OF_FUNCTION_NAME = 64 [static]

Constant indicating the number of characters allocated for a string in a structure.

const iViewResultType IVIEWRESULT_ADD_LICENSE_KEY = 3 [static]

Result for iView_AddLicenseKey()

const iViewResultType IVIEWRESULT_CALIBRATION_CANCEL = 16 [static]

Result for iView_CalibrationCancel()

const iViewResultType IVIEWRESULT_CALIBRATION_END = 17 [static]

Result for iView_CalibrationEnd()

const iViewResultType IVIEWRESULT_CALIBRATION_LIST = 18 [static]

Result for iView_CalibrationList()

const iViewResultType IVIEWRESULT_CALIBRATION_LOAD = 19 [static]

Result for iView_CalibrationLoad()

const iViewResultType IVIEWRESULT_CALIBRATION_SAVE = 20 [static]

Result for iView_CalibrationSave()

const iViewResultType IVIEWRESULT_CALIBRATION_SEND_DATA = 15 [static]

Result for iView_SendCalibrationData()

const iViewResultType IVIEWRESULT_CALIBRATION_SETUP = 13 [static]

Result for iView_CalibrationSetup()

const iViewResultType IVIEWRESULT_CALIBRATION_START = 14 [static]

Result for iView_CalibrationStart()

const iViewResultType IVIEWRESULT_CONNECT = 1 [static]

Result for iView_Connect()

const iViewResultType IVIEWRESULT_DATA_ACQUISITION_START = 11 [static]

Result for iView_StartDataAcquisition()

const iViewResultType IVIEWRESULT_DATA_ACQUISITION_STOP = 12 [static]

Result for iView_StopDataAcquisition()

const iViewResultType IVIEWRESULT_DISCONNECT = 2 [static]

Result for iView_Connect()

const iViewResultType IVIEWRESULT_ERROR = 1 [static]

Result for iView_Connect()

const iViewResultType IVIEWRESULT_GET_DEVICE_INFO = 4 [static]

Result for iView_QueryDeviceVersion()

const iViewResultType IVIEWRESULT_GET_DEVICE_PARAMETERS = 6 [static]

Result for iView_SetDeviceParameters()

const iViewResultType IVIEWRESULT_GET_EXPERIMENT_DATA = 26 [static]

Result for iView_SetParticipantData()

const iViewResultType IVIEWRESULT_GET_GAZE_CORRECTION_DATA = 29 [static]

Result for iView_GetGazeCorrectionData()

const iViewResultType IVIEWRESULT_GET_LOCAL_TIME = 22 [static]

Result for iView_GetServerTime()

const iViewResultType IVIEWRESULT_GET_PARTICIPANT_DATA = 24 [static]

Result for iView_SetParticipantData()

const iViewResultType IVIEWRESULT_GET_SERVER_TIME = 21 [static]

Result for iView_GetServerTime()

const iViewResultType IVIEWRESULT_INVALID = 0 [static]

Never used!

const iViewResultType IVIEWRESULT_INVALID_SPECIFIER = 37 [static]

const iViewResultType IVIEWRESULT_RECORDING_START = 9 [static]

Result for iView_StartRecording()

const iViewResultType IVIEWRESULT_RECORDING_STOP = 10 [static]

Result for iView_StartRecording()

const iViewResultType IVIEWRESULT_SAVE_MESSAGE = 31 [static]

Result for API-generated server error message.

const iViewResultType IVIEWRESULT_SERVER_ERROR = 30 [static]

Result for API-generated server error message.

const iViewResultType IVIEWRESULT_SET_DEVICE_PARAMETERS = 5 [static]

Result for iView_SetDeviceParameters()

const iViewResultType IVIEWRESULT_SET_EXPERIMENT_DATA = 25 [static]

Result for iView_SetParticipantData()

const iViewResultType IVIEWRESULT_SET_GAZE_CORRECTION_DATA = 28 [static]

Result for iView_SetGazeCorrectionData()

const iViewResultType IVIEWRESULT_SET_PARTICIPANT_DATA = 23 [static]

Result for iView_SetParticipantData()

const iViewResultType IVIEWRESULT_SHUTDOWN_SERVER = 27 [static]

Result for iView_SetParticipantData()

const iViewResultType IVIEWRESULT_SUBSCRIBE_DATASTREAM = 7 [static]

Result for iView_SubscribeDataStream()

const iViewResultType IVIEWRESULT_UNSUBSCRIBE_DATASTREAM = 8 [static]

Result for iView_SubscribeDataStream()

const iViewSdkType IVIEWSDK_INVALID = 0 [static]

Invalid value, do not use!

const iViewSdkType IVIEWSDK_IVNG = 1 [static]

Indicate iViewNG API and communication scheme.

const iViewTicketState IVIEWTICKETSTATUS_ACCEPTED = 0x04 [static]

The status of a ticket is 'accepted' when the request was recognized as valid and the request is forwarded to the iViewNG server. and until the processing of the request was not yet finished.

const iViewTicketState IVIEWTICKETSTATUS_CLOSED = 0x10 [static]

The status of a ticket is 'completed' when the request was processed and the triggered operation finished.

The status of a ticket is 'closed' if the processing of the request was finished. Most functions will call the user callback function after setting the state to 'closed'. The user shall then gather the required data from the ticket result data structure and eventually call iView_ReleaseTicket() to release allocated memory. However, streaming tickets will remain in the 'accepted' state until the stream subscription is canceled.

const iViewTicketState IVIEWTICKETSTATUS_IN_PROGRESS = 0x0a [static]

The status of a ticket is 'processed' when the request was processed but the triggered operation is still in progress. This is especially the case with server connections and data stream subscriptions.

const iViewTicketState IVIEWTICKETSTATUS_REJECTED = 0x06 [static]

The status of a ticket is 'rejected' if the request was illegal, e.g. because the ticket data structure was invalid or because the license did not provide access to that request.

const iViewTicketState IVIEWTICKETSTATUS_RELEASED = 0x14 [static]

The status of a ticket is 'abandoned' if the user called iView_ReleaseTicket() to release allocated memory. It is then (not earlier!) safe to delete the ticket data structure. A released ticket's result pointer is NULL.

const iViewTicketState IVIEWTICKETSTATUS_UNPROCESSED = 0x00 [static]

The status of a ticket is 'unprocessed' until the API starts with processing the request. This value is always the initial value.

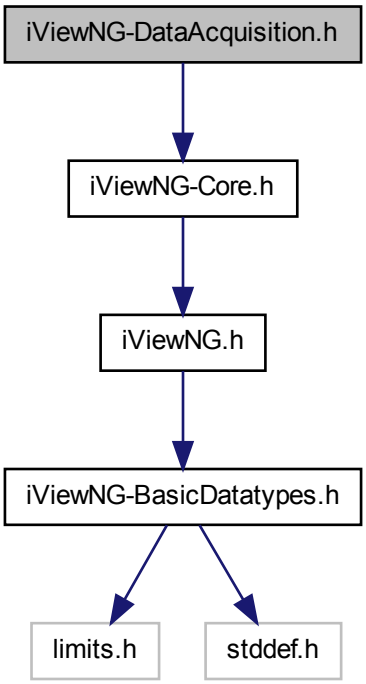
const iViewResultType NUMBER_OF_IVIEWRESULTS = 38 [static]

Never use!

7.5 iViewNG-DataAcquisition.h File Reference

`#include <iViewNG-Core.h>`

Include dependency graph for iViewNG-DataAcquisition.h:



Data Structure Documentation

struct iViewMessage

Declaration of the basic message structure. There are several message structures available, depending on the message type. The element iViewMessage.type indicates the data type of the whole structure. All message structures share the first element: use static casting to access the other elements.

Data Fields

iViewMessage-Type	type	Identify the type of this structure.

struct iViewMessageText

A message including a simple NULL-terminated text string. BeGaze will integrate the string as comment line into the exported IDF.

Data Fields

wchar_t *	text	The text to store. The user is responsible for releasing the data.
iViewMessage-Type	type	Data inherited from iViewMessage. Value must be IVIEWMESS-AGETYPE_TEXT.

Typedef Documentation

typedef uint32_t iViewMessageType

Identifier for the iViewMessage data structure.

Function Documentation

iViewRC iView_SaveMessage (iViewTicket * *ticket*, iViewMessage const * *message*)

Submit a message to the server for proper storage.

Use this function to pass additional information concerning a participant to the iViewNG server for storage. The data will be saved together with the other participant-related files. Use BeGaze's IDF data export function to integrate the recorded messages into the IDF file. The appearance of the data depends on the message type and the used eye tracking device.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Participant data was set.

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
- iViewTicket.progress is 100.

– On success:

iViewTicket.returnCode is RC_NO_ERROR,

- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
in	*message*	Pointer to a message structure. May be released immediately after calling iView_SaveMessage(). The user is responsible for releasing the data.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_STATE	if a server-side precondition is not met
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_StartDataAcquisition (iViewTicket * *ticket*)

Trigger the start of data acquisition.

The ticket progress will be set to 100% when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Device Parameters were set with iView_SetDeviceParameters().
A data stream provided by the sensor to start is acquired.

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- iViewTicket.resultType is IVIEWRESULT_DATA_ACQUISITION_START.
  - On success:
    - iViewTicket.returnCode is RC_NO_ERROR,

iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS until the data acquisition is stopped actively or due to an error.

- iViewTicket.progress is 50.

On failure:

- iViewTicket.returnCode indicates the error code,
- iViewTicket.result is NULL or a pointer to iViewServerError.

iViewTicket.state is IVIEWTICKETSTATUS_CLOSED

- iViewTicket.progress is 100.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_-ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAI-LED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_STATE	if a server-side precondition is not met
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_StartRecording (iViewTicket * *ticket*)

Ask the iViewNG server to start recording.

The ticket progress will be set to 100% when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Device Parameters were set with iView_SetDeviceParameters().
Data acquisition is in progress.

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:

iViewTicket.returnCode is RC_NO_ERROR,

- iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS until the recording is stopped actively or due to an error.
  - iViewTicket.progress is 50.

On failure:

- iViewTicket.returnCode indicates the error code,
- iViewTicket.result is NULL or a pointer to iViewServerError.

iViewTicket.state is IVIEWTICKETSTATUS_CLOSED

- iViewTicket.progress is 100.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_StopDataAcquisition (iViewTicket * *ticket*)

Trigger the end of data acquisition.

The ticket progress will be set to 100% when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Device Parameters were set with iView_SetDeviceParameters().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
- iViewTicket.progress is 100.
  - On success:

iViewTicket.returnCode is RC_NO_ERROR,

- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.

Postcondition

On success, this function will close the ticket which called iView_StartDataAcquisition();

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_StopRecording (iViewTicket * *ticket*)

Ask the iViewNG server to stop recording.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Device Parameters were set with iView_SetDeviceParameters().
Recording is in progress.

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
- iViewTicket.progress is 100.
  - On success:

iViewTicket.returnCode is RC_NO_ERROR,

- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.

Postcondition

On success, this function will close the ticket which called iView_StartRecording();

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_STATE	if a server-side precondition is not met
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

Variable Documentation

const iViewMessageType IVIEWMESSAGETYPE_INVALID = 0 [static]

Invalid type, do not use.

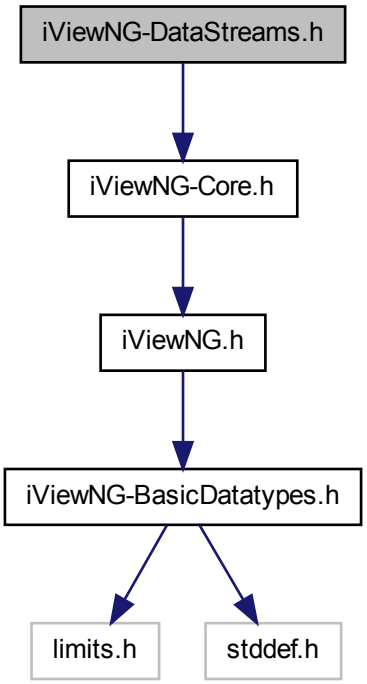
const iViewMessageType IVIEWMESSAGETYPE_TEXT = 1 [static]

The structure is a iViewMessageText structure.

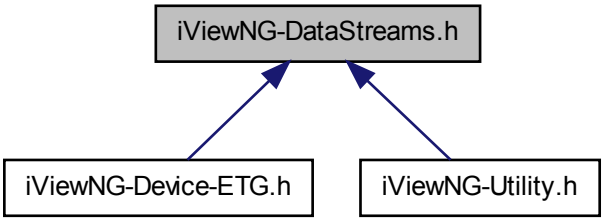
7.6 iViewNG-DataStreams.h File Reference

```
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-DataStreams.h:



This graph shows which files directly or indirectly include this file:



Detailed Description

This file declares the iViewNG API functions and data types which are required for handling data streams. Please refer to SDK documentation sections Synchronous Communication and Step 4: Receiving Gaze Data the for details.

Data Structure Documentation

struct iViewImage

The image structure iViewImage is used by iViewNG for providing non-compressed images. This structure is supposed to be compatible with the OpenCV image structure so that an iViewImage pointer can be cast to an IplImage pointer. OpenCV is published under the License Agreement For Open Source Computer Vision Library:

IMPORTANT: READ BEFORE DOWNLOADING, COPYING, INSTALLING OR USING.

By downloading, copying, installing or using the software you agree to this license. If you do not agree to this license, do not download, install, copy or use the software.

```
                License Agreement
For Open Source Computer Vision Library
```

Copyright (C) 2000-2008, Intel Corporation, all rights reserved. Copyright (C) 2009, Willow Garage Inc., all rights reserved. Third party copyrights are property of their respective owners.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistribution's of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistribution's in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- The name of the copyright holders may not be used to endorse or promote products derived from this software without specific prior written permission.

This software is provided by the copyright holders and contributors "as is" and any express or implied warranties, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose are disclaimed. In no event shall the Intel Corporation or contributors be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) however caused and on any theory of liability, whether in contract, strict liability, or tort (including negligence or otherwise) arising in any way out of the use of this software, even if advised of the possibility of such damage.

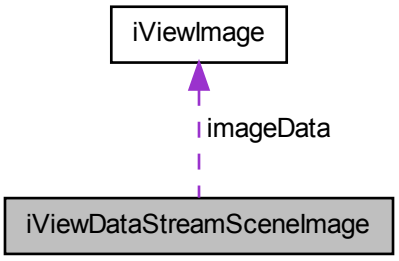
Data Fields

int32_t	align	not used
int32_t	alphaChannel	not used
int32_t	BorderConst	not used
int32_t	BorderMode	not used
int8_t	channelSeq	not used
int8_t	colorModel	not used
int32_t	dataOrder	0 - interleaved color channels, 1 - separate color channels
int32_t	dataSize	sizeof(iViewImage)
int32_t	depth	pixel depth in bits: IVIEW_DEPTH_8U, IVIEW_DEPTH_8S, IVIEW_DEPTH_16S, IVIEW_DEPTH_32S,IVIEW_DEPTH_32F and are supported
int32_t	height	image height in pixels
int32_t	id	version (=0)
int8_t *	imageData	pointer to aligned image data
int8_t *	imageDataOrigin	pointer to very origin of image data (not necessarily aligned) - needed for correct deallocation
void *	imageId	not used, always NULL
int32_t	imageSize	image data size in bytes, (== image->height * image->widthStep in case of interleaved data)
void *	maskROI	not used, always NULL
int32_t	numberOfChannels	number of colour channels
int32_t	origin	0 - top-left origin, 1 - bottom-left origin (Windows bitmaps style)
void *	roi	not used, always NULL
void *	tileInfo	not used, always NULL
int32_t	width	image width in pixels
int32_t	widthStep	size of aligned image row in bytes

struct iViewDataStreamSceneImage

When subscribing to IVIEWDATASTREAM_SCENEIMAGES, IVIEWDATASTREAM_SCENEIMAGES_WITH_GAZE or IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED, the subscription ticket returns instances of this data structure. It contains the full image scene image.

Collaboration diagram for iViewDataStreamSceneImage:



Data Fields

iViewImage *	imageData	The image structure.
uint32_t	sceneFrame-Number	The number of the frame.

struct iViewDataStreamH264SceneImage

When subscribing to IVIEWDATASTREAM_SCENEIMAGES_H264, the subscription ticket returns instances of this data structure. It contains the scene image encoded with the H.264 compression algorithm.

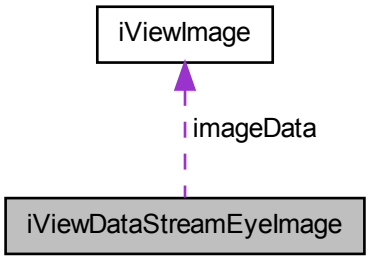
Data Fields

void *	data	The raw H.264 data.
uint32_t	dataSize	the size of H.264 data in bytes
uint32_t	isKeyFrame	1 if this frame is a key frame, 0 otherwise.
uint32_t	sceneFrame-Number	The number of the frame.

struct iViewDataStreamEyeImage

When subscribing to IVIEWDATASTREAM_EYEIMAGES_LEFT or IVIEWDATASTREAM_EYEIMAGES_RIGHT, the subscription ticket returns instances of this data structure. It contains the eye image.

Collaboration diagram for iViewDataStreamEyeImage:



Data Fields

iViewEye-Identifier	eye	
uint32_t	eyeFrame-Number	Which eye is shown?
iViewImage *	imageData	The number of the frame.

struct iViewTrackingStatus

Data

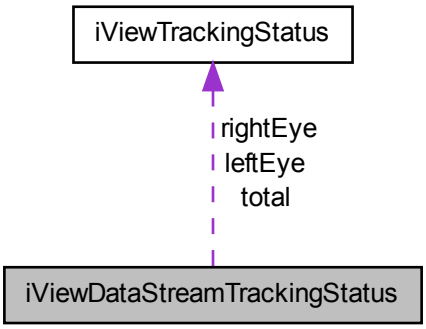
Data Fields

double	positionRatingX	vertical position Valid values are: 0 (optimal position), 1 (almost untrackable: too right/too low/too far), -1 almost untrackable: too left/too high/too near. All values between are possible. This value does NOT behave linear!
double	positionRatingY	horizontal position
double	positionRatingZ	depth/distance position
double	relativePositionX	vertical position This value represents the projection of the eye on the camera sensor. Valid values are: NAN (location unknown), 0 (centered), 1 (upper/right/far maximum), -1 (lower/left/near maximum), +INF (if detected outside, too high), -INF (if detected outside, too low). The shift between upper and lower boundary is a linear movement **See Also** iView_IsNan() iView_IsInf()
double	relativePositionY	horizontal position
double	relativePositionZ	depth/distance position
int32_t	trackingQuality	An indicator for the tracking quality. It indicates the confidence in the calculated gaze. A value in [0,100] where 0 means no confidence and 100 means highest confidence. If the trackingQuality is not calculated or otherwise invalid its value is set to -1.

struct iViewDataStreamTrackingStatus

When subscribing to IVIEWDATASTREAM_TRACKINGSTATUS, the subscription ticket returns instances of this data structure. It contains all information concerning the current tracking quality.

Collaboration diagram for iViewDataStreamTrackingStatus:



Data Fields

uint32_t	eyeFrameIndex	Frame index this trackability belongs to. A value of UINT_MAX indicates an invalid / unknown frame index.
iViewTracking-Status	leftEye	Tracking status of the left eye (if available).
iViewTracking-Status	rightEye	Tracking status of the right eye (if available).
iViewTracking-Status	total	Tracking status of both eyes, merged (if available).
int32_t	trackingRatio-Recent	The tracking ratio (number of frames which resulted in gaze data / number of frames) in the range of [0..100]. The recent tracking ratio describes the tracking ratio for a certain number of recent frames. Please note that this value is only set when recording is active. A value of -1 indicates that there is no tracking ratio available at the moment.
int32_t	trackingRatio-Total	The tracking ratio (number of frames which resulted in gaze data / number of frames) in the range of [0..100]. The total tracking ratio describes the tracking ratio for the current recording. Please note that this value is only set when recording is active. A value of -1 indicates that there is no tracking ratio available at the moment.

struct iViewVector2D

A structure for 2-dimensional position and direction vectors. If all components are NaN, the vector is invalid.

See Also

iView_IsNan()

Data Fields

double	x	
double	y	

struct iViewVector3D

A structure for 3-dimensional position and direction vectors. If all components are NaN, the vector is invalid.

See Also

iView_IsNan()

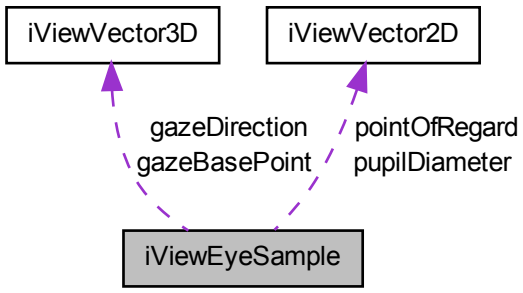
Data Fields

double	x	
double	y	
double	z	

struct iViewEyeSample

Description of the detected parameters of a single eye.

Collaboration diagram for iViewEyeSample:



Data Fields

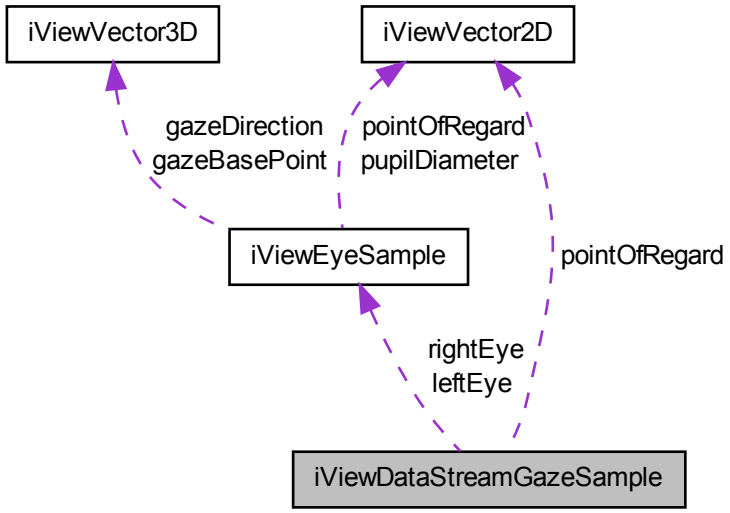
double	eyeball-Uncertainty	This value represents the projection of the eye on the camera sensor. Valid values are: NAN (location unknown), 0 (centered), 1 (upper/right/far maximum), -1 (lower/left/near maximum), +INF (if detected outside, too high), -INF (if detected outside, too low). The shift between upper and lower boundary is a linear movement **See Also** iView_IsNan() iView_IsInf() Measure of uncertainty of eyeball estimator. Lower is better. For ETG: -1.5 is very good, 1.0 is mediore, 4.0 is bad.
iViewVector3D	gazeBasePoint	The point from which the respective gaze vector originates [mm] which may depend on the used eye tracking system. For the ETG it is the eyeball center. A value of NAN means that the coordinate is invalid and the vector could not be calculated. **See Also** iView_IsNan()
iViewVector3D	gazeDirection	3d gaze vector based in 3d gaze base point A value of NAN means that the coordinate is invalid and the vector could not be calculated. **See Also** iView_IsNan()
iViewVector2D	pointOfRegard	The point of regard, in screen coordinates. A value of NAN means that the coordinate is invalid and the vector could not be calculated. **See Also** iView_IsNan()
double	pupil-Confidence	Contains floating point values between 0 and 1 representing the likelihood of having correctly recognized the pupil. A value of NAN means that the confidence was not calculated. **See Also** iView_IsNan()
iViewVector2D	pupilDiameter	Diameter of the left Pupil [eye camera pixels]. Values of NAN means that the diameter was not calculated. **See Also** iView_IsNan()

double	pupilRadius	Pupil radius [mm]. A value of NAN means that the radius was not calculated. **See Also** iView_IsNan()

struct iViewDataStreamGazeSample

When subscribing to IVIEWDATASTREAM_GAZE_INFORMATION, the subscription ticket returns instances of this data structure. It contains all extracted information concerning a person's gaze at the specified moment in time.

Collaboration diagram for iViewDataStreamGazeSample:



Data Fields

uint32_t	eyeFrame-Number	The number of the eye frame this POR corresponds to.
iViewEye-Sample	leftEye	The left eye data.
iViewVector2D	pointOfRegard	The point of regard (POR), in screen coordinates. A value of NA-N means that the coordinate is invalid and the POR could not be calculated. **See Also** iView_IsNan()
iViewEye-Sample	rightEye	The right eye data.
uint32_t	sceneFrame-Number	The number of the scene frame this POR corresponds to.
uint64_t	timestamp	Timestamp of the GazeSample in nanoseconds.

struct iViewDataStreamSpecStruct

The basic layout of all stream specifiers.

Declaration of an optional addition to a stream subscription. There are several stream specifications available, depending on the device and the stream type. The element iViewDataStreamSpec.type indicates the data type of the whole stream specification structure. All stream specification share the first two elements, use static casting to access the other elements.

Data Fields

iViewData-StreamSpec *	next	If there is another stream specifier, the address is contained here.
iViewData-StreamSpec-Type	type	The type of this structure. Use this element for casting the whole structure.

struct iViewDataStreamSpecSampleRate

Stream specifier to define a maximum framerate. Please note that this is an upper limit. You can reduce the sample rate of a sensor or data stream, but not exceed its natural sample rate. It is safe to specify a sufficiently high sample rate to assert that no sample is lost. The decision if a sample is to be transferred is calculated for each sample individually. There will be no artifacts due to sample rate interference.

Data Fields

iViewDataStreamSpec *	next	Data inherited from iViewDataStreamSpec.
double	sampleRate	The desired receiving sample rate. Using this parameter it is possible to reduce the sample rate. Not all data streams understand this parameter (e.g. IVIEWDATASTREAM_SCENEIMAGES_H264). • Use 0 to indicate no sample rate (no data will be sent at all). • Use INF to specify no sample rate limitation (send data as fast as possible). • Use any value between 0 and INF to throttle the sample rate as specified. Using a negative value will have the same effect as using INFINITY (no sample rate limitation). **See Also** iView_IsInf() iView_GetInfValue()
iViewDataStreamSpecType	type	Data inherited from iViewDataStreamSpec. Value must be IVIEWDATASTREAMSPEC_SAMPLE_RATE.

struct iViewDataStreamSpecGazeFilter

Stream specifier to apply a gaze filter. Value must be IVIEWDATASTREAMSPEC_GAZE_FILTER.

Note

not implemented yet

Data Fields

iViewDataStreamSpecGazeFilterType	gazeFilterType	Apply a gaze filter, see IVIEWDATASTREAMSPEC_GAZE_FILTER_TYPE_*.
iViewDataStreamSpec *	next	Data inherited from iViewDataStreamSpec.
iViewDataStreamSpecType	type	Data inherited from iViewDataStreamSpec.

struct iViewDataStreamSpecQualityOfService

Stream specifier to specify the required quality of service for remote communication. This has an impact on the used IP network communication protocol and thus on the data transmission rate and latency. It is only useful for network communication. For shared memory communication, the quality of service has no influence.

Lossless communication is realized by using TCP communication by adding some protocol overhead. As result, data transmitted over TCP is not lost unless the connection is not lost at all, but there may be a lag as result of possible retransmission of data. Lossy communication is realized by using UDP communication which is lean and has only little overhead. As result, there is only a minimal lag from network communication, but data may be lost.

By default, all API commands are transmitted using TCP, all data streams are transmitted using UDP. To increase the safety of a data stream, use this stream specifier.

Data Fields

iViewDataStreamSpec *	next	Data inherited from iViewDataStreamSpec.
iViewQualityOfService	quality	Specify the required quality of service for remote communication.
iViewDataStreamSpecType	type	Data inherited from iViewDataStreamSpec. Value must be IVIEWDATASTREAMSPEC_QUALITY_OF_SERVICE.

struct iViewStreamSubscription

Datastructure to specify a data stream subscription. The contents may vary according to the device type.

Note

If it is some day required to add stuff for other devices, change this to a union, depending on the device.

Data Fields

iViewDataStreamSpec *	streamSpec	Start of a linked list of (possibly device-dependent) stream specifiers. Defaults to NULL. Use iViewDataStreamSpec data structures and link them together.
iViewDataStreamType	streamType	The type of the datastream.

struct iViewDataStreamEndOfStream

With iView_SubscribeDataStream(), the user may subscribe to data streams. In return, the user will receive iViewResult structures containing one or more iViewDataStream entities. When the end of a data stream is reached, e.g. because the data stream was provided by a file player or a sensor terminated submission of data on its own, a iViewDataStream structure is delivered which indicates that the end of the stream was reached. The data pointer of this structure then points to a iViewDataStreamEndOfStream structure.

Data Fields

uint32_t	numberOfSamples	The number of samples which have been created since the last call of startDataAcquisition(), excluding the EndOfStream data stream entity. Use this information to assert that no samples were lost in transmission.

struct iViewDataStream

With iView_SubscribeDataStream(), the user may subscribe to data streams. In return, the user will receive iViewResult structures containing one or more iViewDataStream entities.

Data Fields

void *	data	Pointer to the data (may be NULL, e.g. if lastStreamEntity == 2). Use iViewDataStream.type to determine the type of this structure.
uint32_t	index	A counter which contains a running number. The index enumerates the number of valid stream entities. With each provided data entity, this number is increased by 1, starting at 0 only when the application starts.
iViewDataStreamState	lastStreamEntity	A flag to indicate whether this is the last stream entity (lastStreamEntity := IVIEWDATASTREAM_END) or not and if more stream entities will be provided subsequently (lastStreamEntity := IVIEWDATASTREAM_NO_END). Set to IVIEWDATASTREAM_END_UNKNOWN if unknown. If set to IVIEWDATASTREAM_END, the data pointer refers to a iViewDataStreamEndOfStream structure.
iViewDataStreamType	type	

struct iViewOverlaySpec

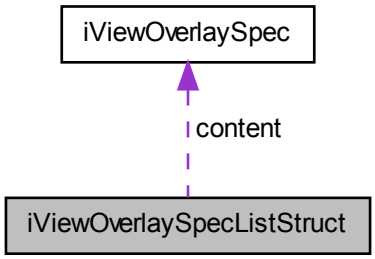
Data Fields

iViewOverlayOption	option	

iViewOverlay-Position	position-Horizontal	
iViewOverlay-Position	positionVertical	
iViewOverlay-Type	type	

struct iViewOverlaySpecListStruct

Collaboration diagram for iViewOverlaySpecListStruct:



Data Fields

iViewOverlay-Spec	content	
iViewOverlay-SpecList *	next	

Typedef Documentation

typedef uint32_t iViewDataStreamSpecGazeFilterType

Identifier for the gazeFilterType in the iViewDataStreamSpecGazeFilter data structure.

typedef uint32_t iViewDataStreamSpecType

Identifier for the iViewDataStreamSpec data structure.

typedef uint32_t iViewDataStreamState

Identifier to differ between alive streams (which will continue to send data) and terminated streams (which will not submit any further data).

typedef uint32_t iViewDataStreamType

If the user wants to asynchronously receive data, the user must subscribe to the data stream in advance. The iViewDataStreamType type is used to specify the kind of data the user wants to receive. Please note that not all eye tracking devices support all data streams!

Data Stream	ETG	Bandwidth
IVIEWDATASTREAM_SCENEIMAGES_WITH_GAZE	yes (1)	675 Mbit/s
IVIEWDATASTREAM_SCENEIMAGES	yes (1)	675 Mbit/s
IVIEWDATASTREAM_SCENEIMAGES_H264	yes	approx. 12 Mbit/s
IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED	yes (2)	approx. 12 Mbit/s
IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED_WITH_GAZE	yes (2)	approx. 12 Mbit/s
IVIEWDATASTREAM_TRACKINGSTATUS	yes	3 kbit/s
IVIEWDATASTREAM_EYEIMAGES	no	-
IVIEWDATASTREAM_EYEIMAGES_LEFT	yes	2.2 - 8.8 Mbit/s
IVIEWDATASTREAM_EYEIMAGES_RIGHT	yes	2.2 - 8.8 Mbit/s
IVIEWDATASTREAM_SERVERSTATUS	yes	up to 1 kbit/s
IVIEWDATASTREAM_GAZE_INFORMATION	yes	approx. 300 kbit/s
IVIEWDATASTREAM_CALIBRATION_DATA	no	-
IVIEWDATASTREAM_EYE_EVENTS	no	-
IVIEWDATASTREAM_TRACKING_MONITOR	no	-

(1,2): these data streams are mutually exclusive.

See Also

FAQ: How many data streams can be subscribed? .

typedef uint32_t iViewEyeIdentifier

Identifier to determine the eye.

typedef uint32_t iViewOverlayOption

Additional parameters for the overlay. The meaning of the option depends on the overlay type.

Attention

Always initialize the structure with zero before using it!

typedef uint32_t iViewOverlayPosition

Define the position of an overlay (bit mask, may be combined to list overlays).

typedef struct __iViewOverlaySpecListStruct iViewOverlaySpecList

typedef uint32_t iViewOverlayType

Definition of overlays to apply to images (bit mask, may be combined to list overlays).

Warning

Not yet implemented.

typedef uint32_t iViewQualityOfService

Identifier of different comunication qualities.

Function Documentation

iViewRC iView_SubscribeDataStream (iViewTicket * *ticket*, iViewStreamSubscription const *const *subscription*)

Subscribe a datastream. As consequence, the API will receive the subscribed data asynchronously and eventually call the callback function to submit the data to the API user.

Do never subscribe twice to the same data stream or expect the result tickets to be mixed.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_-ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:

iViewTicket.returnCode is RC_NO_ERROR,

- iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS until the subscription is stopped actively or due to an error.
  - iViewTicket.progress is 50.

On failure:

- iViewTicket.returnCode indicates the error code,
- iViewTicket.result is NULL or a pointer to iViewServerError.

iViewTicket.state is IVIEWTICKETSTATUS_CLOSED

- iViewTicket.progress is 100.

Note

Please mind that a subscribed data stream consumes communication bandwidth. See iViewData-StreamType for details.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_-ReleaseTicket() instead!
`in`	*subscription*	type and constraints for the subscription, may be released immediately after the function returns

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_UnsubscribeDataStream (iViewTicket * *ticket*, const iViewDataStreamType *streamType*)

Unsubscribe from a datastream of the specified type. As consequence, the API will no longer receive the subscribed data asynchronously.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The specified stream was subscribed.

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.

This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
- iViewTicket.progress is 100.
  - On success:

iViewTicket.returnCode is RC_NO_ERROR,

- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.

Postcondition

On success, this function will close the ticket which called iView_SubscribeDataStream();

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
in	*streamType*	the desired stream type

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

Variable Documentation

const iViewEyeIdentifier EYE_BINOCULAR = 3 [static]

declares that both eyes are included

const iViewEyeIdentifier EYE_LEFT = 1 [static]

identifies a human's left eye

const iViewEyeIdentifier EYE_RIGHT = 2 [static]

identifies a human's right eye

const iViewEyeIdentifier EYE_UNKNOWN = 0 [static]

the eye is unknown

const int32_t IVIEW_DEPTH_16S = (IVIEW_DEPTH_SIGN | 16) [static]

constant for iViewImage.depth

const int32_t IVIEW_DEPTH_16U = 16 [static]

constant for iViewImage.depth

const int32_t IVIEW_DEPTH_1U = 1 [static]

constant for iViewImage.depth

const int32_t IVIEW_DEPTH_32F = 32 [static]

constant for iViewImage.depth

const int32_t IVIEW_DEPTH_32S = (IVIEW_DEPTH_SIGN | 32) [static]

constant for iViewImage.depth

const int32_t IVIEW_DEPTH_8S = (IVIEW_DEPTH_SIGN | 8) [static]

constant for iViewImage.depth

const int32_t IVIEW_DEPTH_8U = 8 [static]

constant for iViewImage.depth

const int32_t IVIEW_DEPTH_SIGN = 0x80000000 [static]

constant for iViewImage.depth

const iViewDataStreamType IVIEWDATASTREAM_CALIBRATION_DATA = 12 [static]

not yet implemented

const iViewDataStreamState IVIEWDATASTREAM_END = 2 [static]

This stream reached its end.

const iViewDataStreamState IVIEWDATASTREAM_END_UNKNOWN = 0 [static]

Stream state is unknown.

const iViewDataStreamType IVIEWDATASTREAM_EYE_EVENTS = 13 [static]

not yet implemented

const iViewDataStreamType IVIEWDATASTREAM_EYEIMAGES = 7 [static]

Receive eye images for devices which do not provide left / right eye images. This stream delivers data packets of the type iViewDataStreamEyeImage.

Please note that this creates a huge amount of data, depending on the image resolution and sample rate.

const iViewDataStreamType IVIEWDATASTREAM_EYEIMAGES_LEFT = 8 [static]

Receive left eye images. This stream delivers data packets of the type iViewDataStreamEyeImage.

Please note that this creates a huge amount of data, depending on the image resolution and sample rate. For example, with VGA resolution of 640∗480 pixels at 30 frames per second, a bandwidth of 8.8 Mbit/s is required.

const iViewDataStreamType IVIEWDATASTREAM_EYEIMAGES_RIGHT = 9 [static]

Receive right eye images. This stream delivers data packets of the type iViewDataStreamEyeImage.

Please note that this creates a huge amount of data, depending on the image resolution and sample rate. For example, with VGA resolution of 640∗480 pixels at 30 frames per second, a bandwidth of 8.8 Mbit/s is required.

const iViewDataStreamType IVIEWDATASTREAM_GAZE_INFORMATION = 11 [static]

const iViewDataStreamType IVIEWDATASTREAM_INVALID = 0 [static]

Initial value, don't use.

const iViewDataStreamState IVIEWDATASTREAM_NO_END = 1 [static]

There will be more events.

const iViewDataStreamType IVIEWDATASTREAM_SCENEIMAGES = 2 [static]

Receive fullframe scene images without gaze overlay. This stream delivers data packets of the type iViewDataStreamSceneImage.

Please note that this creates an enormous amount of data, depending on the image resolution and sample rate. For example, with an ETG device operating with 1280∗960 pixels at 24 frames per second, a bandwidth of 675 Mbit/s is required.

const iViewDataStreamType IVIEWDATASTREAM_SCENEIMAGES_H264 = 3 [static]

Receive the H.264 encoded scene image and audio data stream. This stream delivers data packets of the type iViewDataStreamH264SceneImage.

Use this data stream if you intend to decode the H.264 data on your own. The audio stream is not configurable: mono, 16bit, 16k (results in 256kbit/s) PCM encoded.

const iViewDataStreamType IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED = 4 [static]

Receive fullframe scene images, but instead of sending the uncompressed images, send them compressed and let the client side decode the images. This stream delivers data packets of the type iViewDataStreamSceneImage.

Use this data stream if you intend to receive scene images via low bandwidth networks (WLAN, 100 Mbit Ethernet etc).

const iViewDataStreamType IVIEWDATASTREAM_SCENEIMAGES_H264_DECODED_WITH_GAZE = 5 [static]

Receive fullframe scene images with gaze overlay, but instead of sending the uncompressed images, send them compressed and let the client side decode the images. This stream delivers data packets of the type iViewDataStreamSceneImage.

Use this data stream if you intend to receive scene images via low bandwidth networks (WLAN, 100 Mbit Ethernet etc).

const iViewDataStreamType IVIEWDATASTREAM_SCENEIMAGES_WITH_GAZE = 1 [static]

Receive fullframe scene images with gaze overlay. This stream delivers data packets of the type iViewDataStreamSceneImage.

Please note that this creates an enormous amount of data, depending on the image resolution and sample rate. For example, with an ETG device operating with 1280∗960 pixels at 24 frames per second, a bandwidth of 675 Mbit/s is required.

const iViewDataStreamType IVIEWDATASTREAM_SERVERSTATUS = 10 [static]

Receive server status. This stream delivers data packets of the type iViewDataStreamEtgServerStatus and creates a communication load of approximately 1 kbit/s. Receive the calculated gaze and related data. This stream delivers data packets of the type iViewDataStreamGazeSample and creates a communication load of approximately 300 kbit/s.

const iViewDataStreamType IVIEWDATASTREAM_TRACKING_MONITOR = 14 [static]

not yet implemented

const iViewDataStreamType IVIEWDATASTREAM_TRACKINGSTATUS = 6 [static]

Receive trackability information. This stream delivers data packets of the type iViewDataStreamTracking-Status and creates a communication load of approximately 3 kbit/s.

const iViewDataStreamSpecType IVIEWDATASTREAMSPEC_GAZE_FILTER = 2 [static]

apply a filter to the gaze cursor, the structure type is iViewDataStreamSpecGazeFilter

const iViewDataStreamSpecGazeFilterType IVIEWDATASTREAMSPEC_GAZE_FILTER_TYPE_ETG_FIR = 2 [static]

not implemented yet

const iViewDataStreamSpecGazeFilterType IVIEWDATASTREAMSPEC_GAZE_FILTER_TYPE_ETG_IIR = 1 [static]

not implemented yet

const iViewDataStreamSpecGazeFilterType IVIEWDATASTREAMSPEC_GAZE_FILTER_TYPE_NO_FILTER = 0 [static]

disable filter

const iViewDataStreamSpecType IVIEWDATASTREAMSPEC_NONE = 0 [static]

no specifier, the structure type is iViewDataStreamSpec

const iViewDataStreamSpecType IVIEWDATASTREAMSPEC_QUALITY_OF_SERVICE = 3 [static]

control network communication protocol, the structure type is iViewDataStreamSpecQualityOfService

const iViewDataStreamSpecType IVIEWDATASTREAMSPEC_SAMPLE_RATE = 1 [static]

define upper limit for sample rate, the structure type is iViewDataStreamSpecSampleRate

const iViewOverlayType IVIEWOVERLAY_INVALID = 0 [static]

Indicates an invalid data structure. Do not use.

const iViewOverlayType IVIEWOVERLAY_PARTICIPANTNAME = 0x10 [static]

Display the participant name.

Warning

Not yet implemented.

const iViewOverlayType IVIEWOVERLAY_RECORDINGTIME = 0x02 [static]

Display the recording time.

Warning

Not yet implemented.

const iViewOverlayType IVIEWOVERLAY_SMILOGO = 0x01 [static]

Display the SMI logo.

Warning

Not yet implemented.

const iViewOverlayType IVIEWOVERLAY_SYSTEMDATETIME = 0x08 [static]

Display the system date and time.

Warning

Not yet implemented.

const iViewOverlayType IVIEWOVERLAY_SYSTEMTIME = 0x04 [static]

Display the system time.

Warning

Not yet implemented.

const iViewOverlayPosition IVIEWOVERLAYPOSITION_BOTTOMRIGHT = 0x0004 [static]

const iViewOverlayPosition IVIEWOVERLAYPOSITION_CENTER = 0x0002 [static]

const iViewOverlayPosition IVIEWOVERLAYPOSITION_INVALID = 0 [static]

const iViewOverlayPosition IVIEWOVERLAYPOSITION_TOPLEFT = 0x0001 [static]

const iViewQualityOfService IVIEWQOS_LOSSLESS = 1 [static]

lossless communication required

const iViewQualityOfService IVIEWQOS_LOSSY = 0 [static]

loss of data is acceptable

const iViewQualityOfService IVIEWQOS_UNKNOWN = -1 [static]

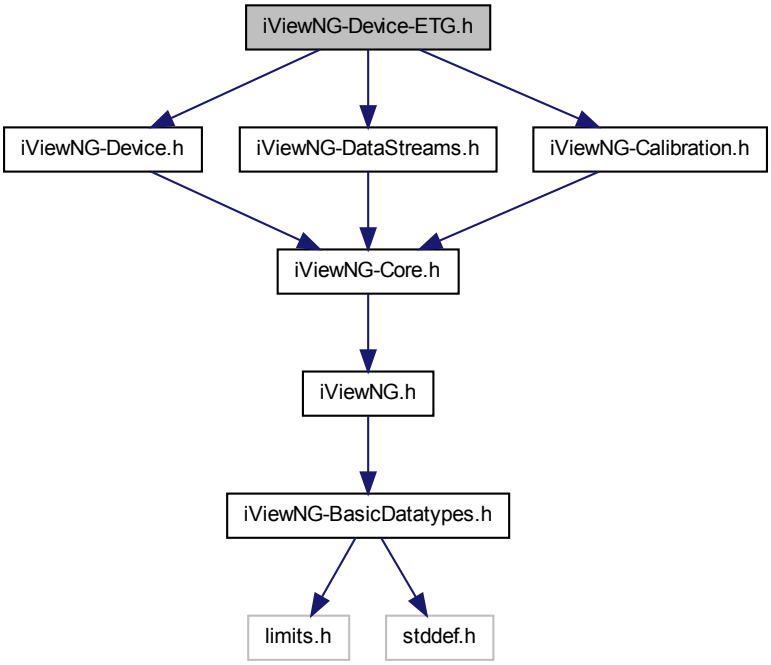
loss of data is acceptable

const iViewDataStreamType NUMBER_OF_IVIEWDATASTREAMS = 15 [static]

7.7 iViewNG-Device-ETG.h File Reference

```
#include <iViewNG-Device.h>
#include <iViewNG-DataStreams.h>
#include <iViewNG-Calibration.h>
```

Include dependency graph for iViewNG-Device-ETG.h:



Detailed Description

This file declares the iViewNG API functions and data types which are specific to the iView ETG eye tracker.

The ETG currently supports the following eye camera modes:

Camera operation	Eye camera resolution	Eye camera sample rate
IVIEWTRACKINGMODE_ET-G_BINOCULAR	IVIEWRESOLUTION_ETG_-640x480	IVIEWSAMPLERATE_ETG_30
IVIEWTRACKINGMODE_ET-G_BINOCULAR	IVIEWRESOLUTION_ETG_-320x240	IVIEWSAMPLERATE_ETG_30
IVIEWTRACKINGMODE_ET-G_BINOCULAR	IVIEWRESOLUTION_ETG_-320x240	IVIEWSAMPLERATE_ETG_60 (∗)

(∗) 60Hz mode requires an ETG-2 device!

The ETG currently supports the following scene camera modes:

Scene camera resolution	Scene camera sample rate
IVIEWRESOLUTION_ETG_1280x960	IVIEWSAMPLERATE_ETG_24

IVIEWRESOLUTION_ETG_960x720	IVIEWSAMPLERATE_ETG_30 (∗)

(∗) 30 Hz mode requires an ETG-2 device!

Data Structure Documentation

struct iViewDeviceVersionEtg

Structure to describe the version of the ETG device components.

Data Fields

wchar_t	driverVersionEyeCam	Eye camera driver version or empty if unknown.
uint32_t	firmwareVersionEyeCam	Eye camera firmware version number or 0 if unknown.
uint32_t	softwareVersionSceneCam	Scene camera software version, format YYYYMMDD or 0 if unknown.

struct iViewLicenseEtg

Structure which describes an ETG license.

Data Fields

iViewLicense ∗	next	Pointer to another license structure, if there are multiple licenses.

struct iViewSerialNumberEtg

Structure which describes an ETG serial number.

Data Fields

wchar_t	chassis	

struct iViewDataStreamEtgServerStatus

Structure to represent the recording device for an ETG system.

Data Fields

int32_t	batteryLifetimeMinutes	The remaining battery lifetime in number of minutes. A value of INT_MAX indicates that the recording device does not operate on battery but on AC power. A value of -1 indicates that the battery state could not be determined.
int32_t	batteryLifetimePercent	The remaining battery lifetime in percent. A value of -1 indicates that the battery state could not be determined.
int32_t	countOfCpuCores	Count of CPU cores.
int32_t	cpuCoreFrequencies	The measured core frequencies in MHz, only countOfCpuCores is used. Sorted ascending (core1, core2, .., coreMAX)
int32_t	cpuCoreTemperatures	The measured core temperatures in Celsius, only countOfCpuCores is used. Sorted ascending (core1, core2, .., coreMAX)
int32_t	eyeCameraAvailable	The status of the eye camera. If 1, the camera is operational and may be started. If 0, the camera is not (yet) ready.
uint32_t	freeStorageMinutes	The currently free storage space on recording unit data tank, given in recording minutes. This describes the device which contains the data repository. A value of UINT_MAX indicates that the capacity exceeds UINT_MAX-1 minutes by an unknown size.
double	measuredAcquisitionFpsOfLeftEye	The measured acquisition frame rate of the left eye, in Hz. The value is refreshed roughly every 3,33s for ETG 30Hz. The value is refreshed roughly every 1,66s for ETG 60Hz. Is NaN if no acquisition is running.
double	measuredAcquisitionFpsOfRightEye	The measured acquisition frame rate of the left eye, in Hz. The value is refreshed roughly every 3,33s for ETG 30Hz. The value is refreshed roughly every 1,66s for ETG 60Hz. Is NaN if no acquisition is running.
uint32_t	recordingTime	The length of the recording in progress or (if recording is not active) the length of the last acquired recording [in milliseconds].
int32_t	sceneCameraAvailable	The status of the scene camera. If 1, the camera is operational and may be started. If 0, the camera is not (yet) ready.
double	systemWideCpuUsage	The current system wide cpu usage over all cpu cores (the load of the computer). In [0,100]. Can be -1 on error.

struct iViewDeviceParametersEtgCameraPlayback

Used in iViewDeviceParameters if iViewDeviceType is IVIEWDEVICE_ETG_CAMERAPLAYBACK. Includes the parameters for using realtime playback of an ETG device (aka Mii, Smiggles, GazeWear, HED5). If all parameters are set to zero, the parameters will not be changed but remain in their current state.

The data may be released immediately after submitting the structure to iViewNG.

Attention

Always initialize the structure with zero before using it!

Data Fields

iViewAudio-ActivationState	audioState	Audio activation state. A value of IVIEWTRACKINGMODE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
wchar_t const *	baseFilename	The filename to append to outputDirectory, NOT including any suffix, NOT including any path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
iViewCamera-Resolution	camera-ResolutionEye	The resolution of the submitted eye images [pixels] A value of IVIEWRESOLUTION_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewCamera-Resolution	camera-Resolution-Scene	The resolution of the submitted scene images [pixels] A value of IVIEWRESOLUTION_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewTracking-Mode	operationMode	The operation mode(s) of the eye tracker. A value of IVIEWTRACKINGMODE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
wchar_t const *	outputDirectory	The directory for storing recorded files, NOT ending with the path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
iViewOverlay-SpecList *	overlaySpec-List	**Note** not yet supported
iView-SamplingRate	samplingRate-EyeCam	Average scene camera frame rate [frames per second] A value of IVIEWSAMPLINGRATE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iView-SamplingRate	samplingRate-SceneCam	Average scene camera frame rate [frames per second] A value of IVIEWSAMPLINGRATE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewWhite-Balance-Program	whiteBalance-Program	Scene camera autowhitebalance program. A value of IVIEWWHITEBALANCE_CURRENT means 'no change' and orders the server to not change the last submitted setting.

struct iViewCalibrationParametersEtg

Data structure including all required parameters for calibrating an ETG device.

Data Fields

double	calibration-Distance	The calibration distance [m] which will be used. A value of 0 indicates that the calibration distance is unknown / not used.

Variable Documentation

const unsigned int DRIVER_ETG_EYECAM_LENGTH = 32 [static]

The maximal number of characters of a serial number, including '\0'.

const iViewCalibrationType IVIEWCALIBRATIONTYPE_ETG_1POINT = 0x02 [static]

Identifier for different calibration types.

not yet supported

const iViewCalibrationType IVIEWCALIBRATIONTYPE_ETG_1POINTAVERAGING = 0x04 [static]

not yet supported

const iViewCalibrationType IVIEWCALIBRATIONTYPE_ETG_1POINTSPHERICAL = 0x08 [static]

Standard 1-pt calibration.

const iViewCalibrationType IVIEWCALIBRATIONTYPE_ETG_2POINT = 0x10 [static]

not yet supported

const iViewCalibrationType IVIEWCALIBRATIONTYPE_ETG_3POINT = 0x20 [static]

Standard 3-pt calibration.

const iViewCalibrationType IVIEWCALIBRATIONTYPE_ETG_5POINT = 0x40 [static]

not yet supported

const iViewDeviceType IVIEWDEVICE_ETG = 0x00010000 [static]

Identifier for different device types / operation modes.

bit mask for all iView ETG device modes

const iViewDeviceType IVIEWDEVICE_ETG_CAMERAPLAYBACK = 0x00010010 [static]

use ETG Controller & iViewNG Server

const iViewCameraResolution IVIEWRESOLUTION_ETG_1280x720 = 0x10 [static]

camera resolution of 1280 pixel horizontal, 720 pixel vertical

const iViewCameraResolution IVIEWRESOLUTION_ETG_1280x960 = 0x20 [static]

camera resolution of 1280 pixel horizontal, 960 pixel vertical

const iViewCameraResolution IVIEWRESOLUTION_ETG_320x240 = 0x01 [static]

Identifier for different camera resolutions.

camera resolution of 320 pixel horizontal, 240 pixel vertical

const iViewCameraResolution IVIEWRESOLUTION_ETG_400x300 = 0x02 [static]

camera resolution of 400 pixel horizontal, 300 pixel vertical

const iViewCameraResolution IVIEWRESOLUTION_ETG_640x480 = 0x04 [static]

camera resolution of 640 pixel horizontal, 480 pixel vertical

const iViewCameraResolution IVIEWRESOLUTION_ETG_960x720 = 0x08 [static]

camera resolution of 980 pixel horizontal, 720 pixel vertical

const iViewSamplingRate IVIEWSAMPLERATE_ETG_120 = 0x08 [static]

60 samples per second

const iViewSamplingRate IVIEWSAMPLERATE_ETG_24 = 0x01 [static]

Identifier for different sampling rates.

const iViewSamplingRate IVIEWSAMPLERATE_ETG_240 = 0x10 [static]

120 samples per second

const iViewSamplingRate IVIEWSAMPLERATE_ETG_30 = 0x02 [static]

24 samples per second

const iViewSamplingRate IVIEWSAMPLERATE_ETG_60 = 0x04 [static]

30 samples per second

const iViewTrackingMode IVIEWTRACKINGMODE_ETG_BINOCULAR = 0x04 [static]

use both eye cameras for tracking

const iViewTrackingMode IVIEWTRACKINGMODE_ETG_MONOCULAR_LEFT = 0x01 [static]

Identifier for different eye camera operation modes.

use only left eye camera for tracking, not yet supported

const iViewTrackingMode IVIEWTRACKINGMODE_ETG_MONOCULAR_RIGHT = 0x02 [static]

use only right eye camera for tracking, not yet supported

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_AUTO = 0x01

240 samples per second

Do not change the current setting. Automatically adjust white balance setting.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_BASIC = 0x04

For using outside, cloudy or in shade.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_DAYLIGHT = 0x02

For using outside, clear days.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_FLUORESCENT = 0x10

For using in white fluorescent lightning.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_FLUORESCENT_H = 0x20

For using daylight fluorescent lightning.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_TUNGSTEN = 0x08

For using in tungsten lightning.

const unsigned int SERIAL_NUMBER_ETG_LENGTH = 32 `[static]`

The maximal number of characters of a serial number, including '\0'.

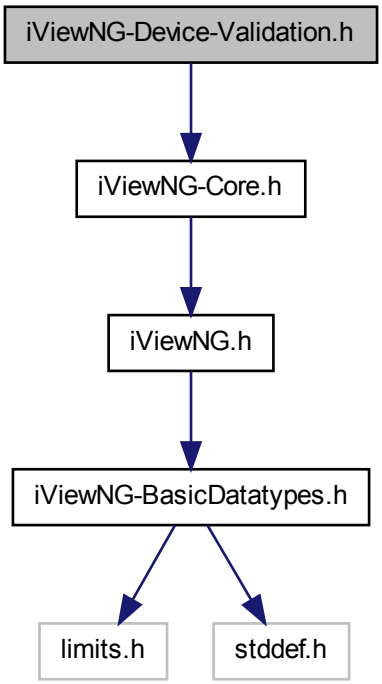
const int SERVERSTATUS_MAX_CORECOUNT = 64 `[static]`

The maximal core count supported by server status.

7.8 iViewNG-Device-Validation.h File Reference

```
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-Device-Validation.h:



Detailed Description

| \ | | ___ | |_ ___ _ | | |/ _ | __/ _ \ (_) | |\ | (_) | || __/ _ |_| _|___/ _____| (_)

Unlike most of the other header files, this file is NOT part of the iViewNG-SDK product release!

This file declares the iViewNG API functions and data types which are specific to devices that supports a validation.

A validation is part of the calibration. See below for workflow details.

Data Structure Documentation

struct iViewValidationElement

This structure defines one element of the validation element.

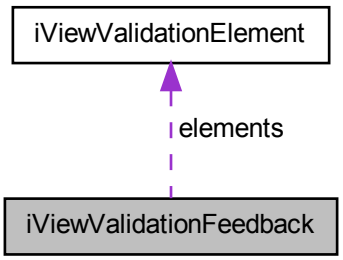
Data Fields

double	leftEyeX	
double	leftEyeY	
double	rightEyeX	
double	rightEyeY	
iView-Validation-ElementType	type	

struct iViewValidationFeedback

This structure is returned from the server when calling iView_GetValidation

Collaboration diagram for iViewValidationFeedback:



Data Fields

uint32_t	count	number of elements in the elements-Array
iView-Validation-Element *	elements	

Typedef Documentation

typedef uint32_t iViewValidationElementType

Function Documentation

iViewRC iView_GetValidation (iViewTicket * *ticket*)

Request the Validation Information.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The calibration parameters were supplied by iView_SetupCalibration().
The calibration procedure was started by calling iView_StartCalibration().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewResult.type is IVIEWRESULT_GET_VALIDATION,
  - iViewResult.data is a pointer to iViewValidationFeedback,
  - iViewResult.next is NULL since there will be no subsequent results.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

The successful delivery of an end request will set the ticket associated with iView_StartCalibration() into the state IVIEWTICKETSTATUS_CLOSED and its return code to RC_NO_ERROR.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_STATE	if iView_SetupCalibration() was not called in advance
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

Variable Documentation

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_FIXATION = 4 `[static]`

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_GAZEPATH = 6 `[static]`

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_GAZEPATH_BEGIN = 5 `[static]`

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_GAZEPATH_FINAL = 7 `[static]`

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_INVALID = 0 `[static]`

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_VALIDATION_ACCURACY = 8 `[static]`

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_VALIDATION_PRECISION = 9 `[static]`

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_VALIDATIONPOINT = 1 `[static]`

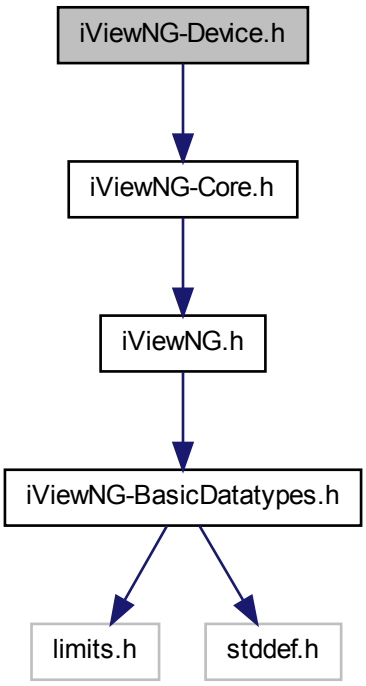
const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_VALIDATIONPOINT_ACCURACY = 2
`[static]`

const iViewValidationElementType IVIEWCALIBRATIONELEMENTTYPE_VALIDATIONPOINT_PRECISION = 3
`[static]`

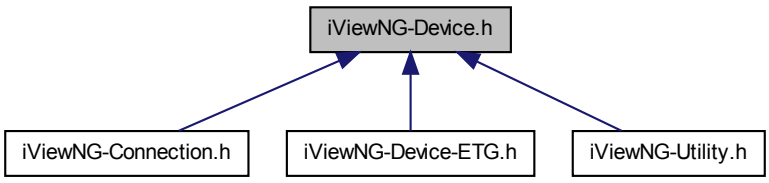
7.9 iViewNG-Device.h File Reference

```
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-Device.h:



This graph shows which files directly or indirectly include this file:



Detailed Description

This file declares the iViewNG API functions and data types which are required for handling eye tracking device parameters. This includes only the device-independent part. Device-specific definitions are located in the according device header file.

Data Structure Documentation

struct _iViewLicenseStruct

Generic structure of a license. Cast pointers to this structure to the device-dependent structure, e.g. iViewLicenseEtg to gain access to the device-dependent attributes.

Data Fields

iViewLicense *	next	Pointer to another license structure, if there are multiple licenses.

struct _iViewDeviceInfoStruct

Generic structure of a serial number. Cast pointers to this structure to the device-dependent structure, e.g. iViewLicenseEtg to gain access to the device-dependent attributes. Using iView_GetDeviceInfo(), the client application may inquire which devices are attached to a certain iViewNG server process. This data structure is returned asynchronously as answer. There may be more than just one device. In this case, the devices are linked via the 'next' element.

Data Fields

void *	capabilities	Device-dependent capability structure. Cast based on 'type' element, e.g. to iViewDeviceCapabilityEtg. Not yet used.
iViewLicense *	license	Device-dependent license structure for this device. Cast based on 'type' element, e.g. to iViewLicenseEtg. Not yet used.
iViewDeviceInfo *	next	If there is another device available, this element points to the next iViewDeviceInfo data structure.
void *	serialNumber	Cast based on 'type' element, e.g. to iViewSerialNumberEtg.
iViewDeviceType	type	Specifies the attached device type.
void *	version	Device-dependent version structure. Cast based on 'type' element, e.g. to iViewDeviceVersionEtg.

struct iViewDeviceParameters

Using iView_SetParameters(), the client application may define parameters for data recording. The available parameters depend on the server configuration and the iViewDeviceType. Use the iView-

DeviceType to determine how to cast the void pointer.

Data Fields

iViewDevice-Type	deviceType	Defines the attached device type.
void *	parameters	The result data. Use the iViewDeviceType to determine how to cast.

Typedef Documentation

typedef uint32_t iViewAudioActivationState

Audio activation state.

typedef uint32_t iViewCameraResolution

The resolution of the submitted images.

typedef uint32_t iViewDeviceType

The iViewDevice structure contains an enumeration attribute which indicates which device is used. See iViewNG-Device-*.h for the proper setting.

typedef uint32_t iViewSamplingRate

Average sampling rate.

typedef uint32_t iViewTrackingMode

Datatype to encode server capabilities. Device-dependent. The operation mode(s) of the eye tracker.

typedef uint32_t iViewWhiteBalanceProgram

Scene camera autowhitebalance program.

Function Documentation

iViewRC iView_GetDeviceInfo (iViewTicket *const *ticket*)

Query the iViewDeviceInfo structure.

This function produces an asynchronously delivered iViewResult data structure of the type iViewDevice-Info.

If there is more than one device available, this data structure enumerates all available devices. If there are more than one device available, the result may be returned via the same ticket subsequently, each call containing one or more new iViewDeviceInfo structures for new device(s). Devices which are recognized after calling iView_GetDeviceInfo() will not be returned as soon as they appear.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.

The ticket progress will be set to 100% when the request was processed by the server.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_UNSPECIFIED	on any other error

iViewRC iView_GetDeviceParameters (iViewTicket * *ticket*)

Get the operation mode which is currently in use by the server.

Warning

Not yet implemented.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.

This function produces asynchronously delivered iViewResult data and calls the user callback function to provide the result of the operation through the ticket:

- if iViewTicket.returnCode is RC_NO_ERROR:
  - iViewTicket.result is NULL

iViewTicket.state is IVIEWTICKETSTATUS_CLOSED

- if iViewTicket.returnCode indicated an error:
  - iViewTicket.result is NULL or a pointer to iViewServerError
  - iViewTicket.state remains at the former state

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

Return values

RC_INVALID_REQUEST	always

iViewRC iView_SetDeviceParameters (iViewTicket *const *ticket,* iViewDeviceParameters const *const *params*)

Set operation mode to use the DeviceType specified in DeviceParameters and provide device parameters to the iViewNG server.

Depending on the device type, a iViewResult may be submitted synchronously or asynchronously.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.

This function produces asynchronously delivered iViewResult data and calls the user callback function to provide the result of the operation through the ticket:

- if iViewTicket.returnCode is RC_NO_ERROR:
  - iViewTicket.result is NULL

iViewTicket.state is IVIEWTICKETSTATUS_CLOSED

- if iViewTicket.returnCode indicated an error:
  - iViewTicket.result is NULL or a pointer to iViewServerError
  - iViewTicket.state remains at the former state

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.
`in`	*params*	The parameter set, depending on the device. The data structure must be allocated by the user and may be released immediately after calling this function.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_PARAMETER	if any parameter is invalid
RC_UNSPECIFIED	on any other error

Variable Documentation

const iViewAudioActivationState IVIEWAUDIOSTATE_ACTIVATED = 1 [static]

Use audio.

const iViewAudioActivationState IVIEWAUDIOSTATE_CURRENT = 0 [static]

Neutral setting. Leave the audio activation state as it is.

const iViewAudioActivationState IVIEWAUDIOSTATE_DEACTIVATED = 2 [static]

Do not use audio.

const iViewDeviceType IVIEWDEVICE_INVALID = 0x00 [static]

Invalid setting.

const iViewDeviceType IVIEWDEVICE_UNKNOWN = 0x01 [static]

Unknown device.

const iViewCameraResolution IVIEWRESOLUTION_CURRENT = 0 [static]

Do not change the resolution / resolution unspecified.

const iViewSamplingRate IVIEWSAMPLINGRATE_CURRENT = 0 [static]

Do not change the sampling rate / sampling rate unspecified.

const iViewTrackingMode IVIEWTRACKINGMODE_CURRENT = 0 [static]

Do not change the tracking mode / tracking mode unspecified.

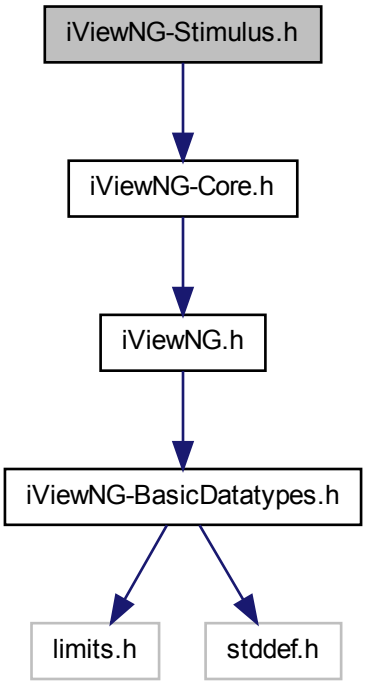
const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_CURRENT = 0

Do not change the sampling rate / sampling rate unspecified.

7.10 iViewNG-Stimulus.h File Reference

`#include <iViewNG-Core.h>`

Include dependency graph for iViewNG-Stimulus.h:



Detailed Description

| \ | | ___ | |_ ___ _ | | |/ _ | __/ _ \ (_) | |\ | (_) | || __/ _ |_| _|___/ _____| (_)

Unlike most of the other header files, this file is NOT part of the iViewNG-SDK product release!

This file declares the iViewNG API functions and data types which are used to manage and set up the stimulus. Whether a stimulus is supported and needed is device dependent. So these declarations are only generic. The actual structures are declared in the device specific header file.

A stimulus is the "area" to which the gaze is mapped. The physical setup of the stimulus relative to the eye tracker is part of the "StimulusParameters". So if the stimulus is a Monitor and the Monitor is rotated, the stimulus has to be adjusted. Either by changing the existing stimulus or by creating a new one.

The resolution of the stimulus is not part of the StimulusParameters! It can be changed independently.

A modification of the stimulus may introduce the need of a re-calibration whereas the resolution can be changed at any time and does not affect the calibration.

The stimulus resolution should be set after the stimulus Parameters.

Data Structure Documentation

struct iViewStimulusParameters

This structure holds a full setup of a stimulus. The exact structure is device dependent.

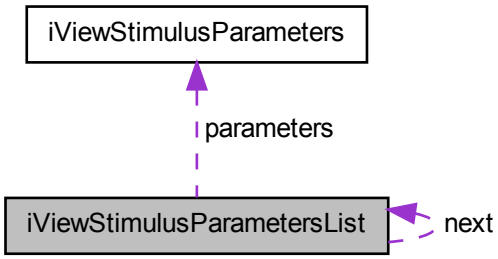
Data Fields

void *	parameters	Stimulus type dependent data structure. Cast this according to the type, for details see the file iViewNG-Device-*.h.
iViewStimulus-Type	type	The stimulus type to use.

struct iViewStimulusParametersList

This structure holds a linked list of iViewStimulusParameters and a name. It is returned by iView_Load-StimulusParameters and represents the directory of available StimulusParameters. This structure is also used with iView_SaveStimulusParameters.

Collaboration diagram for iViewStimulusParametersList:



Data Fields

wchar_t	name	
iViewStimulus-ParametersList *	next	
iViewStimulus-Parameters *	parameters	the name for this parameters

struct iViewStimulusResolution

This structure hold the stimulus resolution information. The exact structure is device dependent and declared in the specific header file.

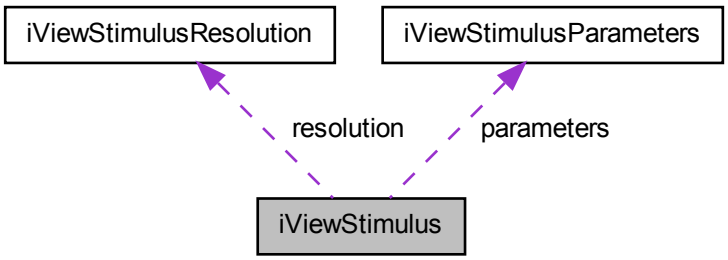
Data Fields

void *	parameters	Stimulus resolution type dependent data structure. Cast this according to the type, for details see the file iViewNG-Device-*.h.
iViewStimulus-ResolutionType	type	

struct iViewStimulus

This structure holds the name of the Stimulus Parameters and Resolution. This structure is passed to the iView_SetActiveStimulus and returned by the iView_GetActiveStimulus

Collaboration diagram for iViewStimulus:



Data Fields

int32_t	applyAtFrame	Only meaningful for offline use The scene frame index at which the stimulus shall be applied. Use a value of -1 if the frame index is not known / not required and the server shall guess the proper value.
wchar_t *	name	The name of the StimulusParameters When calling the iView_SetActiveStimulus this parameter can be NULL to indicate that the Stimulus should not be changed.
iViewStimulus-Parameters *	parameters	the actual StimulusParameters this pointer must be NULL when calling iView_SetActiveStimulus

iViewStimulus-Resolution *	resolution	The resolution to set When calling the iView_SetActiveStimulus this parameter can be NULL to indicate that the resolution should not be changed.

Typedef Documentation

typedef uint32_t iViewStimulusResolutionType

Identifier for different stimulus resolution types.

typedef uint32_t iViewStimulusType

Identifier for different stimulus types.

Function Documentation

iViewRC iView_GetActiveStimulus (iViewTicket * *ticket*)

Get the name of the active stimulus parameters and the current resolution

This function does not change anything it just retrieves the name of the currently active stimulus parameters.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewResult.type is IVIEWRESULT_GET_ACTIVE_STIMULUS,
  - iViewResult.data is a pointer to iViewStimulus,
  - iViewResult.next is NULL since there will be no subsequent results. contains only the currently active StimulusParameters

- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_LoadStimulusParameters (iViewTicket * *ticket*)

Load all available sets of stimulus parameters and return them as a linked list.

This function does not affect the calibration or gaze mapping.

The information is stored on the server side.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewResult.type is IVIEWRESULT_LOAD_STIMULUS_PARAMETERS,
  - iViewResult.data is a pointer to iViewStimulusParametersList,
  - iViewResult.next is NULL since there will be no subsequent results.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

When successful, all gaze data will be mapped to the new resolution.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_-ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAI-LED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_PARAME-TER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_SaveStimulusParameters (iViewTicket * *ticket*, iViewStimulusParametersList const *const parameterlist*)

Save one or more sets of stimulus parameters from a linked list.

This function does not affect the calibration or gaze mapping. Even if the currently active stimulus parameters are changed (and saved) the parameters have to be activated via iView_SetActiveStimulus.

Only those sets of stimulus parameters that are in the list are altered so it is not needed to call this function will a full list of all sets and have only one stimulus parameter changed.

To delete an entry, specify the name and set the "parameter" member to NULL.

To rename an entry, create a new one and delete the old one.

Trying to delete the currently active stimulus parameters will result in RC_INVALID_PARAMETER.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

When successful, all gaze data will be mapped to the new resolution.

Parameters

in,out	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
in	*parameterlist*	Pointer to the structure holding the first iViewStimulusParametersList.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_SetActiveStimulus (iViewTicket * *ticket*, iViewStimulus const *const *stimulus*)

Select the active Stimulus Parameters and/or the Stimulus Resolution

This function is used to select a previously saved set of stimulus parameters. The server store the active stimulus parameter and automatically activate it when starting up, so it is not essentially required to to call this function.

Changing the stimulus parameters can cause the calibration to become invalid. (tbd)

By omitting the name of the Stimulus Parameters it is possible to only change the resolution which does only affect the mapping and does not alter the calibration.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns synchronously and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

`in,out`	*ticket*	The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
`in`	*stimulus*	Pointer to the name of the stimulus parameters to load.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

Variable Documentation

const iViewStimulusResolutionType IVIEWSTIMULUSRESOLUTIONTYPE_INVALID = 0 [static]

Invalid stimulus resolution type to detect unset data structure elements.

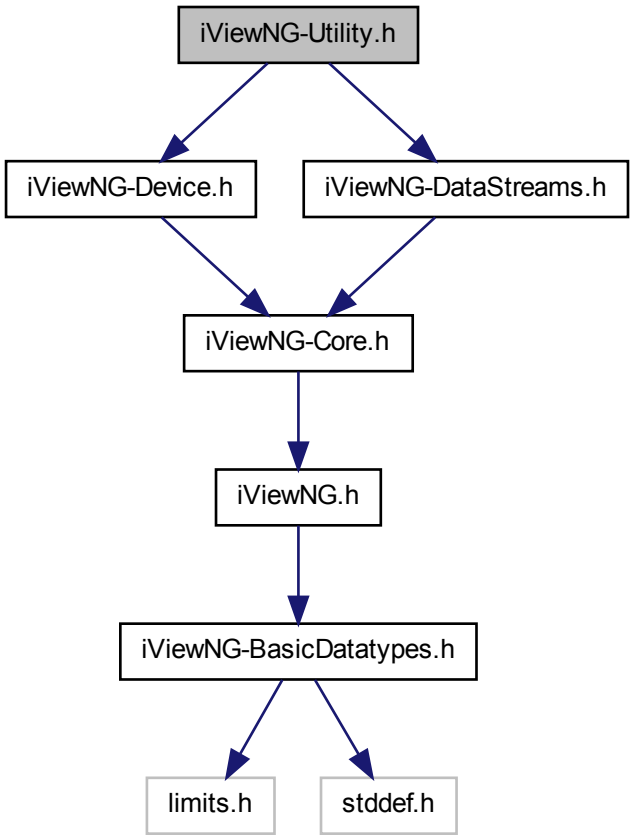
const iViewStimulusType IVIEWSTIMULUSTYPE_INVALID = 0 [static]

Invalid stimulus type to detect unset data structure elements.

7.11 iViewNG-Utility.h File Reference

```
#include <iViewNG-Device.h>
#include <iViewNG-DataStreams.h>
```

Include dependency graph for iViewNG-Utility.h:



Detailed Description

This file declares some useful functions which may be useful while using the API.

Function Documentation

double iView_GetInfValue ( )

Return the value for +INF (positive infinity).

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Note

Please mind that it's not possible to compare INF values using the '==' operator! Use iView_IsInf() instead!

Return values

+INF	as the iViewNG library encodes it.

double iView_GetNanValue ( )

Return the value for NAN (not-a-number).

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Note

Please mind that it's not possible to compare NAN values using the '==' operator! Use iView_IsNan() instead!

Return values

NAN	as the iViewNG library encodes it.

char iView_IsInf (const double *num*)

Test if a given variable contains +INF.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*num*	the variable to test

Return values

0	if 'num' does not contain +INF
1	if 'num' contains +INF Test if a given variable contains INF.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*num*	the variable to test

Return values

0	if 'num' does not contain +/- INF
1	if 'num' contains +INF
-1	if 'num' contains -INF

char iView_IsNan (const double *num*)

Test if a given variable contains NAN.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*num*	the variable to test

Return values

0	if 'num' does not contain NAN
1	if 'num' contains NAN Test if a given variable contains NAN.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*num*	the variable to test

Return values

0	if 'num' does not contain NAN
1	if 'num' contains NAN

iViewRC iView_iViewDataStreamTypeToString (wchar_t * *dst*, size_t * *maxSize*, const iViewDataStreamType *type*)

Print the iViewDataStreamType enum to a char array. Useful for analyzing parameters manually.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in,out	*dst*	Address of pointer to hold the result. The result is allocated with malloc() and must be released by the caller. The result pointer must contain NULL.
in,out	*maxSize*	on call: size of the dst allocation in bytes on return: bytes of the dst allocation used or bytes needed
in	*type*	Parameter to convert.

Return values

RC_NO_ERROR	on success. maxSize contains the used bytes of dst (incl. the terminating 0-char)
RC_INVALID_PARAMETER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	dst ist not large enough to store the result. maxSize contains the minimum required size in bytes. (incl. the terminating 0-char)
RC_UNSPECIFIED	on any other error

iViewRC iView_iViewDeviceParametersToString (wchar_t * *dst*, size_t * *maxSize*, iViewDeviceParameters const *const params*)

Print the DeviceParameters parameter block contents to a char array. Useful for analyzing parameters manually.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in,out	*dst*	Address of pointer to hold the result. The result is allocated with malloc() and must be released by the caller. The result pointer must contain NULL.
in,out	*maxSize*	on call: size of the dst allocation in bytes on return: bytes of the dst allocation used or bytes needed
in	*params*	Parameter block to convert.

Return values

RC_NO_ERROR	on success. maxSize contains the used bytes of dst (incl. the terminating 0-char)
RC_INVALID_PARAMETER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	dst ist not large enough to store the result. maxSize contains the minimum required size in bytes. (incl. the terminating 0-char)
RC_UNSPECIFIED	on any other error

iViewRC iView_iViewRcToString (wchar_t * *dst*, size_t * *maxSize*, const iViewRC *rc*)

Get a human-readable string for a given return code.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in,out	*dst*	Memory where to store the Result. If this parameter is NULL or
in,out	*maxSize*	on call: size of the dst allocation in bytes on return: bytes of the dst allocation used or bytes needed
in	*rc*	The return code to convert.

Return values

RC_NO_ERROR	on success. maxSize contains the used bytes of dst (incl. the terminating 0-char)
RC_OUT_OF_MEMORY	dst ist not large enough to store the result. maxSize contains the minimum required size in bytes. (incl. the terminating 0-char)
RC_UNSPECIFIED	on any other error

iViewRC iView_iViewTicketToString (wchar_t * *dst*, size_t * *maxSize*, iViewTicket const *const *ticket*)

Print the ticket contents to a char array. Useful for analyzing tickets manually.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in,out	*dst*	Address of pointer to hold the result. The result is allocated with malloc() and must be released by the caller. The result pointer must contain NULL.
in,out	*maxSize*	on call: size of the dst allocation in bytes on return: bytes of the dst allocation used or bytes needed
in	*ticket*	Ticket to convert.

Return values

RC_NO_ERROR	on success. maxSize contains the used bytes of dst (incl. the terminating 0-char)
RC_INVALID_PARAMETER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	dst ist not large enough to store the result. maxSize contains the minimum required size in bytes. (incl. the terminating 0-char)
RC_UNSPECIFIED	on any other error

void iView_Sleep (const uint32_t *milliSeconds*)

Put the process into idle mode for at least / approximately the specified amount of time. The purpose of this function is to provide a platform-independent sleep function. The precision of sleeping exactly the specified amount of time depends on the used operating system.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

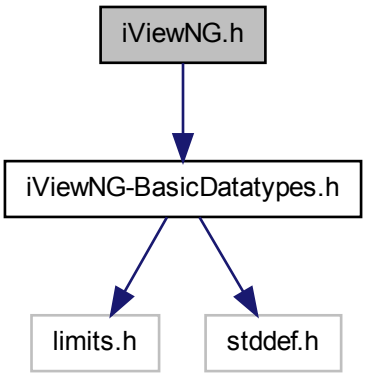
Parameters

in	*milliSeconds*	the amount of time to wait

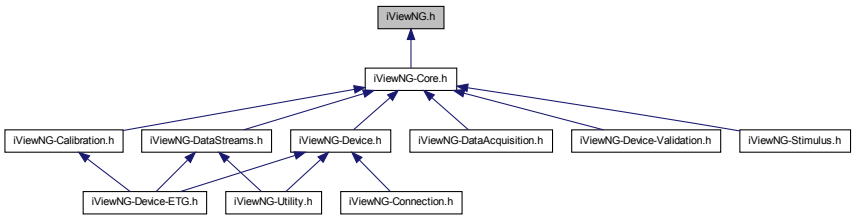
7.12 iViewNG.h File Reference

```
#include "iViewNG-BasicDatatypes.h"
```

Include dependency graph for iViewNG.h:



This graph shows which files directly or indirectly include this file:



Detailed Description

This file declares several C preprocessor macros which are used by the other include files. There is no need to include this file directly, it is automatically included where needed.

Macro Definition Documentation

#define DLL_DEC_POST

#define DLL_DEC_PRE extern "C"

#define DLL_DEFINITION

#define DLLCALL

Typedef Documentation

typedef uint32_t iViewRC

The return code for submitting the immediate result of a function call. This return code does not always indicate the result of the requested operation, but only the call to the function. Operation results are always returned by call-by-reference parameters like the iViewTicket data structure, synchronously and asynchronously.

Variable Documentation

const iViewRC NUMBER_OF_RETURNCODES = 13 [static]

Besides RC_UNSPECIFIED, this is always the highest return code so that it's possible to loop over all return codes.

const iViewRC RC_CONNECTION_FAILED = 8 [static]

The operation failed due to the lack of a connection to the server.

const iViewRC RC_DEVICE_BUSY = 11 [static]

The API requested a command which needs to access an (e.g. ETG) device which is already performing an operation.

const iViewRC RC_DEVICE_NOT_READY = 10 [static]

The API requested a command which needs to access an (e.g. ETG) device which is not yet ready (e.g. still initializing).

const iViewRC RC_INTERNAL_ERROR = 2 [static]

The function call could not be processed due to a serious internal error. Please contact SMI.

const iViewRC RC_INVALID_LICENSE = 12 [static]

The API request failed because the currently activated license(s) do not permit this operation.

const iViewRC RC_INVALID_PARAMETER = 5 [static]

The provided parameters were not valid.

const iViewRC RC_INVALID_REQUEST = 7 [static]

The API request is invalid, e.g. not supported.

const iViewRC RC_INVALID_STATE = 6 [static]

The server is not in a valid state for the operation.

const iViewRC RC_INVALID_TICKET = 4 [static]

The provided iViewTicket is invalid (most likely, it's NULL, the result pointer is NULL or the ticket state is invalid).

const iViewRC RC_NO_ERROR = 0 [static]

No error, the function call was accepted.

const iViewRC RC_NO_INIT = 1 [static]

iView_Init() was not called!

const iViewRC RC_OPERATION_FAILED = 9 [static]

The requested operation failed.

const iViewRC RC_OUT_OF_MEMORY = 3 [static]

The function call failed due to insufficient memory.

const iViewRC RC_TIMEOUT = 13 [static]

An API operation was not answered in time.

const iViewRC RC_UNSPECIFIED = UINT_MAX [static]

Unspecified error, should never happen.

7.13 D:/Research/iViewNG/source/iViewNG-SDK/documentation/iViewNG-SDK--Documentation.dox File Reference

Index

__iViewDeviceInfoStruct, 141
__iViewHostListStruct, 73
__iViewLicenseStruct, 141
__iViewResultStruct, 79

DLL_DEC_POST
    iViewNG.h, 162
DLL_DEC_PRE
    iViewNG.h, 162
DLL_DEFINITION
    iViewNG.h, 162
DLLCALL
    iViewNG.h, 162

EYE_BINOCULAR
    iViewNG-DataStreams.h, 122
EYE_LEFT
    iViewNG-DataStreams.h, 122
EYE_RIGHT
    iViewNG-DataStreams.h, 122
EYE_UNKNOWN
    iViewNG-DataStreams.h, 122

iViewCalibrationData, 62
iViewCalibrationData1Point, 63
iViewCalibrationParameters, 62
iViewCalibrationParametersEtg, 132
iViewDataStream, 116
iViewDataStreamEndOfStream, 115
iViewDataStreamEtgServerStatus, 130
iViewDataStreamEyeImage, 106
iViewDataStreamGazeSample, 112
iViewDataStreamH264SceneImage, 106
iViewDataStreamSceneImage, 105
iViewDataStreamSpecGazeFilter, 114
iViewDataStreamSpecQualityOfService, 114
iViewDataStreamSpecSampleRate, 113
iViewDataStreamSpecStruct, 113
iViewDataStreamTrackingStatus, 108
iViewDate, 78
iViewDeviceParameters, 141
iViewDeviceParametersEtgCameraPlayback, 131
iViewDeviceVersionEtg, 130
iViewEyeSample, 110
iViewHost, 72
iViewHostAddress, 71
iViewHostAddress.ipAddress, 73
iViewImage, 104
iViewLicenseEtg, 130
iViewMessage, 95
iViewMessageText, 95
iViewOverlaySpec, 116
iViewOverlaySpecListStruct, 117
iViewSerialNumberEtg, 130
iViewServerError, 79
iViewStimulus, 149
iViewStimulusParameters, 148
iViewStimulusParametersList, 148
iViewStimulusResolution, 148
iViewStreamSubscription, 115
iViewTicket, 79
iViewTime, 78
iViewTrackingStatus, 107
iViewValidationElement, 137
iViewValidationFeedback, 137
iViewVector2D, 109
iViewVector3D, 110
iViewVersion, 78
IVIEW_CALLBACK
    iViewNG-Core.h, 80
IVIEW_DEPTH_16S
    iViewNG-DataStreams.h, 122
IVIEW_DEPTH_16U
    iViewNG-DataStreams.h, 122
IVIEW_DEPTH_1U
    iViewNG-DataStreams.h, 123
IVIEW_DEPTH_32F

iViewNG-DataStreams.h, 123
IVIEW_DEPTH_32S
iViewNG-DataStreams.h, 123
IVIEW_DEPTH_8S
iViewNG-DataStreams.h, 123
IVIEW_DEPTH_8U
iViewNG-DataStreams.h, 123
IVIEW_DEPTH_SIGN
iViewNG-DataStreams.h, 123
IVIEWDEVICE_ETG
iViewNG-Device-ETG.h, 133
IVIEWQOS_LOSSLESS
iViewNG-DataStreams.h, 128
IVIEWQOS_LOSSY
iViewNG-DataStreams.h, 128
IVIEWQOS_UNKNOWN
iViewNG-DataStreams.h, 128
IVIEWRESULT_ERROR
iViewNG-Core.h, 91
IVIEWSDK_INVALID
iViewNG-Core.h, 93
IVIEWSDK_IVNG
iViewNG-Core.h, 93
iView_AddLicenseKey
iViewNG-Core.h, 81
iView_CancelCalibration
iViewNG-Calibration.h, 64
iView_Connect
iViewNG-Connection.h, 74
iView_CreateTicket
iViewNG-Core.h, 83
iView_Disconnect
iViewNG-Connection.h, 75
iView_GetActiveStimulus
iViewNG-Stimulus.h, 150
iView_GetCallback
iViewNG-Core.h, 83
iView_GetDeviceInfo
iViewNG-Device.h, 142
iView_GetDeviceParameters
iViewNG-Device.h, 143
iView_GetInfValue
iViewNG-Utility.h, 156
iView_GetLibraryVersion
iViewNG-Core.h, 83
iView_GetLocalTime
iViewNG-Core.h, 84
iView_GetNanValue
iViewNG-Utility.h, 157
iView_GetServerTime
iViewNG-Core.h, 85
iView_GetValidation
iViewNG-Device-Validation.h, 138
iView_Init
iViewNG-Core.h, 86
iView_IsInf
iViewNG-Utility.h, 157
iView_IsNan
iViewNG-Utility.h, 158
iView_LoadStimulusParameters
iViewNG-Stimulus.h, 151
iView_ReleaseResult
iViewNG-Core.h, 87
iView_ReleaseTicket
iViewNG-Core.h, 87
iView_SaveMessage
iViewNG-DataAcquisition.h, 96
iView_SaveStimulusParameters
iViewNG-Stimulus.h, 152
iView_SetActiveStimulus
iViewNG-Stimulus.h, 154
iView_SetCalibrationData
iViewNG-Calibration.h, 65
iView_SetCallback
iViewNG-Core.h, 88
iView_SetDeviceParameters
iViewNG-Device.h, 144
iView_SetupCalibration
iViewNG-Calibration.h, 67
iView_Shutdown
iViewNG-Core.h, 88
iView_ShutdownServer
iViewNG-Core.h, 88
iView_Sleep
iViewNG-Utility.h, 161
iView_StartCalibration
iViewNG-Calibration.h, 68
iView_StartDataAcquisition
iViewNG-DataAcquisition.h, 97
iView_StartRecording

iViewNG-DataAcquisition.h, 99
iView_StartServer
iViewNG-Core.h, 89
iView_StopDataAcquisition
iViewNG-DataAcquisition.h, 100
iView_StopRecording
iViewNG-DataAcquisition.h, 101
iView_SubscribeDataStream
iViewNG-DataStreams.h, 119
iView_UnsubscribeDataStream
iViewNG-DataStreams.h, 121
iView_iViewDataStreamTypeToString
iViewNG-Utility.h, 158
iView_iViewDeviceParametersToString
iViewNG-Utility.h, 159
iView_iViewRcToString
iViewNG-Utility.h, 160
iView_iViewTicketToString
iViewNG-Utility.h, 160
iViewAudioActivationState
iViewNG-Device.h, 142
iViewCalibrationDataType
iViewNG-Calibration.h, 64
iViewCalibrationPointId
iViewNG-Calibration.h, 64
iViewCalibrationPointUsage
iViewNG-Calibration.h, 64
iViewCalibrationType
iViewNG-Calibration.h, 64
iViewCalibrationUnit
iViewNG-Calibration.h, 64
iViewCameraResolution
iViewNG-Device.h, 142
iViewConnectionType
iViewNG-Connection.h, 74
iViewDataStreamSpecGazeFilterType
iViewNG-DataStreams.h, 117
iViewDataStreamSpecType
iViewNG-DataStreams.h, 117
iViewDataStreamState
iViewNG-DataStreams.h, 117
iViewDataStreamType
iViewNG-DataStreams.h, 118
iViewDeviceType
iViewNG-Device.h, 142
iViewEyeIdentifier
iViewNG-DataStreams.h, 119
iViewLocationScope
iViewNG-Connection.h, 74
iViewMessageType
iViewNG-DataAcquisition.h, 96
iViewNG-BasicDatatypes.h, 59
int16_t, 60
int32_t, 60
int64_t, 60
int8_t, 60
UINT_MAX, 60
uint16_t, 60
uint32_t, 60
uint64_t, 60
uint8_t, 60
iViewNG-Calibration.h, 61
iView_CancelCalibration, 64
iView_SetCalibrationData, 65
iView_SetupCalibration, 67
iView_StartCalibration, 68
iViewCalibrationDataType, 64
iViewCalibrationPointId, 64
iViewCalibrationPointUsage, 64
iViewCalibrationType, 64
iViewCalibrationUnit, 64
iViewNG-Connection.h, 71
iView_Connect, 74
iView_Disconnect, 75
iViewConnectionType, 74
iViewLocationScope, 74
iViewNG-Core.h, 77
IVIEW_CALLBACK, 80
IVIEWSDK_INVALID, 93
IVIEWSDK_IVNG, 93
iView_AddLicenseKey, 81
iView_CreateTicket, 83
iView_GetCallback, 83
iView_GetLibraryVersion, 83
iView_GetLocalTime, 84
iView_GetServerTime, 85
iView_Init, 86
iView_ReleaseResult, 87
iView_ReleaseTicket, 87
iView_SetCallback, 88

iView_Shutdown, 88
iView_ShutdownServer, 88
iView_StartServer, 89
iViewResultType, 81
iViewSdkType, 81
iViewTicketNumber, 81
iViewTicketProgress, 81
iViewTicketState, 81
iViewNG-DataAcquisition.h, 95
iView_SaveMessage, 96
iView_StartDataAcquisition, 97
iView_StartRecording, 99
iView_StopDataAcquisition, 100
iView_StopRecording, 101
iViewMessageType, 96
iViewNG-DataStreams.h, 103
EYE_BINOCULAR, 122
EYE_LEFT, 122
EYE_RIGHT, 122
EYE_UNKNOWN, 122
IVIEW_DEPTH_16S, 122
IVIEW_DEPTH_16U, 122
IVIEW_DEPTH_1U, 123
IVIEW_DEPTH_32F, 123
IVIEW_DEPTH_32S, 123
IVIEW_DEPTH_8S, 123
IVIEW_DEPTH_8U, 123
IVIEWQOS_LOSSY, 128
iView_SubscribeDataStream, 119
iView_UnsubscribeDataStream, 121
iViewDataStreamSpecGazeFilterType, 117
iViewDataStreamSpecType, 117
iViewDataStreamState, 117
iViewDataStreamType, 118
iViewEyeIdentifier, 119
iViewOverlayOption, 119
iViewOverlayPosition, 119
iViewOverlaySpecList, 119
iViewOverlayType, 119
iViewQualityOfService, 119
iViewNG-Device-ETG.h, 128
iViewNG-Device-Validation.h, 136
iView_GetValidation, 138
iViewValidationElementType, 138
iViewNG-Device.h, 140
iView_GetDeviceInfo, 142
iView_GetDeviceParameters, 143
iView_SetDeviceParameters, 144
iViewAudioActivationState, 142
iViewCameraResolution, 142
iViewDeviceType, 142
iViewSamplingRate, 142
iViewTrackingMode, 142
iViewWhiteBalanceProgram, 142
iViewNG-Stimulus.h, 147
iView_GetActiveStimulus, 150
iView_LoadStimulusParameters, 151
iView_SaveStimulusParameters, 152
iView_SetActiveStimulus, 154
iViewStimulusResolutionType, 150
iViewStimulusType, 150
iViewNG-Utility.h, 156
iView_GetInfValue, 156
iView_GetNanValue, 157
iView_IsInf, 157
iView_IsNan, 158
iView_Sleep, 161
iView_iViewDataStreamTypeToString, 158
iView_iViewDeviceParametersToString, 159
iView_iViewRcToString, 160
iView_iViewTicketToString, 160
iViewNG.h, 161
DLL_DEC_POST, 162
DLL_DEC_PRE, 162
DLL_DEFINITION, 162
DLLCALL, 162
iViewRC, 162
RC_DEVICE_BUSY, 162
RC_INVALID_STATE, 163
RC_NO_ERROR, 163
RC_NO_INIT, 163
RC_TIMEOUT, 163
RC_UNSPECIFIED, 164
iViewOverlayOption
iViewNG-DataStreams.h, 119
iViewOverlayPosition
iViewNG-DataStreams.h, 119
iViewOverlaySpecList
iViewNG-DataStreams.h, 119
iViewOverlayType

iViewNG-DataStreams.h, 119
iViewQualityOfService
iViewNG-DataStreams.h, 119
iViewRC
iViewNG.h, 162
iViewResultType
iViewNG-Core.h, 81
iViewSamplingRate
iViewNG-Device.h, 142
iViewSdkType
iViewNG-Core.h, 81
iViewStimulusResolutionType
iViewNG-Stimulus.h, 150
iViewStimulusType
iViewNG-Stimulus.h, 150
iViewTicketNumber
iViewNG-Core.h, 81
iViewTicketProgress
iViewNG-Core.h, 81
iViewTicketState
iViewNG-Core.h, 81
iViewTrackingMode
iViewNG-Device.h, 142
iViewValidationElementType
iViewNG-Device-Validation.h, 138
iViewWhiteBalanceProgram
iViewNG-Device.h, 142
int16_t
iViewNG-BasicDatatypes.h, 60
int32_t
iViewNG-BasicDatatypes.h, 60
int64_t
iViewNG-BasicDatatypes.h, 60
int8_t
iViewNG-BasicDatatypes.h, 60

RC_DEVICE_BUSY
iViewNG.h, 162
RC_INTERNAL_ERROR
iViewNG.h, 162
RC_INVALID_LICENSE
iViewNG.h, 163
RC_INVALID_REQUEST
iViewNG.h, 163
RC_INVALID_STATE
iViewNG.h, 163
RC_INVALID_TICKET
iViewNG.h, 163
RC_NO_ERROR
iViewNG.h, 163
RC_NO_INIT
iViewNG.h, 163
RC_OUT_OF_MEMORY
iViewNG.h, 163
RC_TIMEOUT
iViewNG.h, 163
RC_UNSPECIFIED
iViewNG.h, 164

UINT_MAX
iViewNG-BasicDatatypes.h, 60
uint16_t
iViewNG-BasicDatatypes.h, 60
uint32_t
iViewNG-BasicDatatypes.h, 60
uint64_t
iViewNG-BasicDatatypes.h, 60
uint8_t
iViewNG-BasicDatatypes.h, 60